

JAVA 17 OCP · YDS STUDY SERIES

Unit 12 · Modules · Bilingual Notes

Teknik İngilizceyi bağlam içinde öğren · kısa tekrarlarla kalıcılaştır

Bu ana kaynak, [OCP_Java_SE17_Chapter1den_Itibaren.pdf](#) içindeki ilgili chapter gövdesini ve Appendix resmî cevaplarını kaynak sırasını koruyan English → Türkçe paragraf çiftleriyle bir araya getirir. Kod ve terminal çıktıları çevrilmeden, bir kez ve kaynak konumunda gösterilir.

[Vocabulary](#) · [Grammar notes](#) · [Teknik hafıza notu](#)

Kaynak kapsam manifesti

- Kaynak: [exam_lecture/OCP_Java_SE17_Chapter1den_Itibaren.pdf](#)
- Chapter: 12 · Modules
- Chapter PDF sayfaları: 661–720
- Appendix cevap sayfaları: 949–951
- Beklenen sayfa marker'ı: 60
- Beklenen resmî cevap: 25
- Eşleme biçimi: English paragraf → Türkçe çeviri → varsa kod

İçindekiler

- 1 [Introducing Modules](#)
- 2 [Creating and Running a Modular Program](#)
- 3 [Updating Our Example for Multiple Modules](#)
- 4 [Diving into the Module Declaration](#)
- 5 [Creating a Service](#)
- 6 [Discovering Modules](#)
- 7 [Comparing Types of Modules](#)
- 8 [Migrating an Application](#)
- 9 [Summary](#)
- 10 [Exam Essentials](#)
- 11 [Review Questions](#)
- 12 [Official Review Question Answers / Resmî Cevaplar](#)

Chapter 12 · Modules · Eksiksiz çift dilli kaynak

Chapter 12 · Modules

English: OCP exam objectives covered in this chapter: Packaging and deploying Java code and using the Java Platform Module System.

Türkçe: Bu bölümde ele alınan OCP sınav hedefleri şunlardır: Java kodunu paketlemek ve dağıtmak; Java Platform Module System'ı (Java Platform Modül Sistemi) kullanmak.

English: Define modules and their dependencies, and expose module content, including content used through reflection. Define services, producers, and consumers.

Türkçe: Modülleri ve bağımlılıklarını tanımlayın; reflection (yansıma) aracılığıyla kullanılan içerik dâhil olmak üzere modül içeriğini dışa açın. Hizmetleri, üreticileri ve tüketicileri tanımlayın.

English: Compile Java code; produce modular and non-modular JARs and runtime images; and implement migration using unnamed and automatic modules.

Türkçe: Java kodunu derleyin; modüler ve modüler olmayan JAR'lar ile çalışma zamanı görüntüleri (runtime images) üretin; adsız (unnamed) ve otomatik (automatic) modülleri kullanarak geçişi (migration) gerçekleştirin.

English: Packages can be grouped into modules. In this chapter, we explain the purpose of modules and how to build your own.

Türkçe: Paketler modüller hâlinde gruplanabilir. Bu bölümde modüllerin amacını ve kendi modüllerinizi nasıl oluşturacağınızı açıklıyoruz.

English: We also show how to run them and how to discover existing modules. Next, we cover strategies for migrating an application to use modules, running a partially modularized application, and dealing with dependencies. We then move on to discuss services and service locators. Finally, we show how to create a runtime image.

Türkçe: Ayrıca bu modüllerin nasıl çalıştırılacağını ve mevcut modüllerin nasıl keşfedileceğini gösteriyoruz. Ardından bir uygulamayı modülleri kullanacak şekilde geçirme, kısmen modülerleştirilmiş bir uygulamayı çalıştırma ve bağımlılıkları yönetme stratejilerini ele alıyoruz. Sonra hizmetleri ve hizmet bulucuları (service locators) inceliyor, son olarak da bir çalışma zamanı görüntüsünün (runtime image) nasıl oluşturulacağını gösteriyoruz.

English: We've made the code in this chapter available online. Since it can be tedious to create the directory structure, this will save you some time. Additionally, the commands need to be exactly right, so we've included those online so you can copy and paste them and compare them with what you typed.

Both are available in our GitHub repo, linked to from www.selikoff.net/ocp-17/

Türkçe: Bu bölümdeki kodu çevrimiçi olarak erişime açtık. Dizin yapısını oluşturmak zahmetli olabileceğinden bu size zaman kazandırır. Komutların da tamamen doğru olması gerekir; bu nedenle onları çevrimiçi olarak sunduk. Böylece komutları kopyalayıp yapıştırabilir ve kendi yazdıklarınızla karşılaştırabilirsiniz. Kod ile komutların ikisi de www.selikoff.net/ocp-17/ adresinden bağlantı verilen GitHub depomuzda bulunur.

Introducing Modules

English: When writing code for the exam, you generally see small classes. After all, exam questions have to fit on a single screen! When you work on real programs, they are much bigger. A real project will consist of hundreds or thousands of classes grouped into packages. These packages are grouped into Java archive (JAR) files. A JAR is a ZIP file with some extra information, and the extension is `.jar`.

Türkçe: Sınav kodlarında genellikle küçük sınıflar görürsünüz; sonuçta sınav sorularının tek ekrana sığması gerekir. Gerçek programlar ise çok daha büyüktür. Gerçek bir proje, paketler hâlinde gruplanmış yüzlerce veya binlerce sınıftan oluşur. Bu paketler Java arşivi (JAR) dosyaları içinde gruplanır. JAR, ek bilgiler içeren bir ZIP dosyasıdır ve `.jar` uzantısına sahiptir.

English: In addition to code written by your team, most applications also use code written by others. Open source is software with the code supplied and is often free to use. Java has a vibrant open source software (OSS) community, and those libraries are also supplied as JAR files. For example, there are libraries to read files, connect to a database, and much more.

Türkçe: Ekibinizin yazdığı koda ek olarak çoğu uygulama, başkalarının yazdığı kodu da kullanır. Açık kaynak yazılım (open source software), kaynak kodu sunulan ve çoğu zaman ücretsiz kullanılabilen yazılımdır. Java'nın canlı ve etkin bir açık kaynak yazılım (OSS) topluluğu vardır; bu kütüphaneler de JAR dosyaları olarak sunulur. Örneğin dosya okumak, veri tabanına bağlanmak ve daha pek çok iş için kütüphaneler bulunur.

English: Some open source projects even depend on functionality in other open source projects. For example, Spring is a commonly used framework, and JUnit is a commonly used testing library. To use either, you need to make sure you have compatible versions of all the relevant JARs available at runtime. This complex chain of dependencies and minimum versions is often referred to by the community as JAR hell. Hell is an excellent way of describing the wrong version of a class being loaded or even a `ClassNotFoundException` at runtime.

Türkçe: Bazı açık kaynak projeler, başka açık kaynak projelerdeki işlevlere de bağımlıdır. Örneğin Spring yaygın kullanılan bir framework (uygulama çatısı), JUnit ise yaygın kullanılan bir test kütüphanesidir. Bunlardan herhangi birini kullanmak için ilgili bütün JAR'ların uyumlu sürümlerinin çalışma zamanında mevcut olduğundan emin olmanız gerekir. Bu karmaşık bağımlılık zinciri ve asgari sürüm gereksinimleri topluluk içinde çoğunlukla JAR hell (JAR cehennemi) olarak adlandırılır. Yanlış bir sınıf sürümünün yüklenmesini, hatta çalışma zamanında `ClassNotFoundException` oluşmasını anlatmak için “cehennem” oldukça uygun bir sözcüktür.

English: The Java Platform Module System (JPMS) groups code at a higher level. The main purpose of a module is to provide groups of related packages that offer developers a

Türkçe: Java Platform Module System (JPMS), kodu daha üst düzeyde gruplandırır. Bir modülün temel amacı, birbiriyle ilişkili paketleri bir araya getirmektir.

English: particular set of functionality. It's like a JAR file, except a developer chooses which packages are accessible outside the module. Let's look at what modules are and what problems they are designed to solve.

Türkçe: Bu paket grubu geliştiricilere belirli bir işlevsellik kümesi sunar. Modül, JAR dosyasına benzer; ancak geliştirici modül dışından hangi paketlere erişilebileceğini seçer. Şimdi modüllerin ne olduğuna ve hangi sorunları çözmek üzere tasarlandığına bakalım.

English: The Java Platform Module System includes the following: ■■ A format for module JAR files ■■ Partitioning of the JDK into modules ■■ Additional command-line options for Java tools

Türkçe: Java Platform Module System şunları içerir: modüler JAR dosyaları için bir biçim; JDK'nın modüllere ayrılması; Java araçları için ek komut satırı seçenekleri.

Exploring a Module

English: In Chapter 1, “Building Blocks,” we had a small Zoo application. It had only one class and just printed out one thing. Now imagine that we had a whole staff of programmers and were automating the operations of the zoo. Many things need to be coded, including the interactions with the animals, visitors, the public website, and outreach.

Türkçe: “Building Blocks” (Yapı Taşları) başlıklı 1. Bölüm’de küçük bir hayvanat bahçesi uygulamamız vardı. Yalnızca bir sınıf içeriyor ve tek bir şey yazdırıyordu. Şimdi bütün bir programcı ekibimiz olduğunu ve hayvanat bahçesinin işleyişini otomatikleştirdiğimizi düşünün. Hayvanlarla ve ziyaretçilerle etkileşimler, halka açık web sitesi ve topluma yönelik tanıtım çalışmaları dâhil pek çok şeyin kodlanması gerekir.

English: A module is a group of one or more packages plus a special file called `module-info.java`. The contents of this file are the module declaration. Figure 12.1 lists just a few of the modules a zoo might need. We decided to focus on the animal interactions in our example. The full zoo could easily have a dozen modules. In Figure 12.1, notice that there are arrows between many of the modules. These represent dependencies, where one module relies on code in another. The staff needs to feed the animals to keep their jobs. The line from `zoo.staff` to `zoo.animal.feeding` shows that the former depends on the latter.

Türkçe: Modül, bir veya daha fazla paket ile `module-info.java` adlı özel bir dosyadan oluşan gruptur. Bu dosyanın içeriğine module declaration (modül bildirimi) denir. Şekil 12.1, bir hayvanat bahçesinin ihtiyaç duyabileceği modüllerden yalnız birkaçını listeler. Örneğimizde hayvanlarla etkileşime odaklanıyoruz; hayvanat bahçesinin bütün sistemi kolaylıkla bir düzine modül içerebilir. Şekil 12.1’de modüller arasındaki oklar bağımlılıkları, yani bir modülün başka bir modüldeki koda dayanmasını gösterir. Personelin işini koruyabilmesi için hayvanları beslemesi gerekir. `zoo.staff` ile `zoo.animal.feeding` arasındaki çizgi, `zoo.staff` modülünün `zoo.animal.feeding` modülüne bağlı olduğunu gösterir.

English: FIGURE 12.1 — Design of a modular system. Dependency arrows: `zoo.animal.care` → `zoo.animal.feeding`; `zoo.animal.talks` → `zoo.animal.feeding`, `zoo.animal.care`; `zoo.staff` → `zoo.animal.feeding`, `zoo.animal.care`, `zoo.animal.talks`.

Türkçe: Şekil 12.1 — Modüler bir sistemin tasarımı. Dependency okları: `zoo.animal.care` → `zoo.animal.feeding`; `zoo.animal.talks` → `zoo.animal.feeding`, `zoo.animal.care`; `zoo.staff` → `zoo.animal.feeding`, `zoo.animal.care`, `zoo.animal.talks`.

English: Now let’s drill down into one of these modules. Figure 12.2 shows what is inside the `zoo.animal.talks` module. There are three packages with two classes each. (It’s a small zoo.) There is also a strange file called `module-info.java`. This file is required to be inside all modules. We explain this in more detail later in the chapter.

Türkçe: Şimdi bu modüllerden birini daha ayrıntılı inceleyelim. Şekil 12.2, `zoo.animal.talks` modülünün içeriğini gösterir. Her birinde iki sınıf bulunan üç paket vardır. (Burası küçük bir hayvanat bahçesi.) Ayrıca `module-info.java` adlı sıra dışı bir dosya bulunur. Bu dosyanın bütün modüllerde yer alması gerekir. Konuyu bölümün ilerleyen kısmında daha ayrıntılı açıklayacağız.

English: FIGURE 12.2 — Looking inside module `zoo.animal.talks`: `zoo.animal.talks.content` contains `ElephantScript.java` and `SeaLionScript.java`; `zoo.animal.talks.schedule` contains `Weekday.java` and `Weekend.java`; `zoo.animal.talks.media` contains `Signage.java` and `Announcement.java`; `module-info.java` is at the module root.

Türkçe: Şekil 12.2 — `zoo.animal.talks` modülünün iç yapısı: `zoo.animal.talks.content`, `ElephantScript.java` ile `SeaLionScript.java`yı; `zoo.animal.talks.schedule`, `Weekday.java` ile `Weekend.java`yı; `zoo.animal.talks.media`, `Signage.java` ile `Announcement.java`yı içerir; `module-info.java` modül root'undadır.

Benefits of Modules

English: Modules look like another layer of things you need to know in order to program. While using modules is optional, it is important to understand the problems they are designed to solve: ■■ Better access control: In addition to the levels of access control covered in Chapter 5, “Methods,” you can have packages that are only accessible to other packages in the module. ■■ Clearer dependency management: Since modules specify what they rely on, Java can complain about a missing JAR when starting up the program rather than when it is first accessed at runtime. ■■ Custom Java builds: You can create a Java runtime that has only the parts of the JDK that your program needs rather than the full one at over 150 MB. ■■ Improved security: Since you can omit parts of the JDK from your custom build, you don't have to worry about vulnerabilities discovered in a part you don't use. ■■ Improved performance: Another benefit of a smaller Java package is improved startup time and a lower memory requirement. ■■ Unique package enforcement: Since modules specify exposed packages, Java can ensure that each package comes from only one module and avoid confusion about what is being run.

Türkçe: Modüller, programlama için öğrenmeniz gereken ek bir katman gibi görünebilir. Modül kullanmak isteğe bağlı olsa da çözmek üzere tasarlandıkları sorunları anlamak önemlidir: Daha iyi access control (erişim denetimi): “Methods” başlıklı 5. Bölüm'de ele alınan erişim düzeylerine ek olarak yalnız aynı modüldeki diğer paketlerin erişebileceği paketler oluşturabilirsiniz. Daha açık dependency management (bağımlılık yönetimi): Modüller neye bağlı olduklarını belirttiği için Java, eksik bir JAR'ı çalışma zamanındaki ilk erişimde değil, uygulama başlatılırken bildirebilir. Custom Java builds (özelleştirilmiş Java derlemeleri): 150 MB'tan büyük tam JDK yerine yalnızca programınızın ihtiyaç duyduğu JDK parçalarını içeren bir Java çalışma zamanı oluşturabilirsiniz. Geliştirilmiş güvenlik: Özelleştirilmiş derlemenizden kullanmadığınız JDK parçalarını çıkarabildiğiniz için bu parçalarda keşfedilen güvenlik açıkları konusunda endişelenmeniz gerekmez. Geliştirilmiş performans: Daha küçük bir Java paketi, daha kısa başlatma süresi ve daha düşük bellek gereksinimi sağlar. Paket benzersizliğinin zorunlu kılınması: Modüller dışarı açılan paketleri belirttiği için Java, her paketin yalnız tek bir modülden gelmesini güvenceye alabilir ve hangi kodun çalıştığına ilişkin karışıklığı önleyebilir.

Creating and Running a Modular Program

English: In this section, we create, build, and run the `zoo.animal.feeding` module. We chose this one to start with because all the other modules depend on it. Figure 12.3 shows the design of this module. In addition to the `module-info.java` file, it has one package with one class inside.

Türkçe: Bu bölümde `zoo.animal.feeding` modülünü oluşturuyor, derliyoruz ve çalıştırıyoruz. Diğer bütün modüller ona bağlı olduğu için başlangıçta bu modülü seçtik. Şekil 12.3 modülün tasarımını gösterir. Modül, `module-info.java` dosyasına ek olarak içinde bir sınıf bulunan bir paket içerir.

English: FIGURE 12.3 — Contents of `zoo.animal.feeding: module-info.java` and `Task.java` in the `zoo.animal.feeding` package.

Türkçe: Şekil 12.3 — `zoo.animal.feeding` içeriği: `module-info.java` ve `zoo.animal.feeding` package'ındaki `Task.java`.

English: In the next sections, we create, compile, run, and package the `zoo.animal.feeding` module.

Türkçe: Sonraki bölümlerde `zoo.animal.feeding` modülünü oluşturacak, derleyecek, çalıştıracak ve paketleyeceğiz.

Creating the Files

English: First we have a really simple class that prints one line in a `main()` method. We know, that's not much of an implementation. All those programmers we hired can fill it in with business logic. In this book, we focus on what you need to know for the exam. So, let's create a simple class.

Türkçe: İlk olarak `main()` metodunda tek satır yazdıran çok basit bir sınıfımız vardır. Bunun kapsamlı bir implementation (gerçekleştirim) sayılmayacağını biliyoruz. İşe aldığımız programcılar bu sınıfı business logic (iş mantığı) ile doldurabilir. Bu kitapta sınav için bilmeniz gerekenlere odaklandığımızdan basit bir sınıf oluşturalım.

```
package zoo.animal.feeding;
public class Task {
    public static void main(String... args) {
        System.out.println("All fed!");
    }
}
```

English: Next comes the `module-info.java` file. This is the simplest possible one:

Türkçe: Ardından `module-info.java` dosyası gelir. Dosyanın mümkün olan en basit hâli şöyledir:

```
module zoo.animal.feeding {
}
```

English: There are a few key differences between a module declaration and a regular Java class declaration: ■■ The `module-info.java` file must be in the root directory of your module. Regular Java classes should be in packages. ■■ The module declaration must use the keyword `module` instead of `class`, `interface`, or `enum`. ■■ The module name follows the naming rules for package names. It often includes periods (.) in its name. Regular class and package names are not allowed to have dashes (-).

Türkçe: Module declaration (modül bildirimi) ile normal Java sınıf bildirimi arasında birkaç önemli fark vardır: `module-info.java` dosyası modülün kök dizininde bulunmalıdır. Normal Java sınıfları paketlerde yer alır. Modül bildirimi; `class`, `interface` veya `enum` yerine `module` anahtar kelimesini kullanılmalıdır. Modül adı, paket adlarının adlandırma kurallarına uyar ve çoğunlukla nokta (.) içerir. Normal sınıf ve paket adlarında tire (-) kullanılamaz.

English: Module names follow the same rule.

Türkçe: Modül adları da aynı kurala uyar.

English: That's a lot of rules for the simplest possible file. There will be many more rules when we flesh out this file later in the chapter.

Türkçe: Mümkün olan en basit dosya için bile epey kural vardır. Bölümün ilerleyen kısmında bu dosyayı ayrıntılandırırdığımızda çok daha fazla kuralla karşılaşacağız.

English: The next step is to make sure the files are in the right directory structure. Figure 12.4 shows the expected directory structure.

Türkçe: Bir sonraki adım, dosyaların doğru dizin yapısında olduğundan emin olmaktır. Şekil 12.4 beklenen dizin yapısını gösterir.

English: FIGURE 12.4 — Module `zoo.animal.feeding` directory structure: `mods/`; `feeding/module-info.java`; and `feeding/zoo/animal/feeding/Task.java`.

Türkçe: Şekil 12.4 — `zoo.animal.feeding` modülünün dizin yapısı: `mods/`; `feeding/module-info.java`; ve `feeding/zoo/animal/feeding/Task.java`.

English: In particular, `feeding` is the module directory, and the `module-info.java` file is directly under it. Just as with a regular JAR file, we also have the `zoo.animal.feeding` package with one subfolder per portion of the name. The `Task` class is in the appropriate subfolder for its package.

Türkçe: Özellikle `feeding`, modül directory'sidir ve `module-info.java` doğrudan onun altındadır. Normal bir JAR dosyasında olduğu gibi `zoo.animal.feeding` package'ı için adın her bölümüne karşılık gelen bir alt directory bulunur. `Task` class'ı da kendi package'ına uygun alt directory'dedir.

English: Also, note that we created a directory called `mods` at the same level as the module. We use it to store the module artifacts a little later in the chapter. This directory can be named anything, but `mods` is a common name. If you are following along with the online code example, note that the `mods` directory is not included, because it is empty.

Türkçe: Ayrıca modülle aynı düzeyde `mods` adlı bir dizin oluşturduğumuza dikkat edin. Bölümün ilerleyen kısmında modül derleme çıktılarını burada saklayacağız. Dizin adı farklı olabilir; ancak `mods` yaygın kullanılan bir addır. Çevrimiçi kod örneğini takip ediyorsanız boş olduğu için `mods` dizininin bulunmadığını unutmayın.

Compiling Our First Module

English: Before we can run modular code, we need to compile it. Other than the `module-path` option, this code should look familiar from Chapter 1:

Türkçe: Modüler kodu çalıştırmadan önce derlememiz gerekir. `module-path` seçeneği dışında bu komut size 1. Bölüm'den tanıdık gelmelidir.

```
javac --module-path mods -d feeding \
    feeding/zoo/animal/feeding/*.java feeding/module-info.java
```

English: When you're entering commands at the command line, they should be typed all on one line. We use line breaks in the book to make the commands easier to read and study. If you want to use multiple lines at the command prompt, the approach varies by operating system. Linux uses a backslash (\) to escape the line break.

Türkçe: Komutları komut satırına girerken tamamını tek satıra yazmalısınız. Kitapta komutların okunmasını ve incelenmesini kolaylaştırmak için satır sonları kullanıyoruz. Komut isteminde birden fazla satır kullanma yöntemi işletim sistemine göre değişir. Linux, satır sonundan kaçış için ters eğik çizgi (\) kullanır.

English: As a review, the `-d` option specifies the directory to place the class files in. The end of the command is a list of the `.java` files to compile. You can list the files individually or use a wildcard for all `.java` files in a subdirectory.

Türkçe: Hatırlatma olarak `-d` seçeneği, sınıf dosyalarının yerleştirileceği dizini belirtir. Komutun son bölümü derlenecek `.java` dosyalarının listesidir. Dosyaları tek tek listeleyebilir veya bir alt dizindeki bütün `.java` dosyaları için wildcard (joker karakter) kullanabilirsiniz.

English: The new part is `module-path`. This option indicates the location of any custom module files. In this example, `module-path` could have been omitted since there are no dependencies. You can think of `module-path` as replacing the `classpath` option when you are working on a modular program.

Türkçe: Yeni bölüm `module-path` seçeneğidir. Bu seçenek, özelleştirilmiş modül dosyalarının konumunu belirtir. Bu örnekte bağımlılık bulunmadığından `module-path` yazılmayabilirdi. Modüler bir program üzerinde çalışırken `module-path` seçeneğini `classpath` seçeneğinin yerini alan seçenek olarak düşünebilirsiniz.

English: What about the `classpath`?

Türkçe: Peki ya `classpath`?

English: The `classpath` option has three possible forms: `-cp`, `--class-path`, and `-classpath`.

Türkçe: `classpath` seçeneğinin üç biçimi vardır: `-cp`, `--class-path` ve `-classpath`.

English: You can still use these options. In fact, it is common to do so when writing nonmodular programs.

Türkçe: Yine de bu seçenekleri kullanabilirsiniz. Aslında, modüler olmayan programlar yazarken bunu yapmak yaygındır.

English: Just like `classpath`, you can use an abbreviation in the command. The syntax `--module-path` and `-p` are equivalent. That means we could have written many other commands in place of the previous command. The following four commands show the `-p` option:

Türkçe: `classpath`te olduğu gibi komutta kısaltma kullanabilirsiniz. `--module-path` ile `-p` eşdeğerdir. Dolayısıyla önceki komut yerine başka birçok geçerli biçim yazılabilir. Aşağıdaki dört komut `-p` seçeneğini gösterir:

```
javac -p mods -d feeding feeding/zoo/animal/feeding/*.java feeding/*.java
javac -p mods -d feeding feeding/zoo/animal/feeding/*.java feeding/module-info.java
javac -p mods -d feeding feeding/zoo/animal/feeding/Task.java feeding/module-info.java
javac -p mods -d feeding feeding/zoo/animal/feeding/Task.java feeding/*.java
```

English: While you can use whichever you like best, be sure that you can recognize all valid forms for the exam. Table 12.1 lists the options you need to know well when compiling modules. There are many more options you can pass to the `javac` command, but these are the ones you can expect to be tested on.

Türkçe: Tercih ettiğiniz biçimi kullanabilirsiniz; ancak sınav için bütün geçerli biçimleri tanıyabildiğinizden emin olun. Tablo 12.1, modülleri derlerken iyi bilmeniz gereken seçenekleri listeler. `javac` komutuna verilebilecek başka birçok seçenek vardır; sınavda sorulması beklenenler bunlardır.

English: TABLE 12.1 — Options you need to know for using modules with `javac`

Columns: Use for; abbreviation; long form.

Directory for class files: `-d <dir>`; n/a.

Module path: `-p <path>`; `--module-path <path>`.

Türkçe: TABLO 12.1 — `javac` ile modül kullanmak için bilmeniz gereken seçenekler

Sütunlar: Kullanım amacı; kısaltma; uzun biçim.

Class dosyalarının yazılacağı directory: `-d <dir>`; yok.

Module path: `-p <path>`; `--module-path <path>`.

English: Building Modules Even without modules, it is rare to run `javac` and `java` commands manually on a real project. They get long and complicated very quickly. Most developers use a build tool such as Maven or Gradle. These build tools suggest directories in which to place the class files, like `target/classes`.

Türkçe: Modülleri Derleme: Modüller olmasa bile gerçek bir projede `javac` ve `java` komutlarını elle çalıştırmak nadirdir; komutlar çok hızlı biçimde uzayıp karmaşılaşır. Çoğu geliştirici Maven veya Gradle gibi bir build tool (derleme aracı) kullanır. Bu araçlar sınıf dosyaları için `target/classes` gibi dizinler önerir.

English: It is likely that the only time you need to know the syntax of these commands is when you take the exam. The concepts themselves are useful, regardless.

Türkçe: Bu komutların sözdizimini bilmeniz gereken tek zaman büyük olasılıkla sınavdır. Yine de kavramların kendileri yararlıdır.

English: Be sure to memorize the module command syntax. You will be tested on it on the exam. We give you lots of practice questions on the syntax to reinforce it.

Türkçe: Modül komutlarının sözdizimini ezberlediğinizden emin olun; sınavda bu konu sorulacaktır. Sözdizimini pekiştirmeniz için çok sayıda çalışma sorusu veriyoruz.

Running Our First Module

English: Before we package our module, we should make sure it works by running it. To do that, we need to learn the full syntax. Suppose there is a module named `book.module`. Inside that module is a package named `com.sybex`, which has a class named `OCP` with a `main()` method. Figure 12.5 shows the syntax for running a module. Pay special attention to the `book.module/com.sybex.OCP` part. It is important to remember that you specify the module name followed by a slash (/) followed by the fully qualified class name.

Türkçe: Modülümüzü paketlemeden önce çalıştırıp doğru çalıştığını doğrulamalıyız. Bunun için tam sözdizimini öğrenmemiz gerekir. `book.module` adlı bir modül bulunduğunu varsayalım. Bu modül içinde `com.sybex` paketi, onun içinde de `main()` metoduna sahip `OCP` sınıfı vardır. Şekil 12.5 bir modülü çalıştırma sözdizimini gösterir. `book.module/com.sybex.OCP` bölümüne özellikle dikkat edin: önce modül adı, ardından eğik çizgi (/), sonra tam nitelikli sınıf adı yazılır.

English: FIGURE 12.5 — Running a module using `java`. The diagram identifies the location of modules, module name, module/package separator, package name, and class name in this command:

Türkçe: ŞEKİL 12.5 — `java` ile modül çalıştırma. Diyagram bu komut içindeki modül konumunu, modül adını, modül/paket ayırıcısını, paket adını ve sınıf adını gösterir:

```
java --module-path mods --module book.module/com.sybex.OCP
```

English: Now that we've seen the syntax, we can write the command to run the `Task` class in the `zoo.animal.feeding` package. In the following example, the package name and module name are the same. It is common for the module name to match either the full package name or the beginning of it.

Türkçe: Sözdizimini gördüğümüze göre `zoo.animal.feeding` paketindeki `Task` sınıfını çalıştıracak komutu yazabiliriz. Aşağıdaki örnekte paket adı ile modül adı aynıdır. Modül adının paket adının tamamıyla veya başlangıcıyla eşleşmesi yaygındır.

```
java --module-path feeding \
  --module zoo.animal.feeding/zoo.animal.feeding.Task
```

English: Since you already saw that `--module-path` uses the short form of `-p`, we bet you won't be surprised to learn there is a short form of `--module` as well. The short option is `-m`. That means the following command is equivalent:

Türkçe: `--module-path` seçeneğinin kısa biçiminin `-p` olduğunu gördüğünüz için `--module` seçeneğinin de kısa biçimi olması sizi şaşırtmayacaktır. Bu kısa seçenek `-m`'dir. Dolayısıyla aşağıdaki komut eşdeğerdir:

```
java -p feeding -m zoo.animal.feeding/zoo.animal.feeding.Task
```

English: In these examples, we used `feeding` as the module path because that's where we compiled the code. This will change once we package the module and run that.

Türkçe: Bu örneklerde kodu `feeding` dizinine derlediğimiz için bu dizini modül yolu olarak kullandık. Modülü paketleyip çalıştırdığımızda bu değişecektir.

English: Table 12.2 lists the options you need to know for the `java` command.

Türkçe: Tablo 12.2, `java` komutu için bilmeniz gereken seçenekleri listeler.

English: TABLE 12.2 — Options you need to know for using modules with `java`

Columns: Use for; abbreviation; long form.

Module name: `-m <name>; --module <name>`.

Module path: `-p <path>; --module-path <path>`.

Türkçe: TABLO 12.2 — `java` ile modül kullanmak için bilmeniz gereken seçenekler

Sütunlar: Kullanım amacı; kısaltma; uzun biçim.

Module adı: `-m <name>; --module <name>`.

Module path: `-p <path>; --module-path <path>`.

Packaging Our First Module

English: A module isn't much use if we can run it only in the folder it was created in. Our next step is to package it. Be sure to create a mods directory before running this command:

Türkçe: Bir modül yalnız oluşturulduğu dizinde çalıştırılabilirse pek kullanışlı değildir. Sıradaki adım onu paketlemektir. Şu komutu çalıştırmadan önce `mods` dizinini oluşturduğunuzdan emin olun:

```
jar -cvf mods/zoo.animal.feeding.jar -C feeding/ .
```

English: There's nothing module-specific here. We are packaging everything under the feeding directory and storing it in a JAR file named `zoo.animal.feeding.jar` under the mods folder. This represents how the module JAR will look to other code that wants to use it.

Türkçe: Burada modüle özgü bir işlem yoktur. `feeding` dizinindeki her şeyi paketleyip `mods` altındaki `zoo.animal.feeding.jar` adlı JAR dosyasında saklarız. Bu, modüler JAR'ın onu kullanmak isteyen diğer kod tarafından nasıl görüleceğini gösterir.

English: Now let's run the program again, but this time using the mods directory instead of the loose classes:

Türkçe: Şimdi programı yeniden çalıştıralım; ancak bu kez paketlenmemiş sınıf dosyaları yerine `mods` dizinini kullanalım:

```
java -p mods -m zoo.animal.feeding/zoo.animal.feeding.Task
```

English: You might notice that this command looks identical to the one in the previous section except for the directory. In the previous example, it was `feeding`. In this one, it is the module path of `mods`. Since the module path is used, a module JAR is being run.

Türkçe: Bu komutun, dizin dışında önceki bölümdekiyle aynı olduğunu fark edebilirsiniz. Önceki örnekte modül yolu `feeding`, bu örnekte ise `mods` dizinidir. Modül yolu kullanıldığı için modüler bir JAR çalıştırılır.

Updating Our Example for Multiple Modules

English: Now that our `zoo.animal.feeding` module is solid, we can start thinking about our other modules. As you can see in Figure 12.6, all three of the other modules in our system depend on the `zoo.animal.feeding` module.

Türkçe: Artık `zoo.animal.feeding` modülümüz sağlam olduğuna göre, diğer modüllerimizi düşünmeye başlayabiliriz. Şekil 12.6'da da görebileceğiniz gibi, sistemimizdeki diğer üç modül de `zoo.animal.feeding` modülüne bağlıdır.

English: FIGURE 12.6 — Modules depending on `zoo.animal.feeding`. Dependency arrows: `zoo.animal.care` → `zoo.animal.feeding`; `zoo.animal.talks` → `zoo.animal.feeding`, `zoo.animal.care`; `zoo.staff` → `zoo.animal.feeding`, `zoo.animal.care`, `zoo.animal.talks`. The three arrows into `zoo.animal.feeding` are highlighted.

Türkçe: Şekil 12.6 — `zoo.animal.feeding` modülüne bağlı modüller. Dependency okları: `zoo.animal.care` → `zoo.animal.feeding`; `zoo.animal.talks` → `zoo.animal.feeding`, `zoo.animal.care`; `zoo.staff` → `zoo.animal.feeding`, `zoo.animal.care`, `zoo.animal.talks`. `zoo.animal.feeding` modülüne gelen üç ok vurgulanmıştır.

Updating the Feeding Module

English: Since we will be having our other modules call code in the `zoo.animal.feeding` package, we need to declare this intent in the module declaration.

Türkçe: Diğer modüllerimizin `zoo.animal.feeding` paketindeki kodu çağırmasını sağlayacağımız için bu amacı module declaration (modül bildirimi) içinde belirtmemiz gerekir.

English: The `exports` directive is used to indicate that a module intends for those packages to be used by Java code outside the module. As you might expect, without an `exports` directive, the module is only available to be run from the command line on its own. In the following example, we export one package:

Türkçe: `exports` directive'i (yönergesi), modülün ilgili paketleri modül dışındaki Java kodunun kullanımına açtığını belirtir. `exports` directive'i olmadığında modül, yalnızca komut satırından tek başına çalıştırılabilir. Aşağıdaki örnekte bir paketi dışa açıyoruz:

```
module zoo.animal.feeding {
    exports zoo.animal.feeding;
}
```

English: Recompiling and repackaging the module will update the `module-info.class` inside our `zoo.animal.feeding.jar` file. These are the same `javac` and `jar` commands you ran previously:

Türkçe: Modülün yeniden derlenmesi ve paketlenmesi, `zoo.animal.feeding.jar` dosyamızın içindeki `module-info.class` dosyasını günceller. Bunlar daha önce çalıştırdığınız `javac` ve `jar` komutlarıdır:

```
javac -p mods -d feeding \
    feeding/zoo/animal/feeding/*.java feeding/module-info.java
jar -cvf mods/zoo.animal.feeding.jar -C feeding/ .
```

Creating a Care Module

English: Next, let's create the `zoo.animal.care` module. This time, we are going to have two packages. The `zoo.animal.care.medical` package will have the classes and methods that are intended for use by other modules. The `zoo.animal.care.details` package is only going to be used by this module. It will not be exported from the module. Think of it as healthcare privacy for the animals.

Türkçe: Ardından `zoo.animal.care` modülünü oluşturalım. Bu kez iki paketimiz olacaktır. `zoo.animal.care.medical`, diğer modüllerin kullanması amaçlanan sınıf ve metotları içerir. `zoo.animal.care.details` ise yalnızca bu modül tarafından kullanılır ve modülden dışa açılmaz. Bunu hayvanların sağlık bilgilerinin gizliliği olarak düşünebilirsiniz.

English: Figure 12.7 shows the contents of this module. Remember that all modules must have a `module-info.java` file.

Türkçe: Şekil 12.7, bu modülün içeriğini gösterir. Bütün modüllerin bir `module-info.java` dosyası olması gerektiğini unutmayın.

English: FIGURE 12.7 — Contents of `zoo.animal.care:module-info.java`;
`zoo.animal.care.medical/Diet.java`; and
`zoo.animal.care.details/HippoBirthday.java`.

Türkçe: Şekil 12.7 — `zoo.animal.care` modülünün içeriği: `module-info.java`;
`zoo.animal.care.medical/Diet.java`; ve
`zoo.animal.care.details/HippoBirthday.java`.

English: The module contains two basic packages and classes in addition to the `module-info.java` file:

Türkçe: Modül, `module-info.java` dosyasına ek olarak iki temel package ve bu package'lardaki class'ları içerir:

```
// HippoBirthday.java
package zoo.animal.care.details;
import zoo.animal.feeding.*;
public class HippoBirthday {
    private Task task;
}
// Diet.java
package zoo.animal.care.medical;
public class Diet { }
```

English: This time the `module-info.java` file specifies three things:

Türkçe: Bu kez `module-info.java` dosyası üç şeyi belirtir:

```
module zoo.animal.care {
    exports zoo.animal.care.medical;
    requires zoo.animal.feeding;
}
```

English: Line 1 specifies the name of the module. Line 2 lists the package we are exporting so it can be used by other modules. So far, this is similar to the `zoo.animal.feeding` module.

Türkçe: 1. satır modülün adını belirtir. 2. satır, diğer modüller tarafından kullanılabilmesi için dışa açtığımız paketi listeler. Buraya kadar bu yapı `zoo.animal.feeding` modülüne benzer.

English: On line 3, we see a new directive. The `requires` statement specifies that a module is needed. The `zoo.animal.care` module depends on the `zoo.animal.feeding` module.

Türkçe: 3. satırda yeni bir directive (yönerge) görürüz. `requires` ifadesi, bir modüle ihtiyaç duyulduğunu belirtir. `zoo.animal.care` modülü `zoo.animal.feeding` modülüne bağlıdır.

English: Next, we need to figure out the directory structure. We will create two packages. The first is `zoo.animal.care.details` and contains one class named `HippoBirthday`. The second is `zoo.animal.care.medical`, which contains one class named `Diet`. Try to draw the directory structure on paper or create it on your computer. If you are trying to run these examples without using the online code, just create classes without variables or methods for everything except the `module-info.java` files.

Türkçe: Ardından dizin yapısını belirlememiz gerekir. İki paket oluşturacağız. İlki `zoo.animal.care.details` paketidir ve `HippoBirthday` adlı bir sınıf içerir. İkincisi `zoo.animal.care.medical` paketidir ve `Diet` adlı bir sınıf içerir. Dizin yapısını kâğıda çizmeyi veya bilgisayarınızda oluşturmayı deneyin. Örnekleri çevrimiçi kodu kullanmadan çalıştırıyorsanız `module-info.java` dosyaları dışındaki sınıfları alan veya metot içermeyecek şekilde oluşturmanız yeterlidir.

English: You might have noticed that the packages begin with the same prefix as the module name. This is intentional. You can think of it as if the module name “claims” the matching package and all subpackages.

Türkçe: Paketlerin modül adıyla aynı prefix (önek) ile başladığını fark etmiş olabilirsiniz. Bu bilinçli bir tercihtir. Modül adının eşleşen paketi ve onun bütün alt paketlerini “sahiplendiğini” düşünebilirsiniz.

English: To review, we now compile and package the module:

Türkçe: Tekrar etmek gerekirse şimdi modülü derleyip paketliyoruz:

```
javac -p mods -d care \
  care/zoo/animal/care/details/*.java \
  care/zoo/animal/care/medical/*.java \
  care/module-info.java
```

English: We compile both packages and the `module-info.java` file. In the real world, you’ll use a build tool rather than doing this by hand. For the exam, you just list all the packages and/or files you want to compile.

Türkçe: Her iki paketi ve `module-info.java` dosyasını derliyoruz. Gerçek bir projede bunu elle yapmak yerine bir build tool (derleme aracı) kullanırsınız. Sınavda derlemek istediğiniz bütün paketleri ve/veya dosyaları listelersiniz.

English: Now that we have compiled code, it’s time to create the module JAR:

Türkçe: Kodu derlediğimize göre artık modül JAR’ını oluşturabiliriz:

```
jar -cvf mods/zoo.animal.care.jar -C care/ .
```

Creating the Talks Module

English: So far, we've used only one `exports` and `requires` statement in a module. Now you'll learn how to handle exporting multiple packages or requiring multiple modules. In Figure 12.8, observe that the `zoo.animal.talks` module depends on two modules: `zoo.animal.feeding` and `zoo.animal.care`. This means that there must be two `requires` statements in the `module-info.java` file.

Türkçe: Şimdiye kadar bir modülde yalnızca birer `exports` ve `requires` ifadesi kullandık. Artık birden fazla paketi dışa açmayı veya birden fazla modülü gerektirmeyi nasıl yöneteceğinizi öğreneceksiniz. Şekil 12.8'de `zoo.animal.talks` modülünün `zoo.animal.feeding` ve `zoo.animal.care` olmak üzere iki modüle bağlı olduğuna dikkat edin. Bu nedenle `module-info.java` dosyasında iki `requires` ifadesi bulunmalıdır.

English: FIGURE 12.8 — Dependencies for `zoo.animal.talks`. Dependency arrows:

`zoo.animal.care` → `zoo.animal.feeding`; `zoo.animal.talks` → `zoo.animal.feeding`, `zoo.animal.care`; `zoo.staff` → `zoo.animal.feeding`, `zoo.animal.care`, `zoo.animal.talks`. The two arrows leaving `zoo.animal.talks` are highlighted.

Türkçe: Şekil 12.8 — `zoo.animal.talks` bağımlılıkları. Dependency okları: `zoo.animal.care` → `zoo.animal.feeding`; `zoo.animal.talks` → `zoo.animal.feeding`, `zoo.animal.care`; `zoo.staff` → `zoo.animal.feeding`, `zoo.animal.care`, `zoo.animal.talks`. `zoo.animal.talks` modülünden çıkan iki ok vurgulanmıştır.

English: Figure 12.9 shows the contents of this module. We are going to export all three packages in this module.

Türkçe: Şekil 12.9 bu modülün içeriğini gösterir. Modüldeki üç package'ı da export edeceğiz.

English: FIGURE 12.9 — Contents of `zoo.animal.talks:module-info.java`;

`zoo.animal.talks.content/ElephantScript.java` and `SeaLionScript.java`; `zoo.animal.talks.schedule/Weekday.java` and `Weekend.java`; and `zoo.animal.talks.media/Announcement.java` and `Signage.java`.

Türkçe: Şekil 12.9 — `zoo.animal.talks` modülünün içeriği: `module-info.java`; `zoo.animal.talks.content/ElephantScript.java` ve `SeaLionScript.java`; `zoo.animal.talks.schedule/Weekday.java` ve `Weekend.java`; ayrıca `zoo.animal.talks.media/Announcement.java` ve `Signage.java`.

English: First let's look at the `module-info.java` file for `zoo.animal.talks`:

Türkçe: Önce `zoo.animal.talks` için `module-info.java` dosyasına bakalım:

```
module zoo.animal.talks {
    exports zoo.animal.talks.content;
    exports zoo.animal.talks.media;
    exports zoo.animal.talks.schedule;

    requires zoo.animal.feeding;
    requires zoo.animal.care;
}
```

English: Line 1 shows the module name. Lines 2–4 allow other modules to reference all three packages. Lines 6 and 7 specify the two modules that this module depends on.

Türkçe: 1. satır modül adını gösterir. 2–4. satırlar, diğer modüllerin üç pakete de başvurmasına izin verir. 6. ve 7. satırlar bu modülün bağlı olduğu iki modülü belirtir.

English: Then we have the six classes, as shown here:

Türkçe: Burada da gösterildiği gibi altı sınıf var:

```
// ElephantScript.java
package zoo.animal.talks.content;
public class ElephantScript { }
// SeaLionScript.java
package zoo.animal.talks.content;
public class SeaLionScript { }
// Announcement.java
package zoo.animal.talks.media;
public class Announcement {
    public static void main(String[] args) {
        System.out.println("We will be having talks");
    }
}
```

```
// Signage.java
package zoo.animal.talks.media;
public class Signage { }
// Weekday.java
package zoo.animal.talks.schedule;
public class Weekday { }
// Weekend.java
package zoo.animal.talks.schedule;
public class Weekend {}
```

English: If you are still following along on your computer, create these classes in the packages. The following are the commands to compile and build the module:

Türkçe: Bilgisayarınızda hala takip ediyorsanız, bu sınıfları paketlerde oluşturun. Modülü derlemek ve oluşturmak için komutlar şunlardır:

```
javac -p mods -d talks \
    talks/zoo/animal/talks/content/*.java \
    talks/zoo/animal/talks/media/*.java \
    talks/zoo/animal/talks/schedule/*.java \
    talks/module-info.java
jar -cvf mods/zoo.animal.talks.jar -C talks/ .
```

Creating the Staff Module

English: Our final module is zoo.staff. Figure 12.10 shows that there is only one package inside. We will not be exposing this package outside the module.

Türkçe: Son modülümüz `zoo.staff`'tir. Şekil 12.10, modülün yalnızca bir paket içerdiğini gösterir. Bu paketi modül dışına açmayacağız.

English: FIGURE 12.10 — Contents of `zoo.staff: module-info.java` and `zoo.staff/Jobs.java`.

Türkçe: Şekil 12.10 — `zoo.staff` modülünün içeriği: `module-info.java` ve `zoo.staff/Jobs.java`.

English: Based on Figure 12.11, do you know what should go in the `module-info`?

Türkçe: Şekil 12.11'e bakarak `module-info.java` dosyasında nelerin yer alması gerektiğini biliyor musunuz?

English: FIGURE 12.11 — Dependencies for `zoo.staff`: arrows point from `zoo.staff` to `zoo.animal.feeding`, `zoo.animal.care`, and `zoo.animal.talks`.

Türkçe: Şekil 12.11 — `zoo.staff` bağımlılıkları: Oklar `zoo.staff` modülünden `zoo.animal.feeding`, `zoo.animal.care` ve `zoo.animal.talks` modüllerine yönelir.

English: There are three arrows in Figure 12.11 pointing from `zoo.staff` to other modules. These represent the three modules that are required. Since no packages are to be exposed from `zoo.staff`, there are no `exports` statements. This gives us:

Türkçe: Şekil 12.11'de `zoo.staff` modülünden diğer modüllere yönelen üç ok vardır. Bunlar gerekli üç modülü gösterir. `zoo.staff` içindeki hiçbir package export edilmeyeceği için `exports` statement'ı yoktur. Böylece şu declaration elde edilir:

```
module zoo.staff {
  requires zoo.animal.feeding;
  requires zoo.animal.care;
  requires zoo.animal.talks;
}
```

English: In this module, we have a single class in the `Jobs.java` file:

Türkçe: Bu modülde, `Jobs.java` dosyasında tek bir sınıf vardır:

```
package zoo.staff;
public class Jobs { }
```

English: For those of you following along on your computer, create a class in the package. The following are the commands to compile and build the module:

Türkçe: Bilgisayarınızda takip edenler için, pakette bir sınıf oluşturun. Modülü derlemek ve oluşturmak için komutlar şunlardır:

```
javac -p mods -d staff staff/zoo/staff/*.java staff/module-info.java
jar -cvf mods/zoo.staff.jar -C staff/ .
```

Diving into the Module Declaration

English: Now that we've successfully created modules, we can learn more about the module declaration. In these sections, we look at exports, requires, and opens. In the following section on services, we explore provides and uses. Now would be a good time to mention that these directives can appear in any order in the module declaration.

Türkçe: Modülleri başarıyla oluşturduğumuza göre artık module declaration (modül bildirimi) hakkında daha fazla bilgi edinebiliriz. Bu bölümlerde `exports`, `requires` ve `opens` directive'lerini inceleyeceğiz. Hizmetlerle ilgili sonraki bölümde `provides` ve `uses` directive'lerini ele alacağız. Bu directive'lerin modül bildirimi içinde herhangi bir sırada yer alabileceğini de belirtelim.

Exporting a Package

English: We've already seen how exports packageName exports a package to other modules. It's also possible to export a package to a specific module. Suppose the zoo decides that only staff members should have access to the talks. We could update the module declaration as follows:

Türkçe: `exports packageName` ifadesinin bir paketi diğer modüllere nasıl dışa açtığını daha önce gördük. Bir paketi yalnızca belirli bir modüle dışa açmak da mümkündür. Hayvanat bahçesinin konuşmalara yalnızca personelin erişmesini istediğini varsayalım. Modül bildirimini şöyle güncelleyebiliriz:

```
module zoo.animal.talks {  
  exports zoo.animal.talks.content to zoo.staff;  
  exports zoo.animal.talks.media;  
  exports zoo.animal.talks.schedule;  
  requires zoo.animal.feeding;  
  requires zoo.animal.care;  
}
```

English: From the zoo.staff module, nothing has changed. However, no other modules would be allowed to access that package.

Türkçe: `zoo.staff` modülü açısından hiçbir şey değişmez. Ancak başka hiçbir modülün bu pakete erişmesine izin verilmez.

English: You might have noticed that none of our other modules requires zoo.animal.talks in the first place. However, we don't know what other modules will exist in the future. It is important to consider future use when designing modules. Since we want only the one module to have access, we only allow access for that module.

Türkçe: Diğer modüllerimizin hiçbirinde `requires zoo.animal.talks` bulunmadığını fark etmiş olabilirsiniz. Ancak gelecekte başka hangi modüllerin bulunacağını bilmiyoruz. Modül tasarlarken gelecekteki kullanımı göz önünde bulundurmak önemlidir. Yalnızca bir modülün erişmesini istediğimiz için erişim iznini yalnız o modüle veririz.

English: Exported Types We've been talking about exporting a package. But what does that mean, exactly? All public classes, interfaces, enums, and records are exported. Further, any public and protected fields and methods in those files are visible.

Türkçe: Dışa Açılan Türler: Bir paketi dışa açmaktan söz ediyoruz; peki bu tam olarak ne anlama gelir? Bütün `public` sınıflar, `interface`'ler, `enum`'lar ve `record`'lar dışa açılır. Ayrıca bu türlerdeki bütün `public` ve `protected` alanlar ile metotlar görünürdür.

English: Fields and methods that are private are not visible because they are not accessible outside the class. Similarly, package fields and methods are not visible because they are not accessible outside the package.

Türkçe: `private` alan ve metotlar sınıf dışından erişilemediği için görünür değildir. Benzer biçimde `package-private` alan ve metotlar da paket dışından erişilemediği için görünür değildir.

English: The `exports` directive essentially gives us more levels of access control. Table 12.3 lists the full access control options.

Türkçe: `exports` directive'i özünde ek access control (erişim denetimi) düzeyleri sağlar. Tablo 12.3, erişim denetimi seçeneklerinin tamamını listeler.

English: TABLE 12.3 — Access control with modules

Columns: Level; within module code; outside module.

`private`: Available only within the class; no access.

`package-private`: Available only within the package; no access.

`protected`: Available only within the package or to subclasses; accessible to subclasses only if the package is exported.

`public`: Available to all classes; accessible only if the package is exported.

Türkçe: TABLO 12.3 — Modüllerle access control

Sütunlar: Erişim düzeyi; modül içindeki kod; modül dışındaki kod.

`private`: Yalnızca sınıf içinde kullanılabilir; erişim yok.

`package-private`: Yalnızca package içinde kullanılabilir; erişim yok.

`protected`: Yalnızca package içinde veya alt sınıflar tarafından kullanılabilir; package export edilmişse yalnızca alt sınıflar erişebilir.

`public`: Bütün sınıflar erişebilir; yalnızca package export edilmişse erişilebilir.

Requiring a Module Transitively

English: As you saw earlier in this chapter, `requires moduleName` specifies that the current module depends on `moduleName`. There's also a `requires transitive moduleName`, which means that any module that requires this module will also depend on `moduleName`.

Türkçe: Bu bölümde daha önce gördüğünüz gibi `requires moduleName`, geçerli modülün `moduleName` modülüne bağlı olduğunu belirtir. `requires transitive moduleName` ise geçerli modülü gerektiren her modülün `moduleName` modülüne de bağımlı olacağı anlamına gelir.

English: Well, that was a mouthful. Let's look at an example. Figure 12.12 shows the modules with dashed lines for the redundant relationships and solid lines for relationships specified in the `module-info`. This shows how the module relationships would look if we were to only use transitive dependencies.

Türkçe: Açıklaması biraz uzun oldu; bir örneğe bakalım. Şekil 12.12'de gereksiz ilişkiler kesikli çizgilerle, `module-info.java` içinde belirtilen ilişkiler ise düz çizgilerle gösterilir. Böylece yalnızca transitive dependencies (geçişli bağımlılıklar) kullandığımızda modül ilişkilerinin nasıl görüneceğini görebiliriz.

English: FIGURE 12.12 — Transitive dependency version of our modules. Solid arrows: `zoo.animal.care` → `zoo.animal.feeding`; `zoo.animal.talks` → `zoo.animal.care`; `zoo.staff` → `zoo.animal.talks`. Dashed arrows show redundant readability edges: `zoo.animal.talks` □ `zoo.animal.feeding`; `zoo.staff` □ `zoo.animal.care`, `zoo.animal.feeding`.

Türkçe: Şekil 12.12 — Modüllerin transitive dependency kullanan sürümü. Düz oklar: `zoo.animal.care` → `zoo.animal.feeding`; `zoo.animal.talks` → `zoo.animal.care`; `zoo.staff` → `zoo.animal.talks`. Kesikli oklar redundant readability ilişkilerini gösterir: `zoo.animal.talks` □ `zoo.animal.feeding`; `zoo.staff` □ `zoo.animal.care`, `zoo.animal.feeding`.

English: For example, `zoo.animal.talks` depends on `zoo.animal.care`, which depends on `zoo.animal.feeding`. That means the direct solid dependency arrow between `zoo.animal.talks` and `zoo.animal.feeding` no longer appears in Figure 12.12.

Türkçe: Örneğin `zoo.animal.talks`, `zoo.animal.care` modülüne; o da `zoo.animal.feeding` modülüne bağlıdır. Bu nedenle `zoo.animal.talks` ile `zoo.animal.feeding` arasındaki doğrudan düz dependency oku Şekil 12.12'de artık görünmez; kesikli ok transitive readability'yi göstermeye devam eder.

English: Now let's look at the four module declarations. The first module remains unchanged. We are exporting one package to any packages that use the module.

Türkçe: Şimdi dört module declaration'a (modül bildirimine) bakalım. İlk modül değişmeden kalır. Bir paketi, bu modülü kullanan bütün modüllere dışa açarız.

```
module zoo.animal.feeding {
  exports zoo.animal.feeding;
}
```

English: The `zoo.animal.care` module is the first opportunity to improve things. Rather than forcing all remaining modules to explicitly specify `zoo.animal.feeding`, the code uses `requires transitive`.

Türkçe: `zoo.animal.care` modülü yapıyı iyileştirmek için ilk fırsatı sunar. Kalan modüllerin `zoo.animal.feeding` modülünü açıkça belirtmesini zorunlu kılmak yerine kodda `requires transitive` kullanırız.

```
module zoo.animal.care {
  exports zoo.animal.care.medical;
  requires transitive zoo.animal.feeding;
}
```

English: In the `zoo.animal.talks` module, we make a similar change and don't force other modules to specify `zoo.animal.care`. We also no longer need to specify `zoo.animal.feeding`, so that line is commented out.

Türkçe: `zoo.animal.talks` modülünde benzer bir değişiklik yapar ve diğer modülleri `zoo.animal.care` modülünü açıkça belirtmeye zorlamayız. Artık `zoo.animal.feeding` modülünü de belirtmemiz gerekmediği için ilgili satır yorum satırı hâline getirilmiştir.

```
module zoo.animal.talks {
  exports zoo.animal.talks.content to zoo.staff;
  exports zoo.animal.talks.media;
  exports zoo.animal.talks.schedule;
  // no longer needed requires zoo.animal.feeding;
  // no longer needed requires zoo.animal.care;
  requires transitive zoo.animal.care;
}
```

English: Finally, in the `zoo.staff` module, we can get rid of two `requires` statements.

Türkçe: Son olarak `zoo.staff` modülünde iki `requires` ifadesini kaldırabiliriz.

```
module zoo.staff {
  // no longer needed requires zoo.animal.feeding;
  // no longer needed requires zoo.animal.care;
  requires zoo.animal.talks;
}
```

English: The more modules you have, the greater the benefits of the `requires transitive` compound. It is also more convenient for the caller. If you were trying to work with this zoo, you could just require `zoo.staff` and have the remaining dependencies automatically inferred.

Türkçe: Modül sayısı arttıkça `requires transitive` birleşik ifadesinin sağladığı avantajlar da artar. Çağırılan kod için de daha kullanışlıdır. Bu hayvanat bahçesi uygulamasıyla çalışırken yalnızca `zoo.staff` modülünü gerektirebilir ve kalan bağımlılıkların otomatik olarak çıkarılmasını sağlayabilirsiniz.

Effects of `requires transitive`

English: Given our new module declarations, and using Figure 12.12, what is the effect of applying the transitive modifier to the `requires` statement in our `zoo.animal.care` module? Applying the transitive modifiers has the following effects: ■■ Module `zoo.animal.talks` can optionally declare that it requires the `zoo.animal.feeding` module, but it is not required.

Türkçe: Yeni modül bildirimlerimizi ve Şekil 12.12'yi dikkate alırsak `zoo.animal.care` modülündeki `requires` ifadesine `transitive` modifier'ını (değiştiricisini) uygulamanın etkisi nedir? `transitive` modifier şu sonuçları doğurur: `zoo.animal.talks` modülü isterse `requires zoo.animal.feeding` bildirimini ekleyebilir, ancak bunu yapması zorunlu değildir.

English: ■■ Module `zoo.animal.care` cannot be compiled or executed without access to the `zoo.animal.feeding` module. ■■ Module `zoo.animal.talks` cannot be compiled or executed without access to the `zoo.animal.feeding` module.

Türkçe: `zoo.animal.care` modülü, `zoo.animal.feeding` modülüne erişim olmadan derlenemez veya çalıştırılmaz. `zoo.animal.talks` modülü de `zoo.animal.feeding` modülüne erişim olmadan derlenemez veya çalıştırılmaz.

English: These rules hold even if the `zoo.animal.care` and `zoo.animal.talks` modules do not explicitly reference any packages in the `zoo.animal.feeding` module. On the other hand, without the transitive modifier in our module declaration of `zoo.animal.care`, the other modules would have to explicitly use `requires` in order to reference any packages in the `zoo.animal.feeding` module.

Türkçe: Bu kurallar, `zoo.animal.care` ve `zoo.animal.talks` modülleri `zoo.animal.feeding` içindeki hiçbir pakete açıkça başvurmasa bile geçerlidir. Öte yandan `zoo.animal.care` modülünün bildiriminde `transitive` modifier bulunmasaydı, diğer modüllerin `zoo.animal.feeding` içindeki paketlere başvurabilmek için açıkça `requires` kullanması gerekirdi.

Duplicate requires Statements

English: One place the exam might try to trick you is mixing `requires` and `requires transitive`. Can you think of a reason this code doesn't compile?

Türkçe: Sınavın sizi yanıltabileceği noktalardan biri `requires` ile `requires transitive` ifadelerini birlikte kullanmaktır. Bu kodun neden derlenmediğini bulabilir misiniz?

```
module bad.module {
    requires zoo.animal.talks;
    requires transitive zoo.animal.talks;
}
```

English: Java doesn't allow you to repeat the same module in a `requires` clause. It is redundant and most likely an error in coding. Keep in mind that `requires transitive` is like `requires` plus some extra behavior.

Türkçe: Java aynı modülün bir `requires` clause'unda (yan tümcesinde) iki kez belirtilmesine izin vermez. Bu tekrar gereksizdir ve büyük olasılıkla bir kodlama hatasıdır. `requires transitive` ifadesinin, ek davranışlar kazandırılmış `requires` gibi olduğunu unutmayın.

Opening a Package

English: Java allows callers to inspect and call code at runtime with a technique called reflection. This is a powerful approach that allows calling code that might not be available at compile time. It can even be used to subvert access control! Don't worry—you don't need to know how to write code using reflection for the exam.

Türkçe: Java, çağırıcı kodun reflection (yansıma) adlı teknikle çalışma zamanında kodu incelemesine ve çağırmasına izin verir. Bu güçlü yaklaşım, derleme zamanında erişilebilir olmayabilecek kodun çağrılmasını sağlar. Hatta erişim denetimini aşmak için bile kullanılabilir! Sınav için reflection kullanan kod yazmayı bilmeniz gerekmez.

English: The `opens` directive is used to enable reflection of a package within a module. You only need to be aware that the `opens` directive exists rather than understanding it in detail for the exam.

Türkçe: `opens` directive'i, modül içindeki bir pakette reflection kullanımını etkinleştirir. Sınavda ayrıntılarını anlamanız gerekmez; yalnızca `opens` directive'inin var olduğunu bilmeniz yeterlidir.

English: Since reflection can be dangerous, the module system requires developers to explicitly allow reflection in the module declaration if they want calling modules to be allowed to use it. The following shows how to enable reflection for two packages in the `zoo.animal.talks` module:

Türkçe: Reflection tehlikeli olabileceğinden modül sistemi, geliştiricilerin çağıran modüllere reflection izni vermek istiyorlarsa bunu modül bildirimi içinde açıkça belirtmelerini zorunlu kılar. Aşağıdaki kod, `zoo.animal.talks` modülündeki iki paket için reflection'ın nasıl etkinleştirileceğini gösterir:

```
module zoo.animal.talks {
  opens zoo.animal.talks.schedule;
  opens zoo.animal.talks.media to zoo.staff;
}
```

English: The first example allows any module using this one to use reflection. The second example only gives that privilege to the `zoo.staff` module. There are two more directives you need to know for the exam—`provides` and `uses`—which are covered in the following section.

Türkçe: İlk örnek bu modülü kullanan her modülün reflection kullanmasına izin verir. İkinci örnek bu ayrıcalığı yalnızca `zoo.staff` modülüne tanır. Sınav için bilmeniz gereken iki directive daha vardır: `provides` ve `uses`. Bunlar sonraki bölümde ele alınır.

English: Opening an Entire Module In the previous example, we opened two packages in the `zoo.animal.talks` module, but suppose we instead wanted to open all packages for reflection. No problem. We can use the `open` module modifier, rather than the `opens` directive (notice the `s` difference):

Türkçe: Bir Modülün Tamamını Açma: Önceki örnekte `zoo.animal.talks` modülünde iki paket açtık. Bunun yerine bütün paketleri reflection için açmak istediğimizi varsayalım. `opens` directive'i yerine `open module` modifier'ını kullanabiliriz (`s` farkına dikkat edin):

```
open module zoo.animal.talks {
}
```

English: With this module modifier, Java knows we want all the packages in the module to be open. What happens if you apply both together?

Türkçe: Bu module modifier (modül değiştiricisi) sayesinde Java, modül içindeki bütün paketleri açmak istediğimizi bilir. İkisini birlikte uygularsak ne olur?

```
open module zoo.animal.talks {
  opens zoo.animal.talks.schedule; // DOES NOT COMPILE
}
```

English: This does not compile because a modifier that uses the `open` modifier is not permitted to use the `opens` directive. After all, the packages are already open!

Türkçe: Bu kod **derlenmez**; çünkü `open` modifier'ı kullanılan bir modülde `opens` directive'ine izin verilmez. Zaten bütün paketler açıktır.

Creating a Service

English: In this section, you learn how to create a service. A service is composed of an interface, any classes the interface references, and a way of looking up implementations of the interface. The implementations are not part of the service.

Türkçe: Bu bölümde bir service'in (hizmetin) nasıl oluşturulacağını öğreneceksiniz. Service; bir `interface`, bu interface'in başvurduğu sınıflar ve interface'in implementation'larını (gerçekleştirimlerini) bulmanın bir yolundan oluşur. Implementation'lar service'in parçası değildir.

English: We will be using a tour application in the services section. It has four modules shown in Figure 12.13. In this example, the `zoo.tours.api` and `zoo.tours.reservations` modules make up the service since they consist of the interface and lookup functionality.

Türkçe: Hizmetler bölümünde bir tur uygulaması kullanacağız. Uygulama, Şekil 12.13'te gösterilen dört modülden oluşur. Bu örnekte `zoo.tours.api` ve `zoo.tours.reservations` modülleri, interface'i ve lookup (bulma) işlevini içerdikleri için birlikte service'i oluşturur.

English: FIGURE 12.13 — Modules in the tour application: the service consists of `zoo.tours.api` (service provider interface) and `zoo.tours.reservations` (service locator); `zoo.visitor` is the consumer, and `zoo.tours.agency` is the service provider. Dependency arrows: `zoo.tours.reservations` → `zoo.tours.api`; `zoo.visitor` → `zoo.tours.api`, `zoo.tours.reservations`; `zoo.tours.agency` → `zoo.tours.api`. A dashed lookup arrow points from `zoo.tours.reservations` to `zoo.tours.agency`.

Türkçe: Şekil 12.13 — Tur uygulamasındaki modüller: Service, `zoo.tours.api` (service provider interface) ile `zoo.tours.reservations` (service locator) modüllerinden oluşur; `zoo.visitor` consumer, `zoo.tours.agency` ise service provider'dır. Dependency okları: `zoo.tours.reservations` → `zoo.tours.api`; `zoo.visitor` → `zoo.tours.api`, `zoo.tours.reservations`; `zoo.tours.agency` → `zoo.tours.api`. Kesikli lookup oku `zoo.tours.reservations` modülünden `zoo.tours.agency` modülüne yönelir.

English: You aren't required to have four separate modules. We do so to illustrate the concepts. For example, the service provider interface and service locator could be in the same module.

Türkçe: Dört ayrı modüle sahip olmanız gerekmez. Bunu kavramları örneklemek için yapıyoruz. Örneğin service provider interface (hizmet sağlayıcı arayüzü) ile service locator (hizmet bulucu) aynı modüle olabilir.

Declaring the Service Provider Interface

English: First, the `zoo.tours.api` module defines a Java object called `Souvenir`. It is considered part of the service because it will be referenced by the interface.

Türkçe: İlk olarak `zoo.tours.api` modülü, `Souvenir` adlı bir Java nesnesi tanımlar. `Souvenir`, interface tarafından referans verileceği için service'in bir parçası kabul edilir.

```
// Souvenir.java
package zoo.tours.api;
public record Souvenir(String description) { }
```

English: Next, the module contains a Java interface type. This interface is called the service provider interface because it specifies what behavior our service will have. In this case, it is a simple API with three methods.

Türkçe: Ardından modül bir Java interface türü içerir. Bu interface, service'in hangi davranışları sunacağını belirttiği için service provider interface olarak adlandırılır. Burada üç metottan oluşan basit bir API söz konusudur.

```
// Tour.java
package zoo.tours.api;
public interface Tour {
    String name();
    int length();
    Souvenir getSouvenir();
}
```

English: All three methods use the implicit public modifier. Since we are working with modules, we also need to create a module-info.java file so our module definition exports the package containing the interface.

Türkçe: Üç metod da örtük (implicit) `public` modifier'a sahiptir. Modüllerle çalıştığımız için modül tanımının interface'i içeren paketi dışa açmasını sağlayan bir `module-info.java` dosyası da oluşturmamız gerekir.

```
// module-info.java
module zoo.tours.api {
    exports zoo.tours.api;
}
```

English: Now that we have both files, we can compile and package this module.

Türkçe: Artık her iki dosyaya da sahip olduğumuza göre, bu modülü derleyebilir ve paketleyebiliriz.

```
javac -d serviceProviderInterfaceModule \
    serviceProviderInterfaceModule/zoo/tours/api/*.java \
    serviceProviderInterfaceModule/module-info.java
jar -cvf mods/zoo.tours.api.jar -C serviceProviderInterfaceModule/ .
```

English: A service provider “interface” can be an abstract class rather than an actual interface. Since you will only see it as an interface on the exam, we use that term in the book.

Türkçe: Bir service provider “interface”, gerçek bir interface yerine `abstract` sınıf olabilir. Sınavda yalnızca interface biçimiyle karşılaşacağınız için kitapta bu terimi kullanıyoruz.

English: To review, the service includes the service provider interface and supporting classes it references. The service also includes the lookup functionality, which we define next.

Türkçe: Özetlemek gerekirse service; service provider interface'i ve bu interface'in referans verdiği destekleyici sınıfları içerir. Service ayrıca birazdan tanımlayacağımız lookup (bulma) işlevini de kapsar.

Creating a Service Locator

English: To complete our service, we need a service locator. A service locator can find any classes that implement a service provider interface.

Türkçe: Service'imizi tamamlamak için bir service locator gerekir. Service locator, service provider interface'i implement eden bütün sınıfları bulabilir.

English: Luckily, Java provides a `ServiceLoader` class to help with this task. You pass the service provider interface type to its `load()` method, and Java will return any implementation services it can find. The following class shows it in action:

Türkçe: Neyse ki Java bu görev için `ServiceLoader` sınıfını sağlar. Service provider interface türünü `load()` metoduna geçirirsiniz; Java da bulabildiği service implementation'larını döndürür. Aşağıdaki sınıf bunun kullanımını gösterir:

```
// TourFinder.java
package zoo.tours.reservations;
import java.util.*;
import zoo.tours.api.*;
public class TourFinder {
    public static Tour findSingleTour() {
        ServiceLoader<Tour> loader = ServiceLoader.load(Tour.class);
        for (Tour tour: loader)
            return tour;
        return null;
    }
    public static List<Tour> findAllTours() {
        List<Tour> tours = new ArrayList<>();
        ServiceLoader<Tour> loader = ServiceLoader.load(Tour.class);
        for (Tour tour: loader)
            tours.add(tour);
        return tours;
    }
}
```

English: As you can see, we provided two lookup methods. The first is a convenience method if you are expecting exactly one `Tour` to be returned. The other returns a `List`, which accommodates any number of service providers. At runtime, there may be many service providers (or none) that are found by the service locator.

Türkçe: Gördüğünüz gibi iki lookup metodu sağladık. İlki, tam olarak bir `Tour` döndürülmesini beklediğiniz durumlar için bir convenience method'dur (kolaylık metodudur). Diğeri, istenen sayıda service provider'ı barındırabilen bir `List` döndürür. Çalışma zamanında service locator sıfır, bir veya birden fazla service provider bulabilir.

English: The `ServiceLoader` call is relatively expensive. If you are writing a real application, it is best to cache the result.

Türkçe: `ServiceLoader` çağırısı nispeten pahalıdır. Gerçek bir uygulama yazıyorsanız sonucu önbelleğe almak en iyisidir.

English: Our module definition exports the package with the lookup class `TourFinder`. It requires the service provider interface package. It also has the `uses` directive since it will be looking up a service.

Türkçe: Modül tanımımız, lookup sınıfı `TourFinder`'i içeren paketi dışa açar. Service provider interface'i içeren modülü `requires` ile bağımlılık olarak bildirir. Ayrıca bir service arayacağı için `uses` directive'ini içerir.

```
// module-info.java
module zoo.tours.reservations {
  exports zoo.tours.reservations;
  requires zoo.tours.api;
  uses zoo.tours.api.Tour;
}
```

English: Remember that both `requires` and `uses` are needed, one for compilation and one for lookup. Finally, we compile and package the module.

Türkçe: Hem `requires` hem de `uses` gerektiğini unutmayın: biri derleme, diğeri service lookup işlemi içindir. Son olarak modülü derleyip paketliyoruz.

```
javac -p mods -d serviceLocatorModule \
  serviceLocatorModule/zoo/tours/reservations/*.java \
  serviceLocatorModule/module-info.java
jar -cvf mods/zoo.tours.reservations.jar -C serviceLocatorModule/ .
```

English: Now that we have the interface and lookup logic, we have completed our service.

Türkçe: Artık interface'e ve lookup mantığına sahip olduğumuza göre service'imizi tamamladık.

English: Using `ServiceLoader` There are two methods in `ServiceLoader` that you need to know for the exam. The declaration is as follows, sans the full implementation:

Türkçe: `ServiceLoader` içinde sınav için bilmeniz gereken iki metot vardır. Tam implementation gösterilmediğinde declaration (bildirim) aşağıdaki gibidir:

```
public final class ServiceLoader<S> implements Iterable<S> {
  public static <S> ServiceLoader<S> load(Class<S> service) {... }
  public Stream<Provider<S>> stream() {... }
  // Additional methods
}
```

English: As we already saw, calling `ServiceLoader.load()` returns an object that you can loop through normally. However, requesting a `Stream` gives you a different type. The reason for this is that a `Stream` controls when elements are evaluated. Therefore, a `ServiceLoader` returns a `Stream` of `Provider` objects. You have to call `get()` to retrieve the value you wanted out of each `Provider`, such as in this example:

Türkçe: Daha önce gördüğümüz gibi `ServiceLoader.load()` çağrısı, normal biçimde üzerinde dönebileceğiniz bir nesne döndürür. Ancak bir `Stream` istediğinizde farklı bir tür elde edersiniz; çünkü `Stream`, elemanların ne zaman değerlendirileceğini denetler. Bu nedenle `ServiceLoader`, `Provider` nesnelerinden oluşan bir `Stream` döndürür. Aşağıdaki örnekte olduğu gibi her `Provider` içinden istediğiniz değeri almak için `get()` çağrısı yapmanız gerekir:

```
ServiceLoader.load(Tour.class)
    .stream()
    .map(Provider::get)
    .mapToInt(Tour::length)
    .max()
    .ifPresent(System.out::println);
```

Invoking from a Consumer

English: Next up is to call the service locator by a consumer. A consumer (or client) refers to a module that obtains and uses a service. Once the consumer has acquired a service via the service locator, it is able to invoke the methods provided by the service provider interface.

Türkçe: Sıradaki adım, consumer'ın service locator'ı çağırmasıdır. Consumer (tüketici veya client), bir service'i edininip kullanan modülü ifade eder. Consumer, service locator üzerinden service'i edindikten sonra service provider interface'in sunduğu metotları çağırabilir.

```
// Tourist.java
package zoo.visitor;
import java.util.*;
import zoo.tours.api.*;
import zoo.tours.reservations.*;
public class Tourist {
    public static void main(String[] args) {
        Tour tour = TourFinder.findSingleTour();
        System.out.println("Single tour: " + tour);
        List<Tour> tours = TourFinder.findAllTours();
        System.out.println("# tours: " + tours.size());
    }
}
```

English: Our module definition doesn't need to know anything about the implementations since the `zoo.tours.reservations` module is handling the lookup.

Türkçe: Lookup işlemini `zoo.tours.reservations` modülü yürüttüğü için modül tanımımızın concrete implementation'lar (somut gerçekleştirmeler) hakkında hiçbir şey bilmesi gerekmez.

```
// module-info.java
module zoo.visitor {
    requires zoo.tours.api;
    requires zoo.tours.reservations;
}
```

English: This time, we get to run a program after compiling and packaging.

Türkçe: Bu sefer, derleme ve paketlemeden sonra bir program çalıştıracakız.

```
javac -p mods -d consumerModule \
    consumerModule/zoo/visitor/*.java consumerModule/module-info.java
```

```
jar -cvf mods/zoo.visitor.jar -C consumerModule/ .
java -p mods -m zoo.visitor/zoo.visitor.Tourist
```

English: The program outputs the following:

Türkçe: Program şu çıktıyı verir:

```
Single tour: null
# tours: 0
```

English: Well, that makes sense. We haven't written a class that implements the interface yet.

Türkçe: Bu sonuç mantıklıdır; henüz interface'i implement eden bir sınıf yazmadık.

Adding a Service Provider

English: A service provider is the implementation of a service provider interface. As we said earlier, at runtime it is possible to have multiple implementation classes or modules. We will stick to one here for simplicity.

Türkçe: Service provider, service provider interface'in implementation'ıdır. Daha önce belirttiğimiz gibi çalışma zamanında birden fazla implementation sınıfı veya modülü bulunabilir. Basitlik için burada yalnızca bir tane kullanacağız.

English: Our service provider is the zoo.tours.agency package because we've outsourced the running of tours to a third party.

Türkçe: Turların yürütülmesini bir third party'ye (üçüncü tarafa) devrettiğimiz için service provider'ımız `zoo.tours.agency` paketinde yer alır.

```
// TourImpl.java
package zoo.tours.agency;
import zoo.tours.api.*;
public class TourImpl implements Tour {
    public String name() {
        return "Behind the Scenes";
    }
    public int length() {
        return 120;
    }
    public Souvenir getSouvenir() {
        return new Souvenir("stuffed animal");
    }
}
```

English: Again, we need a module-info.java file to create a module.

Türkçe: Yine, bir modül oluşturmak için bir `module-info.java` dosyasına ihtiyacımız vardır.

```
// module-info.java
module zoo.tours.agency {
    requires zoo.tours.api;
    provides zoo.tours.api.Tour with zoo.tours.agency.TourImpl;
}
```

English: The module declaration requires the module containing the interface as a dependency. We don't export the package that implements the interface since we don't want callers referring to it directly. Instead, we use the `provides` directive. This allows us to specify that we provide an implementation of the interface with a specific implementation class. The syntax looks like this:

Türkçe: Modül bildirimi, interface'i içeren modülü bağımlılık olarak `requires` eder. Çağırın kodun implementation'a doğrudan referans vermesini istemediğimiz için interface'i implement eden paketi dışa açmayız. Bunun yerine `provides` directive'ini kullanırız. Böylece interface için belirli bir implementation sınıfı sağladığımızı bildiririz. Sözdizimi şöyledir:

```
provides interfaceName with className;
```

English: We have not exported the package containing the implementation.

Türkçe: Implementation'ı içeren paketi dışa açmadık.

English: Instead, we have made the implementation available to a service provider using the interface.

Türkçe: Bunun yerine implementation'ı, interface üzerinden service-loading (hizmet yükleme) mekanizmasının bulabileceği hâle getirdik.

English: Finally, we compile it and package it up.

Türkçe: Son olarak, derleyip paketliyoruz.

```
javac -p mods -d serviceProviderModule \
    serviceProviderModule/zoo/tours/agency/*.java \
    serviceProviderModule/module-info.java
jar -cvf mods/zoo.tours.agency.jar -C serviceProviderModule/ .
```

English: Now comes the cool part. We can run the Java program again.

Türkçe: Şimdi işin güzel kısmına geldik. Java programını yeniden çalıştırabiliriz.

```
java -p mods -m zoo.visitor/zoo.visitor.Tourist
```

English: This time, we see the following output:

Türkçe: Bu sefer aşağıdaki çıktıyı görürüz:

```
Single tour: zoo.tours.agency.TourImpl@1936f0f5
# tours: 1
```

Java 17 editör notu: Varsayılan `Object.toString()` gösterimindeki onaltılık identity-hash son ekinin her çalıştırmada aynı olması garanti edilmez.

English: Notice how we didn't recompile the `zoo.tours.reservations` or `zoo.visitor` package. The service locator was able to observe that there was now a service provider implementation available and find it for us.

Türkçe: `zoo.tours.reservations` veya `zoo.visitor` paketini yeniden derlemediğimize dikkat edin. Service locator, artık kullanılabilir bir service provider implementation'ı bulunduğunu algılayıp onu bizim için bulabildi.

English: This is useful when you have functionality that changes independently of the rest of the code base. For example, you might have custom reports or logging.

Türkçe: Bu yaklaşım, codebase'in (kod tabanının) geri kalanından bağımsız değişen işlevler için kullanışlıdır. Örneğin özelleştirilmiş rapor veya logging implementation'larınız olabilir.

English: In software development, the concept of separating different components into stand-alone pieces is referred to as loose coupling. One advantage of loosely coupled code is that it can be easily swapped out or replaced with minimal (or zero) changes to code that uses it. Relying on a loosely coupled structure allows service modules to be easily extensible at runtime.

Türkçe: Yazılım geliştirmede farklı component'ları (bileşenleri) bağımsız parçalar hâlinde ayırma yaklaşımına loose coupling (gevşek bağlılık) denir. Loosely coupled (gevşek bağlı) kodun bir avantajı, onu kullanan kodda çok az değişiklikle veya hiç değişiklik yapmadan kolayca başka bir implementation'la değiştirilebilmesidir. Gevşek bağlı bir yapı kullanmak, service modüllerinin çalışma zamanında kolayca genişletilebilmesini sağlar.

Reviewing Directives and Services

English: Table 12.4 summarizes what we've covered in the section about services. We recommend learning really well what is needed when each artifact is in a separate module. That is most likely what you will see on the exam and will ensure that you understand the concepts. Table 12.5 lists all the directives you need to know for the exam.

Türkçe: Tablo 12.4, service'ler hakkında bu bölümde ele aldıklarımızı özetler. Her artifact (öge) ayrı bir modülde bulunduğunda hangi öğelerin gerektiğini çok iyi öğrenmenizi öneririz. Sınavda büyük olasılıkla bu düzenle karşılaşacaksınız; ayrıca bu düzen kavramları anladığınızdan emin olmanızı sağlar. Tablo 12.5, sınav için bilmeniz gereken bütün directive'leri listeler.

English: TABLE 12.4 — Reviewing services

Columns: Artifact; part of the service?; directives required.

Service provider interface: Yes; `exports`.

Service provider: No; `requires`, `provides`.

Service locator: Yes; `exports`, `requires`, `uses`.

Consumer: No; `requires`.

Türkçe: TABLO 12.4 — Service'leri gözden geçirme

Sütunlar: Artifact (öge); service'in parçası mı?; gerekli directive'ler.

Service provider interface (hizmet sağlayıcı arayüzü): Evet; `exports`.

Service provider (hizmet sağlayıcı): Hayır; `requires`, `provides`.

Service locator (hizmet bulucu): Evet; `exports`, `requires`, `uses`.

Consumer (tüketicisi): Hayır; `requires`.

English: TABLE 12.5 — Reviewing directives

Columns: Directive; description.

`exports package; / exports package to module;`: Makes a package available outside the module.

`requires module; / requires transitive module;`: Specifies another module as a dependency.

`opens package; / opens package to module;`: Allows a package to be used with reflection.

`provides serviceInterface with implName;`: Makes a service available.

`uses serviceInterface;`: References a service.

Türkçe: TABLO 12.5 — Directive'leri gözden geçirme

Sütunlar: Directive; açıklama.

`exports package; / exports package to module;`: Bir package'ı modül dışından erişilebilir kılar.

`requires module; / requires transitive module;`: Başka bir modülü dependency olarak belirtir.

`opens package; / opens package to module;`: Bir package'ın reflection ile kullanılmasına izin verir.

`provides serviceInterface with implName;`: Bir service'i kullanılabilir hâle getirir.

`uses serviceInterface;`: Bir service'e başvurur.

Discovering Modules

English: So far, we've been working with modules that we wrote. Even the classes built into the JDK are modularized. In this section, we show you how to use commands to learn about modules.

Türkçe: Şimdiye kadar kendi yazdığımız modüllerle çalıştık. JDK ile birlikte gelen sınıflar bile modüllere ayrılmıştır. Bu bölümde modüller hakkında bilgi edinmek için komutların nasıl kullanılacağını gösteriyoruz.

English: You do not need to know the output of the commands in this section. You do, however, need to know the syntax of the commands and what they do. We include the output where it facilitates remembering what is going on. But you don't need to memorize that (which frees up more space in your head to memorize command-line options).

Türkçe: Bu bölümdeki komutların çıktısını bilmeniz gerekmez. Buna karşılık komut sözdizimini ve komutların ne yaptığını bilmelisiniz. Neler olduğunu hatırlamayı kolaylaştırdığı yerlerde çıktıyı gösteriyoruz; ancak bunu ezberlemeniz gerekmez. Böylece komut satırı seçeneklerini ezberlemek için zihninizde daha fazla yer kalır.

Identifying Built-in Modules

English: The most important module to know is `java.base`. It contains most of the packages you have been learning about for the exam. In fact, it is so important that you don't even have to use the `requires` directive; it is available to all modular applications. Your `module-info.java` file will still compile if you explicitly require `java.base`. However, it is redundant, so it's better to omit it. Table 12.6 lists some common modules and what they contain.

Türkçe: Bilmeniz gereken en önemli modül `java.base`'dir. Sınav için öğrendiğiniz paketlerin çoğunu içerir. Hatta o kadar önemlidir ki `requires` directive'ini kullanmanız bile gerekmez; bütün modüller uygulamalar tarafından kullanılabilir. `module-info.java` dosyanız açıkça `requires java.base;` bildirse de derlenir. Ancak bu tekrar gereksiz olduğundan directive'i yazmamak daha iyidir. Tablo 12.6 bazı yaygın modülleri ve içeriklerini listeler.

English: TABLE 12.6 — Common modules**Columns:** Module name; what it contains; coverage in the book.`java.base`: Collections, math, I/O, NIO.2, concurrency, and more; most of this book.`java.desktop`: Abstract Window Toolkit (AWT) and Swing; not on the exam beyond the module name.`java.logging`: Logging; not on the exam beyond the module name.`java.sql`: JDBC; Chapter 15, “JDBC.”`java.xml`: Extensible Markup Language (XML); not on the exam beyond the module name.**Türkçe:** TABLO 12.6 — Yaygın modüller**Sütunlar:** Modül adı; içeriği; kitaptaki kapsam.`java.base`: Collections, matematik, I/O, NIO.2, concurrency ve daha fazlası; bu kitabın büyük bölümü.`java.desktop`: Abstract Window Toolkit (AWT) ve Swing; modül adı dışında sınav kapsamında değil.`java.logging`: Logging; modül adı dışında sınav kapsamında değil.`java.sql`: JDBC; Bölüm 15, “JDBC.”`java.xml`: Extensible Markup Language (XML); modül adı dışında sınav kapsamında değil.**English:** The exam creators consider it important to recognize the names of modules supplied by the JDK. You do not need to know the names by heart, but you do need to be able to pick them out of a lineup.**Türkçe:** Sınavı hazırlayanlar, JDK tarafından sağlanan modül adlarını tanımanın önemli olduğunu düşünür. Bu adları ezbere bilmeniz gerekmez; ancak seçenekler arasından ayırt edebilmelisiniz.**English:** For the exam, you need to know that module names begin with `java` for APIs you are likely to use and with `jdk` for APIs that are specific to the JDK. Table 12.7 lists all the modules that begin with `java`.**Türkçe:** Sınav için, kullanmanız olası API'lerin modül adlarının `java` ile; JDK'ya özgü API'lerin modül adlarının ise `jdk` ile başladığını bilmelisiniz. Tablo 12.7, `java` ile başlayan bütün modülleri listeler.**English:** TABLE 12.7 — Java modules prefixed with `java` (Part 1 of 2)`java.base`; `java.naming`; `java.smartcardio``java.compiler`; `java.net.http`; `java.sql``java.datatransfer`; `java.prefs`; `java.sql.rowset``java.desktop`; `java.rmi`; `java.transaction.xa`**Türkçe:** TABLO 12.7 — `java` önekli Java modülleri (1/2)`java.base`; `java.naming`; `java.smartcardio``java.compiler`; `java.net.http`; `java.sql``java.datatransfer`; `java.prefs`; `java.sql.rowset``java.desktop`; `java.rmi`; `java.transaction.xa`**English:** TABLE 12.7 — Java modules prefixed with `java` (continued, Part 2 of 2)`java.instrument`; `java.scripting`; `java.xml``java.logging`; `java.se`; `java.xml.crypto``java.management`; `java.security.jgss``java.management.rmi`; `java.security.sasl`**Türkçe:** TABLO 12.7 — `java` önekli Java modülleri (devam, 2/2)`java.instrument`; `java.scripting`; `java.xml``java.logging`; `java.se`; `java.xml.crypto``java.management`; `java.security.jgss``java.management.rmi`; `java.security.sasl`

English: Table 12.8 lists selected modules that begin with `jdk`. We recommend reviewing it right before the exam so the names are more likely to sound familiar. Remember that you do not have to memorize them.

Türkçe: Tablo 12.8, `jdk` ile başlayan seçilmiş modülleri listeler. Adların tanıdık gelme olasılığını artırmak için sınavdan hemen önce bu tabloyu gözden geçirmenizi öneririz. Bunları ezberlemeniz gerekmediğini unutmayın.

Java 17 editör notu: Tablo exhaustive değildir. Java 17 runtime'ında `jdk.unsupported`, `jdk.unsupported.desktop`, `jdk.random` ve `jdk.jpackage` gibi burada gösterilmeyen başka `jdk.*` modülleri de bulunabilir.

English: TABLE 12.8 — Java modules prefixed with `jdk`

```
jdk.accessibility; jdk.javadoc; jdk.management.agent
jdk.attach; jdk.jcmd; jdk.management.jfr
jdk.charsets; jdk.jconsole; jdk.naming.dns
jdk.compiler; jdk.jdeps; jdk.naming.rmi
jdk.crypto.cryptoki; jdk.jdi; jdk.net
jdk.crypto.ec; jdk.jdwp.agent; jdk.nio.mapmode
jdk.dynalink; jdk.jfr; jdk.sctp
jdk.editpad; jdk.jlink; jdk.security.auth
jdk.hotspot.agent; jdk.jshell; jdk.security.jgss
jdk.httpserver; jdk.jsobject; jdk.xml.dom
jdk.incubator.foreign; jdk.jstatd; jdk.zipfs
jdk.incubator.vector; jdk.localedata
jdk.jartool; jdk.management
```

Türkçe: TABLO 12.8 — `jdk` önekli Java modülleri

```
jdk.accessibility; jdk.javadoc; jdk.management.agent
jdk.attach; jdk.jcmd; jdk.management.jfr
jdk.charsets; jdk.jconsole; jdk.naming.dns
jdk.compiler; jdk.jdeps; jdk.naming.rmi
jdk.crypto.cryptoki; jdk.jdi; jdk.net
jdk.crypto.ec; jdk.jdwp.agent; jdk.nio.mapmode
jdk.dynalink; jdk.jfr; jdk.sctp
jdk.editpad; jdk.jlink; jdk.security.auth
jdk.hotspot.agent; jdk.jshell; jdk.security.jgss
jdk.httpserver; jdk.jsobject; jdk.xml.dom
jdk.incubator.foreign; jdk.jstatd; jdk.zipfs
jdk.incubator.vector; jdk.localedata
jdk.jartool; jdk.management
```

Getting Details with java

English: The `java` command has three module-related options. One describes a module, another lists the available modules, and the third shows the module resolution logic.

Türkçe: `java` komutunun modüllerle ilgili üç seçeneği vardır. Biri bir modülü açıklar, diğeri kullanılabilir modülleri listeler, üçüncüsü ise module resolution (modül çözümü) mantığını gösterir.

English: It is also possible to add modules, exports, and more at the command line. But please don't. It's confusing and hard to maintain. Note that these flags are available on java but not all commands.

Türkçe: Komut satırından modül, export ve başka öğeler eklemek de mümkündür. Ancak bunu yapmamanızı öneririz; kafa karıştırıcıdır ve bakımı zordur. Bu flag'lerin (bayrakların) `java` komutunda bulunduğunu, fakat bütün komutlarda bulunmadığını unutmayın.

Describing a Module

English: Suppose you are given the `zoo.animal.feeding` module JAR file and want to know about its module structure. You could “unjar” it and open the `module-info.java` file. This would show you that the module exports one package and doesn't explicitly require any modules.

Türkçe: Elinizde `zoo.animal.feeding` modülünün JAR dosyası bulunduğunu ve modül yapısını öğrenmek istediğinizi varsayalım. JAR'ın içeriğini çıkarıp `module-info.java` kaynağından derlenen `module-info.class` descriptor'ını (tanımlayıcısını) inceleyebilirsiniz. Bu descriptor, modülün bir paketi dışa açtığını ve hiçbir modülü açıkça `requires` etmediğini gösterir.

```
module zoo.animal.feeding {  
  exports zoo.animal.feeding;  
}
```

English: However, there is an easier way. The java command has an option to describe a module. The following two commands are equivalent:

Türkçe: Ancak daha kolay bir yol vardır. `java` komutu bir modülü açıklayan bir seçenek sunar. Aşağıdaki iki komut eşdeğerdir:

```
java -p mods -d zoo.animal.feeding  
java -p mods --describe-module zoo.animal.feeding
```

English: Each prints information about the module. For example, it might print this:

Türkçe: İkisi de modül hakkındaki bilgileri yazdırır. Örneğin şu çıktı üretilebilir:

```
zoo.animal.feeding file:///absolutePath/mods/zoo.animal.feeding.jar  
exports zoo.animal.feeding  
requires java.base mandated
```

English: The first line is the module we asked about: `zoo.animal.feeding`. The second line starts with information about the module. In our case, it is the same package exports statement we had in the module declaration file.

Türkçe: İlk satır, hakkında bilgi istediğimiz `zoo.animal.feeding` modülünü gösterir. İkinci satır modül hakkındaki bilgilerle başlar. Buradaki `exports` ifadesi, modül bildirimi dosyasındaki paket dışa açma ifadesiyle aynıdır.

English: On the third line, we see `requires java.base mandated`. Now, wait a minute. The module declaration very clearly does not specify any modules that `zoo.animal.feeding` has as dependencies.

Türkçe: Üçüncü satırda `requires java.base mandated` ifadesini görürüz. Oysa modül bildirimi, `zoo.animal.feeding` için herhangi bir modül bağımlılığını açıkça belirtmemektedir.

English: Remember, the `java.base` module is special. It is automatically added as a dependency to all modules. This module has frequently used packages like `java.util`. That's what the mandated is about. You get `java.base` regardless of whether you asked for it.

Türkçe: `java.base` modülünün özel olduğunu unutmayın. `java.base` dışındaki bütün adlandırılmış modüllere (named modules) otomatik olarak bağımlılık şeklinde eklenir ve `java.util` gibi sık kullanılan paketleri içerir. Çıktıdaki `mandated` bunun göstergesidir: açıkça istemeseniz de `java.base` kullanılabilir olur.

English: In classes, the `java.lang` package is automatically imported whether you type it or not. The `java.base` module works the same way. It is automatically available to all other modules.

Türkçe: Sınıflarda, `java.lang` paketi yazsanız da yazmasanız da otomatik olarak içe aktarılır. `java.base` modülü aynı şekilde çalışır. Diğer bütün modüller için otomatik olarak kullanılabilir.

English: More about Describing Modules

Türkçe: Modülleri Açıklama Hakkında Ek Bilgi

English: You only need to know how to run `--describe-module` for the exam rather than interpret the output. However, you might encounter some surprises when experimenting with this feature, so we describe them in a bit more detail here.

Türkçe: Sınav için yalnızca `--describe-module` komutunu nasıl çalıştıracağınızı bilmeniz gerekir; çıktıyı yorumlamanız gerekmez. Bununla birlikte, bu özellikle denemeler yaparken bazı sürprizlerle karşılaşabilirsiniz. Bu nedenle ayrıntıları burada biraz daha açıklıyoruz.

English: Assume the following are the contents of `module-info.java` in `zoo.animal.care`:

Türkçe: Aşağıdakilerin `zoo.animal.care` içindeki `module-info.java` dosyasının içeriği olduğunu varsayalım:

```
module zoo.animal.care {
  exports zoo.animal.care.medical to zoo.staff;
  requires transitive zoo.animal.feeding;
}
```

English: Now we have the command to describe the module and the output.

Türkçe: Şimdi modülü açıklayan komuta ve komutun çıktısına bakalım.

```
java -p mods -d zoo.animal.care
```

```
zoo.animal.care file:///absolutePath/mods/zoo.animal.care.jar
requires zoo.animal.feeding transitive
requires java.base mandated
qualified exports zoo.animal.care.medical to zoo.staff
contains zoo.animal.care.details
```

English: The first line of the output is the absolute path of the module file. The two `requires` lines should look familiar as well. The first is in the module-info, and the other is added to all modules. Next comes something new. The qualified exports is the full name of the package we are exporting to a specific module.

Türkçe: `requires zoo.animal.feeding transitive, requires java.base mandated, qualified exports zoo.animal.care.medical to zoo.staff` ve `contains zoo.animal.care.details` satırları görülür. Çıktının ilk satırı modül dosyasının absolute path'idir (mutlak yoludur). İki `requires` satırı da tanıtık gelmelidir. Birincisi `module-info.java` içindedir; diğeri (`requires java.base mandated`) `java.base` dışındaki bütün adlandırılmış modüllere (named modules) otomatik eklenir. `qualified exports`, yalnızca belirli bir modüle dışa açılan paketin tam adını gösterir.

English: Finally, the contains means that there is a package in the module that is not exported at all. This is true. Our module has two packages, and one is available only to code inside the module.

Türkçe: Son olarak `contains`, modül içinde hiç dışa açılmayan bir paket bulunduğunu gösterir. Gerçekten de modülümüz iki pakete sahiptir ve bunlardan biri yalnızca modül içindeki kod tarafından kullanılabilir.

Listing Available Modules

English: In addition to describing modules, you can use the `java` command to list the modules that are available. The simplest form lists the modules that are part of the JDK.

Türkçe: Modülleri açıklamanın yanı sıra kullanılabilir modülleri listelemek için `java` komutunu kullanabilirsiniz. En basit biçim, JDK'nın parçası olan modülleri listeler.

```
java --list-modules
```

English: When we ran it, the output went on for 70 lines and looked like this:

Türkçe: Bunu çalıştırdığımızda, çıktı 70 satır boyunca devam etti ve şöyle görünüyordu:

```
java.base@17
java.compiler@17
java.datatransfer@17
```

English: This is a listing of all the modules that come with Java and their version numbers. You can tell that we were using Java 17 when testing this example.

Türkçe: Bu, Java ile gelen bütün module'lerin ve sürüm numaralarının listesidir. Bu örneğin Java 17 ile test edildiği çıktıdan anlaşılabilir.

English: More interestingly, you can use this command with custom code. Let's try again with the directory containing our zoo modules.

Türkçe: Daha ilginç olarak, bu komutu özel kodla kullanabilirsiniz. Hayvanat bahçesi modüllerimizi içeren dizinle tekrar deneyelim.

```
java -p mods --list-modules
```

English: How many lines do you expect to be in the output this time? There are 78 lines now: the 70 built-in modules plus the 8 we've created in this chapter. Two of the custom lines look like this:

Türkçe: Bu kez çıktıda kaç satır beklersiniz? Çıktı artık 78 satırdır: 70 built-in module ve bu chapter'da oluşturduğumuz 8 custom module. Custom satırlardan ikisi şöyledir:

```
zoo.animal.care file:///absolutePath/mods/zoo.animal.care.jar
zoo.animal.feeding file:///absolutePath/mods/zoo.animal.feeding.jar
```

English: Since these are custom modules, we get a location on the file system. If the project had a module version number, it would have both the version number and the file system path.

Türkçe: Bunlar özel modüller olduğundan, dosya sisteminde bir konum elde ederiz. Projenin bir modül sürüm numarası olsaydı, hem sürüm numarasına hem de dosya sistemi yoluna sahip olurdu.

English: Note that `--list-modules` exits as soon as it prints the observable modules. It does not run the program.

Türkçe: `--list-modules`, observable module'leri yazdırır yazdırmaz sonlanır; programı çalıştırmaz.

Showing Module Resolution

English: If listing the modules doesn't give you enough output, you can also use the `--show-module-resolution` option. You can think of it as a way of debugging modules. It spits out a lot of output when the program starts up. Then it runs the program.

Türkçe: Module'leri listelemek yeterli bilgi vermiyorsa `--show-module-resolution` option'ını kullanabilirsiniz. Bunu module'lerde hata ayıklamanın bir yolu gibi düşünebilirsiniz. Program başlarken çok miktarda çıktı üretir ve ardından programı çalıştırır.

```
java --show-module-resolution \
  -p feeding \
  -m zoo.animal.feeding/zoo.animal.feeding.Task
```

English: Luckily, you don't need to understand this output. That said, having seen it will make it easier to remember. Here's a snippet of the output:

Türkçe: Sınav için bu çıktıyı anlamanız gerekmez; ancak daha önce görmüş olmak option'ı hatırlamayı kolaylaştırır. Çıktının bir bölümü şöyledir:

```
root zoo.animal.feeding file:///absolutePath/feeding/
java.base binds java.desktop jrt:/java.desktop
java.base binds jdk.jartool jrt:/jdk.jartool
...
jdk.security.auth requires java.naming jrt:/java.naming
jdk.security.auth requires java.security.jgss jrt:/java.security.jgss
...
All fed!
```

English: It starts by listing the root module. That's the one we are running: `zoo.animal.feeding`. Then it lists many lines of packages included by the mandatory `java.base` module. After a while, it lists modules that have dependencies. Finally, it outputs the result of the program: `All fed!`.

Türkçe: Çıktı önce root module'ü listeler; çalıştırdığımız module `zoo.animal.feeding`dir. Ardından mandatory `java.base` module'ünün eklediği çok sayıda satır ve daha sonra dependency'si bulunan module'ler gösterilir. Son olarak program sonucu olan `All fed!` yazdırılır.

Describing with jar

English: Like the `java` command, the `jar` command can describe a module. These commands are equivalent:

Türkçe: `java` komutu gibi `jar` komutu da bir module'ü tanımlayabilir. Bu komutlar eşdeğerdir:

```
jar -f mods/zoo.animal.feeding.jar -d
jar --file mods/zoo.animal.feeding.jar --describe-module
```

English: The output is slightly different from when we used the `java` command to describe the module. With `jar`, it outputs the following:

Türkçe: Çıktı, module'ü tanımlamak için `java` komutunu kullandığımızdaki çıktıdan biraz farklıdır. `jar` aşağıdaki çıktıyı üretir:

```
zoo.animal.feeding jar:file:///absolutePath/mods/zoo.animal.feeding.jar!/module-info.class
exports zoo.animal.feeding
requires java.base mandated
```

English: The JAR version includes the `module-info.class` in the filename, which is not a particularly significant difference in the scheme of things. You don't need to know this difference. You do need to know that both commands can describe a module.

Türkçe: Çıktıda `zoo.animal.feeding.jar!/module-info.class, exports zoo.animal.feeding` ve `requires java.base mandated` bulunur. JAR sürümü, dosya adında `module-info.class` bilgisini de gösterir; ancak bu önemli bir fark değildir ve sınav için bilinmesi gerekmez. Bilmeniz gereken, her iki komutun da bir module'ü tanımlayabildiğidir.

Learning about Dependencies with jdeps

English: The `jdeps` command gives you information about dependencies within a module. Unlike describing a module, it looks at the code in addition to the module declaration. This tells you what dependencies are actually used rather than simply declared. Luckily, you are not expected to memorize all the options for the exam.

Türkçe: `jdeps` komutu bir module içindeki dependency'ler hakkında bilgi verir. Module'ü tanımlayan komutlardan farklı olarak yalnızca module declaration'a değil, koda da bakar. Böylece yalnızca declare edilenleri değil, gerçekte kullanılan dependency'leri gösterir. Sınav için bütün option'ları ezberlemeniz beklenmez.

English: You are expected to understand how to use `jdeps` with projects that have not yet been modularized to assist in identifying dependencies and problems. First, we will create a JAR file from this class. If you are following along, feel free to copy the class from the online examples referenced at the beginning of the chapter rather than typing it in.

Türkçe: Henüz modularize edilmemiş projelerde dependency'leri ve sorunları belirlemek için `jdeps` kullanımını anlamanız beklenir. Önce bu class'tan bir JAR oluşturacağız. Örneği uyguluyorsanız class'ı yeniden yazmak yerine chapter başında belirtilen online örneklerden kopyalayabilirsiniz.

```
// Animatronic.java
package zoo.dinos;
import java.time.*;
import java.util.*;
import sun.misc.Unsafe;
public class Animatronic {
    private List<String> names;
    private LocalDate visitDate;
    public Animatronic(List<String> names, LocalDate visitDate) {
        this.names = names;
        this.visitDate = visitDate;
    }
}
```

```
    public void unsafeMethod() {
        Unsafe unsafe = Unsafe.getUnsafe();
    }
}
```

English: This example is silly. It uses a number of unrelated classes. The Bronx Zoo really did have electronic moving dinosaurs for a while, so at least the idea of having dinosaurs in a zoo isn't beyond the realm of possibility.

Türkçe: Bu örnek kasıtlı olarak biraz tuhaftır ve birbiriyle ilgisiz çeşitli class'lar kullanır. Bronx Hayvanat Bahçesi'nde bir dönem gerçekten elektronik, hareketli dinazorlar bulunduğundan hayvanat bahçesinde dinazor fikri en azından bütünüyle hayal ürünü değildir.

English: Now we can compile this file. You might have noticed that there is no `module-info.java` file. That is because we aren't creating a module. We are looking into what dependencies we will need when we do modularize this JAR.

Türkçe: Şimdi bu dosyayı derleyebiliriz. `module-info.java` bulunmadığına dikkat edin; çünkü henüz bir module oluşturmuyoruz. Bu JAR'ı modularize ettiğimizde hangi dependency'lere gerek duyacağımızı araştırıyoruz.

```
javac zoo/dinos/*.java
```

English: Compiling works, but it gives you some warnings about `Unsafe` being an internal API. Don't worry about those for now—we discuss that shortly. (Maybe the dinosaurs went extinct because they did something unsafe.)

Türkçe: Derleme başarılıdır; ancak `Unsafe`ın internal API olduğu konusunda bazı uyarılar verir. Şimdilik bunlar için endişelenmeyin; konuyu kısa süre içinde tartışıyoruz. (Belki de dinazorlar güvensiz bir şey yaptıkları için nesli tükendi.)

English: Next, we create a JAR file.

Türkçe: Daha sonra JAR dosyasını oluşturuyoruz.

```
jar -cvf zoo.dino.jar .
```

English: We can run the `jdeps` command against this JAR to learn about its dependencies. First, let's run the command without any options. On the first two lines, the command prints the modules that we would need to add with a `requires` directive to migrate to the module system. It also prints a table showing what packages are used and what modules they correspond to.

Türkçe: Dependency'leri öğrenmek için bu JAR üzerinde `jdeps` çalıştırabiliriz. Önce komutu hiçbir option vermeden çalıştıralım. İlk iki satır, module system'e migration sırasında bir `requires` directive'iyle eklememiz gereken module'leri gösterir. Ayrıca hangi package'ların kullanıldığını ve bunların hangi module'lere karşılık geldiğini gösteren bir tablo yazdırır.

```
jdeps zoo.dino.jar
```

```
zoo.dino.jar -> java.base
zoo.dino.jar -> jdk.unsupported
zoo.dinos -> java.lang java.base
zoo.dinos -> java.time java.base
zoo.dinos -> java.util java.base
zoo.dinos -> sun.misc JDK internal API (jdk.unsupported)
```

English: Note that `java.base` is always included. It also says which modules contain classes used by the JAR. If we run in summary mode, we only see just the first part where `jdeps` lists the modules. There are two formats for the summary flag:

Türkçe: `java.base` her zaman eklendiğini unutmayın. Çıktı, JAR'ın kullandığı class'ların hangi module'lerde bulunduğunu da gösterir. Summary mode kullanırsak yalnızca `jdeps`in module'leri listelediği ilk bölümü görürüz. Summary flag'in iki biçimi vardır:

```
jdeps -s zoo.dino.jar
jdeps --summary zoo.dino.jar
```

```
zoo.dino.jar -> java.base
zoo.dino.jar -> jdk.unsupported
```

English: For a real project, the dependency list could include dozens or even hundreds of packages. It's useful to see the summary of just the modules. This approach also makes it easier to see whether `jdk.unsupported` is in the list.

Türkçe: Gerçek bir projede dependency listesi onlarca, hatta yüzlerce package içerebilir. Yalnızca module özetini görmek yararlıdır. Bu yaklaşım `jdk.unsupported`in listede bulunup bulunmadığını görmeyi de kolaylaştırır.

English: There is also a `--module-path` option that you can use if you want to look for modules outside the JDK. Unlike other commands, there is no short form for this option on `jdeps`.

Türkçe: JDK dışındaki module'lerde arama yapmak için `--module-path` option'ı da kullanılabilir. Diğer komutlardan farklı olarak bu option'ın `jdeps` için kısa biçimi yoktur.

English: You might have noticed that `jdk.unsupported` is not in the list of modules you saw in Table 12.8. It's special because it contains internal libraries that developers in previous versions of Java were discouraged from using, although many people ignored this warning. You should not reference it, as it may disappear in future versions of Java.

Türkçe: `jdk.unsupported`ın Tablo 12.8'deki module listesinde bulunmadığına dikkat etmiş olabilirsiniz. Bu özel module, önceki Java sürümlerinde developer'ların kullanmaması tavsiye edilen internal library'leri içerir; ancak birçok kişi bu uyarıyı görmezden gelmiştir. Gelecekteki Java sürümlerinde kaldırılabilirdiğinden ona doğrudan reference vermemelisiniz.

Using the `--jdk-internals` Flag

English: The `jdeps` command has an option to provide details about these unsupported APIs. The output looks something like this:

Türkçe: `jdeps`, desteklenmeyen bu API'lerle ilgili ayrıntı veren bir option'a sahiptir. Çıktı şuna benzer:

```
jdeps --jdk-internals zoo.dino.jar
```

```
zoo.dino.jar -> jdk.unsupported
zoo.dinos.Animatronic -> sun.misc.Unsafe
  JDK internal API (jdk.unsupported)
Warning: <omitted warning>
JDK Internal API          Suggested Replacement
sun.misc.Unsafe See http://openjdk.java.net/jeps/260
```

English: The `--jdk-internals` option lists any classes you are using that call an internal API along with which API. At the end, it provides a table suggesting what you should do about it. If you wrote the code calling the internal API, this message is useful. If not, the message would be useful to the team that did write the code. You, on the other hand, might need to update or replace that JAR file entirely with one that fixes the issue. Note that `-jdkinternals` is equivalent to `--jdk-internals`.

Türkçe: `--jdk-internals`, internal API çağıran class'ları ve kullanılan API'yi listeler. Çıktının sonunda ne yapılabileceğini öneren bir tablo sunar. Internal API çağrısını siz yazdıysanız bu mesaj yararlıdır. Aksi takdirde, mesaj kodu yazan ekip için yararlı olacaktır. Öte yandan, bu JAR dosyasını tamamen sorunu gideren bir dosyayla güncellemeniz veya değiştirmeniz gerekebilir. `-jdkinternals`'ın `--jdk-internals` ile eşdeğer olduğunu unutmayın.

English: About `sun.misc.Unsafe` Prior to the Java Platform Module System, classes had to be public if you wanted them to be used outside the package. It was reasonable to use the class in JDK code since that is low-level code that is already tightly coupled to the JDK. Since it was needed in multiple

Türkçe: `sun.misc.Unsafe` hakkında: Java Platform Module System'den önce bir class'ın package dışından kullanılabilmesi için `public` olması gerekiyordu. Düşük düzeydeki JDK kodu zaten JDK'ye sıkı bağlı olduğundan class'ın burada kullanılması makuldü. Birden çok

English: packages, the class was made public. Sun even named it `Unsafe`, figuring that would prevent anyone from using it outside the JDK.

Türkçe: package tarafından gerekli olduğu için class `public` yapılmıştı. Sun, JDK dışında kullanılmasını caydıracağını düşünerek ona `Unsafe` adını bile verdi.

English: However, developers are clever and used the class since it was available. A number of widely used open source libraries started using `Unsafe`. While it is quite unlikely that you are using this class in your project directly, you probably use an open source library that is using it.

Türkçe: Ancak developer'lar erişilebilir olan bu class'ı kullanmanın yollarını buldu ve yaygın birçok open-source library `Unsafe` kullanmaya başladı. Projenizde bu class'ı doğrudan kullanmanız pek olası değildir; fakat bağımlı olduğunuz bir library onu kullanıyor olabilir.

English: The `jdeps` command allows you to look at these JARs to see whether you will have any problems when Oracle finally prevents the usage of this class. If you find any uses, you can look at whether there is a later version of the JAR that you can upgrade to.

Türkçe: `jdeps`, Oracle gelecekte bu class'ın kullanımını engellediğinde sorun yaşayıp yaşamayacağınızı görmek için JAR'ları incelemenizi sağlar. Böyle bir kullanım bulursanız JAR'ın upgrade edebileceğiniz daha yeni bir sürümü olup olmadığına bakabilirsiniz.

Using Module Files with `jmod`

English: The final command you need to know for the exam is `jmod`. You might think a JMOD file is a Java module file. Not quite. Oracle recommends using JAR files for most modules. JMOD files are recommended only when you have native libraries or something that can't go inside a JAR file. This is unlikely to affect you in the real world.

Türkçe: Sınav için bilmeniz gereken son komut `jmod`dur. JMOD dosyasının sıradan bir Java module dosyası olduğunu düşünebilirsiniz; ancak tam olarak öyle değildir. Oracle çoğu modül için JAR kullanılmasını, JMOD'un ise yalnızca native library veya JAR içine yerleştirilemeyen içerik bulunduğu kullanıldığını önerir. Gerçek projelerde bununla karşılaşma olasılığınız düşüktür.

English: The most important thing to remember is that `jmod` is only for working with the JMOD files. Conveniently, you don't have to memorize the syntax for `jmod`. Table 12.9 lists the common modes.

Türkçe: Hatırlanması gereken en önemli nokta, `jmod`un yalnızca JMOD dosyalarıyla çalıştığıdır. `jmod` syntax'ını ezberlemek zorunda değilsiniz. Tablo 12.9 yaygın mode'ları listeler.

English: TABLE 12.9 — Modes using `jmod`

Columns: Operation; description.

`create`: Creates a JMOD file.

`extract`: Extracts all files from a JMOD file; works like unzipping.

`describe`: Prints module details such as `requires`.

`list`: Lists all files in a JMOD file.

`hash`: Prints or records hashes.

Türkçe: TABLO 12.9 — `jmod` mode'ları

Sütunlar: Operation; açıklama.

`create`: Bir JMOD dosyası oluşturur.

`extract`: JMOD içindeki bütün dosyaları çıkarır; unzip işlemine benzer.

`describe`: `requires` gibi module ayrıntılarını yazdırır.

`list`: JMOD içindeki bütün dosyaları listeler.

`hash`: Hash değerlerini yazdırır veya kaydeder.

Creating Java Runtimes with jlink

English: One of the benefits of modules is being able to supply just the parts of Java you need. Our zoo example from the beginning of the chapter doesn't have many dependencies. If the user

Türkçe: Module'lerin avantajlarından biri, Java'nın yalnızca gereken parçalarını dağıtabilmektir. Chapter başındaki zoo örneğimizin çok fazla dependency'si yoktur. Eğer kullanıcı

English: already doesn't have Java or is on a device without much memory, downloading a JDK that is over 150 MB is a big ask. Let's see how big the package actually needs to be! This command creates our smaller distribution:

Türkçe: zaten Java'ya sahip değilse veya belleği sınırlı bir cihaz kullanıyorsa 150 MB'tan büyük bir JDK indirmek ciddi bir yükür. Dağıtımın gerçekte ne kadar büyük olması gerektiğine bakalım. Bu komut daha küçük dağıtımımızı oluşturur:

```
jlink --module-path mods --add-modules zoo.animal.talks --output zooApp
```

English: First we specify where to find the custom modules with `-p` or `--module-path`. Then we specify our module names with `--add-modules`. This will include the dependencies it requires as long as they can be found. Finally, we specify the folder name of our smaller JDK with `--output`.

Türkçe: Önce custom module'lerin konumunu `-p` veya `--module-path` ile belirtiriz. Ardından module adlarını `--add-modules` ile veririz. Gerekli dependency'ler bulunabildiği sürece bunlar da eklenir. Son olarak küçük JDK'nin output directory adını `--output` ile belirtiriz.

English: The output directory contains the `bin`, `conf`, `include`, `legal`, `lib`, and `man` directories along with a release file. These should look familiar as you find them in the full JDK as well.

Türkçe: Output directory; `bin`, `conf`, `include`, `legal`, `lib` ve `man` directory'leriyle birlikte bir `release` dosyası içerir. Bunlar full JDK'de de bulunduğu için tanıdık görünmelidir.

English: When we run this command and zip up the `zooApp` directory, the file is only 15 MB. This is an order of magnitude smaller than the full JDK. Where did this space savings come from? There are many modules in the JDK we don't need. Additionally, development tools like `javac` don't need to be in a runtime distribution.

Türkçe: Komutu çalıştırıp `zooApp` directory'sini ziplediğimizde dosya yalnızca 15 MB olur. Bu, full JDK'den yaklaşık bir büyüklük mertebesi daha küçüktür. Tasarruf, ihtiyaç duymadığımız çok sayıdaki JDK module'ünün ve `javac` gibi development tool'ların runtime distribution'a alınmamasından gelir.

English: There are a lot more items to customize this process that you don't need to know for the exam. For example, you can skip generating the help documentation and save even more space.

Türkçe: Sınav için bilmeniz gerekmeyen bu süreci özelleştirmek için çok daha fazla öğe var. Örneğin, yardım dokümantasyonunu oluşturmayı atlayabilir ve daha da fazla yer tasarrufu sağlayabilirsiniz.

Reviewing Command-Line Options

English: This section presents a number of tables that cover what you need to know about running command-line options for the exam.

Türkçe: Bu bölüm, sınav için bilmeniz gereken command-line operation ve option'ları bir dizi tabloyla özetler.

English: Table 12.10 shows the command-line operations you should expect to encounter on the exam. There are many more options in the documentation. For example, there is a `--module` option on `javac` that limits compilation to that module. Luckily, you don't need to know those for the exam.

Türkçe: Tablo 12.10 sınavda karşılaşılabileceğiniz command-line operation'ları gösterir. Documentation'da çok daha fazla option bulunur. Örneğin `javac`, compilation'ı belirli bir module ile sınırlayan `--module` option'ına sahiptir; ancak bunu sınav için bilmeniz gerekmez.

English: TABLE 12.10 — Comparing command-line operations (Part 1 of 2)

Columns: Description; syntax.

Compile nonmodular code: `javac -cp classpath -d directory classesToCompile`; `javac --class-path classpath -d directory classesToCompile`; `javac -classpath classpath -d directory classesToCompile`.

Run nonmodular code: `java -cp classpath package.className`; `java -classpath classpath package.className`; `java --class-path classpath package.className`.

Türkçe: TABLO 12.10 — Command-line operation karşılaştırması (1/2)

Sütunlar: Açıklama; syntax.

Nonmodular kodu derleme: `javac -cp classpath -d directory classesToCompile`; `javac --class-path classpath -d directory classesToCompile`; `javac -classpath classpath -d directory classesToCompile`.

Nonmodular kodu çalıştırma: `java -cp classpath package.className`; `java -classpath classpath package.className`; `java --class-path classpath package.className`.

English: TABLE 12.10 — Comparing command-line operations (continued, Part 2 of 2)

Compile a module: ``javac -p moduleFolderName -d directory classesToCompileIncludingModuleInfo; javac --module-path moduleFolderName -d directory classesToCompileIncludingModuleInfo``.

Run a module: ``java -p moduleFolderName -m moduleName/package.className; java --module-path moduleFolderName --module moduleName/package.className``.

Describe a module: `java -p moduleFolderName -d moduleName; java --module-path moduleFolderName --describe-module moduleName; jar --file jarName --describe-module; jar -f jarName -d.`

List available modules: `java --module-path moduleFolderName --list-modules; java -p moduleFolderName --list-modules; java --list-modules.`

View dependencies: `jdeps -summary --module-path moduleFolderName jarName; jdeps -s --module-path moduleFolderName jarName; jdeps --jdk-internals jarName; jdeps -jdkinternals jarName.`

Show module resolution: ``java --show-module-resolution -p moduleFolderName -m moduleName; java --show-module-resolution --module-path moduleFolderName --module moduleName``.

Create a runtime image: ``jlink -p moduleFolderName --add-modules moduleName --output zooApp; jlink --module-path moduleFolderName --add-modules moduleName --output zooApp``.

TABLE 12.11 lists the `javac` options, TABLE 12.12 the `java` options, TABLE 12.13 the `jar` options, TABLE 12.14 the `jdeps` options, and TABLE 12.15 the `jlink` options.

Türkçe: TABLO 12.10 — Command-line operation karşılaştırması (devam, 2/2)

Module derleme: ``javac -p moduleFolderName -d directory classesToCompileIncludingModuleInfo; javac --module-path moduleFolderName -d directory classesToCompileIncludingModuleInfo``.

Module çalıştırma: ``java -p moduleFolderName -m moduleName/package.className; java --module-path moduleFolderName --module moduleName/package.className``.

Module tanımlama: `java -p moduleFolderName -d moduleName; java --module-path moduleFolderName --describe-module moduleName; jar --file jarName --describe-module; jar -f jarName -d.`

Kullanılabilir module'leri listeleme: ``java --module-path moduleFolderName --list-modules; java -p moduleFolderName --list-modules; java --list-modules``.

Dependency'leri görüntüleme: `jdeps -summary --module-path moduleFolderName jarName; jdeps -s --module-path moduleFolderName jarName; jdeps --jdk-internals jarName; jdeps -jdkinternals jarName.`

Module resolution gösterme: ``java --show-module-resolution -p moduleFolderName -m moduleName; java --show-module-resolution --module-path moduleFolderName --module moduleName``.

Runtime image oluşturma: ``jlink -p moduleFolderName --add-modules moduleName --output zooApp; jlink --module-path moduleFolderName --add-modules moduleName --output zooApp``.

TABLO 12.11 `javac`, TABLO 12.12 `java`, TABLO 12.13 `jar`, TABLO 12.14 `jdeps` ve TABLO 12.15 `jlink` option'larını ayrı ayrı özetler.

English: TABLE 12.11 — Options you need to know for the exam: `javac`

`-cp <classpath> / -classpath <classpath> / --class-path <classpath>`: Location of JARs in a nonmodular program.

`-d <dir>`: Directory in which to place generated class files.

`-p <path> / --module-path <path>`: Location of JARs in a modular program.

Türkçe: TABLO 12.11 — Sınav için bilinmesi gereken `javac` option'ları

`-cp <classpath> / -classpath <classpath> / --class-path <classpath>`: Nonmodular programdaki JAR'ların konumu.

`-d <dir>`: Üretilen class dosyalarının yazılacağı directory.

`-p <path> / --module-path <path>`: Modular programdaki JAR'ların konumu.

English: TABLE 12.12 — Options you need to know for the exam: `java`

- `-p <path> / --module-path <path>`: Location of JARs in a modular program.
- `-m <name> / --module <name>`: Module name to run.
- `-d / --describe-module`: Describes module details.
- `--list-modules`: Lists observable modules without running a program.
- `--show-module-resolution`: Shows modules while running a program.

Türkçe: TABLO 12.12 — Sınav için bilinmesi gereken `java` option'ları

- `-p <path> / --module-path <path>`: Modular programdaki JAR'ların konumu.
- `-m <name> / --module <name>`: Çalıştırılacak module adı.
- `-d / --describe-module`: Module ayrıntılarını tanımlar.
- `--list-modules`: Programı çalıştırmadan observable module'leri listeler.
- `--show-module-resolution`: Program çalışırken module'leri gösterir.

English: TABLE 12.13 — Options you need to know for the exam: `jar` (Part 1 of 2)

- `-c / --create`: Creates a new JAR file.
- `-v / --verbose`: Prints details while working with JAR files.
- `-f / --file`: Specifies the JAR filename.

Türkçe: TABLO 12.13 — Sınav için bilinmesi gereken `jar` option'ları (1/2)

- `-c / --create`: Yeni bir JAR dosyası oluşturur.
- `-v / --verbose`: JAR dosyalarıyla çalışırken ayrıntıları yazdırır.
- `-f / --file`: JAR filename'ini belirtir.

English: TABLE 12.13 — Options you need to know for the exam: `jar` (continued, Part 2 of 2)

- `-C`: Directory containing files to be used to create the JAR.
- `-d / --describe-module`: Describes module details.

Türkçe: TABLO 12.13 — Sınav için bilinmesi gereken `jar` option'ları (devam, 2/2)

- `-C`: JAR oluşturmak için kullanılacak dosyaları içeren directory.
- `-d / --describe-module`: Module ayrıntılarını tanımlar.

English: TABLE 12.14 — Options you need to know for the exam: `jdeps`

- `--module-path <path>`: Location of JARs in a modular program.
- `-s / --summary`: Summarizes the output.
- `--jdk-internals / -jdkinternals`: Lists uses of internal APIs.

Türkçe: TABLO 12.14 — Sınav için bilinmesi gereken `jdeps` option'ları

- `--module-path <path>`: Modular programdaki JAR'ların konumu.
- `-s / --summary`: Çıktıyı özetler.
- `--jdk-internals / -jdkinternals`: Internal API kullanımlarını listeler.

English: TABLE 12.15 — Options you need to know for the exam: `jlink`

- `-p <path> / --module-path <path>`: Location of JARs in a modular program.
- `--add-modules`: List of modules to package.
- `--output`: Name of the output directory.

Türkçe: TABLO 12.15 — Sınav için bilinmesi gereken `jlink` option'ları

- `-p <path> / --module-path <path>`: Modular programdaki JAR'ların konumu.
- `--add-modules`: Paketlenecek module listesi.
- `--output`: Output directory'nin adı.

Comparing Types of Modules

English: All the modules we've used so far in this chapter are called named modules. There are two other types of modules: automatic modules and unnamed modules. In this section, we describe these three types of modules. On the exam, you will need to be able to compare them.

Türkçe: Bu chapter'da şimdiye kadar kullandığımız module'lerin tamamı named module'dür. Diğer iki tür automatic module ve unnamed module'dür. Bu bölüm üç türü açıklar; sınavda bunları karşılaştırabilmeniz gerekir.

Named Modules

English: A named module is one containing a `module-info.java` file. To review, this file appears in the root of the JAR alongside one or more packages. Unless otherwise specified, a module is a named module. Named modules appear on the module path rather than the classpath. Later, you learn what happens if a JAR containing a `module-info.java` file is on the classpath. For now, just know it is not considered a named module because it is not on the module path.

Türkçe: Named module, `module-info.java` içeren bir module'dür. Bu dosya bir veya daha fazla package ile birlikte JAR'ın root'unda bulunur. Aksi belirtilmedikçe "module" sözcüğü named module anlamındadır. Named module'ler classpath yerine module path üzerinde bulunur. `module-info.java` içeren bir JAR classpath'e konursa module path üzerinde olmadığı için named module sayılmaz.

English: As a way of remembering this, a named module has the name inside the `module-info.java` file and is on the module path.

Türkçe: Hatırlamak için şu iki özelliği birlikte düşünün: Named module'ün adı `module-info.java` içinde tanımlıdır ve kendisi module path üzerinde bulunur.

English: Remember from Chapter 7, "Beyond Classes," that the only way for subclasses of sealed classes to be in a different package is to be within the same-named module.

Türkçe: Chapter 7, "Beyond Classes" bölümünden hatırlayın: sealed class'ın subclass'ı farklı bir package'ta olarsa aynı named module içinde bulunmalıdır.

Automatic Modules

English: An automatic module appears on the module path but does not contain a `module-info.java` file. It is simply a regular JAR file that is placed on the module path and gets treated as a module.

Türkçe: Automatic module, module path üzerinde bulunan fakat `module-info.java` içermeyen normal bir JAR'dır; module path'e konduğu için module olarak değerlendirilir.

English: As a way of remembering this, Java automatically determines the module name. The code referencing an automatic module treats it as if there is a `module-info.java` file present. It automatically exports all packages. It also determines the module name. How does it determine the module name, you ask? Excellent question.

Türkçe: Java automatic module'ün adını otomatik belirler. Bu module'e reference veren kod, `module-info.java` varmış gibi davranır. Automatic module bütün package'larını otomatik olarak export eder. Peki module adı nasıl belirlenir?

English: To answer this, we need to provide a bit of history on JAR files and module adoption. Every JAR file contains a special folder called META-INF and, within it, a text file called MANIFEST.MF. It can be created automatically when the JAR is created or by hand by the JAR's author. Getting back to modules, many Java libraries weren't quite ready to modularize when the feature was introduced. The authors were encouraged to declare the name they intended to use for the module by adding a property named Automatic-Module-Name into their MANIFEST.MF file.

Türkçe: Bunun için JAR'ların ve module adoption'ın geçmişine bakmak gerekir. Her JAR, META-INF directory'si içinde MANIFEST.MF adlı bir metin dosyası içerir. Bu dosya JAR oluşturulurken otomatik üretilir veya JAR'ın yazarı tarafından elle hazırlanabilir. Module özelliği geldiğinde birçok library henüz modularize edilmeye hazır değildi. Yazarların, ileride kullanmayı planladıkları module adını MANIFEST.MF içine Automatic-Module-Name property'si olarak eklemeleri teşvik edildi.

English: About the MANIFEST.MF File A JAR file contains a special text file called META-INF/MANIFEST.MF that contains information about the JAR. It's been around significantly longer than modules—since the early days of Java and JARs, to be exact. The figure shows how the manifest fits into the directory structure of a JAR file.

Türkçe: MANIFEST.MF dosyası hakkında: Bir JAR, kendisiyle ilgili bilgileri içeren META-INF/MANIFEST.MF adlı özel bir metin dosyasına sahiptir. Bu dosya modüllerden çok daha eskidir; Java ve JAR'ların ilk dönemlerinden beri kullanılır. Şekil, manifest dosyasının JAR dizin yapısındaki yerini gösterir.

English: zoo META-INF MANIFEST.MF sales holiday data The manifest contains extra information about the JAR file. For example, it often contains the version of Java used to build the JAR file. For command-line programs, the class with the main() method is commonly specified.

Türkçe: zoo META-INF MANIFEST.MF sales holiday data dizilimindeki manifest, JAR hakkında ek bilgiler içerir. Örneğin çoğu zaman JAR'ı build etmekte kullanılan Java sürümünü, command-line programlarında ise main() metodunu içeren sınıfı belirtir.

English: Each line in the manifest is a key/value pair separated by a colon. You can think of the manifest as a map of property names and values. The default manifest in Java 17 looks like this:

Türkçe: Manifest içindeki her satır colon ile ayrılmış bir key/value pair'dir. Manifest'i property adları ve değerlerinden oluşan bir map gibi düşünebilirsiniz. Java 17'nin default manifest'i şöyledir:

English: Manifest-Version: 1.0 Created-By: 17 (Oracle Corporation)

Türkçe: Manifest-Version: 1.0 Created-By: 17 (Oracle Corporation)

English: Specifying a single property in the manifest allowed library providers to make things easier for applications that wanted to use their library in a modular application. You can think of it as a promise that when the library becomes a named module, it will use the specified module name.

Türkçe: Manifest'te tek bir property belirtmek, library provider'ların modüler uygulamalara geçişi kolaylaştırmasını sağladı. Bunu, library ileride named module olduğunda belirtilen module adını kullanacağına ilişkin bir taahhüt gibi düşünebilirsiniz.

English: If the JAR file does not specify an automatic module name, Java will still allow you to use it in the module path. In this case, Java will determine the module name for you. We'd say that this happens automatically, but the joke is probably wearing thin by now.

Türkçe: JAR bir automatic module adı belirtmese de Java onun module path üzerinde kullanılmasına izin verir ve module adını kendisi belirler.

English: Java determines the automatic module name by basing it on the filename of the JAR file. Let's go over the rules by starting with an example. Suppose we have a JAR file named `holiday-calendar-1.0.0.jar`.

Türkçe: Java automatic module adını JAR dosyasının adına göre belirler. Kuralları bir örnekle inceleyelim: `holiday-calendar-1.0.0.jar` adlı bir JAR bulunduğunu varsayalım.

English: First Java will remove the extension `.jar` from the name. Then Java will remove the version from the end of the JAR filename. This is important because we want module names to be consistent. Having a different automatic module name every time you upgraded to a new version would not be good! After all, this would force you to change the module declaration of your nice, clean, modularized application every time you pulled in a later version of the holiday calendar JAR.

Türkçe: Java önce `.jar` uzantısını, sonra JAR dosya adının sonundaki sürüm bilgisini kaldırır. Böylece module adı sürümler arasında sabit kalır; aksi halde library her güncellendiğinde modüler uygulamanın module declaration'ını değiştirmek gerekirdi.

English: Removing the version and extension gives us `holiday-calendar`. This leaves us with a problem. Dashes (-) are not allowed in module names. Java solves this problem by converting any special characters in the name to dots (.). As a result, the module name is `holiday.calendar`. Any characters other than letters and numbers are considered special characters in this replacement. Finally, any adjacent dots or leading/trailing dots are removed.

Türkçe: Version ve extension kaldırılınca `holiday-calendar` kalır. Dash (-) module adında kullanılamaz. Java harf ve rakam dışındaki special character'ları dot'a (.) çevirir; böylece module adı `holiday.calendar` olur. Ardışık dot'lar birleştirilir ve baştaki ya da sondaki dot'lar kaldırılır.

English: Since that's a number of rules, let's review the algorithm in a list for determining the name of an automatic module: ■ If the MANIFEST.MF specifies an Automatic-Module-Name, use that. Otherwise, proceed with the remaining rules. ■ Remove the file extension from the JAR name. ■ Remove any version information from the end of the name. A version is digits and dots with possible extra information at the end: for example, `-1.0.0` or `-1.0-RC`. ■ Replace any remaining characters other than letters and numbers with dots. ■ Replace any sequences of dots with a single dot. ■ Remove the dot if it is the first or last character of the result.

Türkçe: Automatic module adı şu algoritmayla belirlenir: `MANIFEST.MF` içinde

`Automatic-Module-Name` varsa doğrudan o değer kullanılır. Yoksa JAR adından dosya uzantısı ve sondaki sürüm bilgisi kaldırılır. Sürüm; `-1.0.0` veya `-1.0-RC` gibi, rakam ve noktalardan sonra ek bilgi içerebilir. Kalan harf ve rakam dışı karakterler noktaya çevrilir, ardışık noktalar tek noktaya indirilir ve baştaki ya da sondaki nokta kaldırılır.

English: Table 12.16 shows how to apply these rules to two examples where there is no automatic module name specified in the manifest.

Türkçe: Tablo 12.16, bu kuralların manifest içinde automatic module adı belirtilmeyen iki örneğe nasıl uygulanacağını gösterir.

English: TABLE 12.16 — Practicing with automatic module names

Step — Description — Example 1 — Example 2

- 1 — Beginning JAR name — `commons2-x-1.0.0-SNAPSHOT.jar` — `mod_$-1.0.jar.jar`
- 2 — Remove file extension — `commons2-x-1.0.0-SNAPSHOT` — `mod_$-1.0.jar`
- 3 — Remove version information — `commons2-x` — `mod_$`
- 4 — Replace special characters — `commons2.x` — `mod..`
- 5 — Replace sequences of dots — `commons2.x` — `mod.`
- 6 — Remove leading/trailing dots; the result is the automatic module name — `commons2.x` — `mod`

Türkçe: TABLO 12.16 — Automatic module adlarıyla alıştırma

Adım — Açıklama — Örnek 1 — Örnek 2

- 1 — Başlangıç JAR adı — `commons2-x-1.0.0-SNAPSHOT.jar` — `mod_$-1.0.jar.jar`
- 2 — Dosya uzantısını kaldır — `commons2-x-1.0.0-SNAPSHOT` — `mod_$-1.0.jar`
- 3 — Sürüm bilgisini kaldır — `commons2-x` — `mod_$`
- 4 — Özel karakterleri değiştir — `commons2.x` — `mod..`
- 5 — Ardışık noktaları tek noktaya indir — `commons2.x` — `mod.`
- 6 — Baştaki/sondaki noktaları kaldır; sonuç automatic module adıdır — `commons2.x` — `mod`

English: While the algorithm for creating automatic module names does its best, it can't always come up with a good name. For example, `1.2.0-calendar-1.2.2-good-1.jar` isn't conducive. Luckily, such names are rare and out of scope for the exam.

Türkçe: Automatic module adı oluşturma algoritması her zaman kullanışlı bir ad üretemez. Örneğin

`1.2.0-calendar-1.2.2-good-1.jar` elverişli bir dosya adı değildir. Böyle örnekler nadirdir ve sınav kapsamı dışındadır.

Unnamed Modules

English: An unnamed module appears on the classpath. Like an automatic module, it is a regular JAR. Unlike an automatic module, it is on the classpath rather than the module path. This means an unnamed module is treated like old code and a second-class citizen to modules.

Türkçe: Unnamed module classpath üzerinde bulunan normal bir JAR'dır. Automatic module'den farklı olarak module path'te değil classpath'tedir. Bu nedenle eski Java kodu gibi ve module system içinde ikinci sınıf bir öge olarak değerlendirilir.

English: An unnamed module does not usually contain a `module-info.java` file. If it happens to contain one, that file will be ignored since it is on the classpath.

Türkçe: Bir unnamed module genellikle bir `module-info.java` dosyası içermez. Eğer bir tane içeriyorsa, bu dosya classpath üzerinde olduğu için göz ardı edilir.

English: Unnamed modules do not export any packages to named or automatic modules. The unnamed module can read from any JARs on the classpath or module path. You can think of an unnamed module as code that works the way Java worked before modules. Yes, we know it is confusing for something that isn't really a module to have the word module in its name.

Türkçe: Unnamed module, named veya automatic module'lere hiçbir package export etmez. Buna karşılık classpath ya da module path üzerindeki JAR'ları okuyabilir. Unnamed module'ü Java'nın module'lerden önceki çalışma biçimindeki kod olarak düşünebilirsiniz.

Reviewing Module Types

English: You can expect to get questions on the exam comparing the three types of modules. Please study Table 12.17 thoroughly and be prepared to answer questions about these items in any combination. A key point to remember is that code on the classpath can access the module path. By contrast, code on the module path is unable to read from the classpath.

Türkçe: Sınavda üç modül türünü farklı yönlerden karşılaştıran sorularla karşılaşabilirsiniz; bu nedenle Tablo 12.17'yi iyi öğrenin. Temel ayrım şudur: Classpath üzerindeki kod module path'e erişebilir, fakat module path üzerindeki kod classpath'ten okuyamaz.

English: TABLE 12.17 — Properties of module types

Property — Named — Automatic — Unnamed

Contains a `module-info.java` file? — Yes — No — Ignored if present

Packages exported to other modules — Those listed in `module-info.java` — All packages — No packages

Readable by other modules on the module path? — Yes — Yes — No

Readable by other JARs on the classpath? — Yes — Yes — Yes

Türkçe: TABLO 12.17 — Module türlerinin özellikleri

Özellik — Named — Automatic — Unnamed

`module-info.java` dosyası içerir mi? — Evet — Hayır — Varsa göz ardı edilir

Diğer module'lere export edilen package'lar — `module-info.java` içinde belirtilenler — Bütün package'lar — Hiçbir package

Module path üzerindeki diğer module'ler tarafından okunabilir mi? — Evet — Evet — Hayır

Classpath üzerindeki diğer JAR'lar tarafından okunabilir mi? — Evet — Evet — Evet

Java 17 teknik düzeltme: Kaynak metin ve Tablo 12.17, unnamed module için export ile readability kavramlarını birleştirir. JLS 17 §7.7.5'e göre unnamed module ilişkili bütün package'ları her module'e export ve open eder. Ordinary explicit named module'ün bu package'lara erişememesinin nedeni, default olarak unnamed module'ü **okumamasıdır**. Automatic module ise JVM'deki unnamed module'leri okuyabilir. Bu nedenle tablonun formal “Packages exported” hücresi unnamed module için **All packages** olmalıdır.

Migrating an Application

English: Many applications were not designed to use the Java Platform Module System because they were written before it was created or chose not to use it. Ideally, they were at least designed with projects instead of as a big ball of mud. This section gives you an overview of strategies for migrating an existing application to use modules. We cover ordering modules, bottom-up migration, top-down migration, and how to split up an existing project.

Türkçe: Birçok uygulama JPMS oluşturulmadan önce yazıldığı veya JPMS kullanmamayı seçtiği için module system'e göre tasarlanmamıştır. Bu bölüm mevcut bir uygulamayı module'lere taşıma stratejilerine genel bakış sunar: dependency sırasını belirleme, bottom-up migration, top-down migration ve mevcut projeyi module'lere bölme.

English: Migrating Your Applications at Work The exam exists in a pretend universe where there are no open source dependencies and applications are very small. These scenarios make learning and discussing migration far easier. In the real world, applications have libraries that haven't been updated in 10 or more years, complex dependency graphs, and all sorts of surprises.

Türkçe: İş Yerindeki Uygulamaları Taşımak: Sınav soruları, open-source dependency'lerin bulunmadığı ve uygulamaların çok küçük olduğu varsayımsal bir dünyada geçer; böylece migration'ı öğrenmek ve tartışmak kolaylaşır. Gerçek uygulamalarda ise on yıldan uzun süredir güncellenmemiş library'ler, karmaşık dependency graph'ları ve beklenmedik sorunlar bulunabilir.

English: Note that you can use all the features of Java 17 without converting your application to modules (except the features in this module chapter, of course!). Please make sure you have a reason for migration and don't think it is required.

Türkçe: Uygulamanızı modüllere dönüştürmeden Java 17'nin tüm özelliklerini kullanabileceğinizi unutmayın (bu bölüm bölümündeki özellikler hariç!). Lütfen migration için bir nedeniniz olduğundan emin olun ve bunun gerekli olduğunu düşünmeyin.

English: This chapter does a great job teaching you what you need to know for the exam. However, it does not adequately prepare you to convert real applications to use modules. If you find yourself in that situation, consider reading *The Java Module System* by Nicolai Parlog (Manning Publications, 2019).

Türkçe: Bu bölüm sınav için gerekenleri öğretir; ancak gerçek uygulamaları module system'e taşımak için tek başına yeterli değildir. Böyle bir görevle karşılaşırsanız Nicolai Parlog'un *The Java Module System* (Manning Publications, 2019) kitabından yararlanabilirsiniz.

Determining the Order

English: Before we can migrate our application to use modules, we need to know how the packages and libraries in the existing application are structured. Suppose we have a simple application with three JAR files, as shown in Figure 12.14. The dependencies between projects form a graph. Both of the representations in Figure 12.14 are equivalent. The arrows show the dependencies by pointing from the project that will require the dependency to the one that makes it available. In the language of modules, the arrow will go from `requires` to `exports`.

Türkçe: Migration'dan önce mevcut uygulamadaki package ve library yapısını bilmemiz gerekir. Şekil 12.14'te üç JAR'dan oluşan basit bir uygulama vardır. Projeler arasındaki dependency'ler bir graph oluşturur ve şekildeki iki gösterim eşdeğerdir. Ok, bağımlılığı isteyen projeden onu sağlayan projeye yönelir; module terminolojisiyle `requires` tarafından `exports` tarafına gider.

English: FIGURE 12.14 — Determining the order. Dependency arrows: `chicken` → `nest`, `chicken` → `egg`, and `nest` → `egg`. The only valid top-to-bottom order is `chicken`, `nest`, `egg`.

Türkçe: Şekil 12.14 — Sıranın belirlenmesi. Dependency okları: `chicken` → `nest`, `chicken` → `egg` ve `nest` → `egg`. Tek geçerli top-to-bottom sıra `chicken`, `nest`, `egg` şeklindedir.

English: The right side of the diagram makes it easier to identify the top and bottom that topdown and bottom-up migration refer to. Projects that do not have any dependencies are at the bottom. Projects that do have dependencies are at the top.

Türkçe: Diyagramın sağ tarafı, top-down ve bottom-up migration'daki üst ve alt düzeyleri belirlemeyi kolaylaştırır. Herhangi bir bağımlılığı olmayan projeler en altta yer almaktadır. Bağımlılıkları olan projeler en üst sırada yer almaktadır.

English: In this example, there is only one order from top to bottom that honors all the dependencies. Figure 12.15 shows that the order is not always unique. Since two of the projects do not have an arrow between them, either order is allowed when deciding migration order.

Türkçe: Bu örnekte bütün dependency'lere uyan yalnızca bir top-to-bottom sıra vardır. Şekil 12.15 sıranın her zaman tek olmadığını gösterir. İki proje arasında ok yoksa migration sırası belirlenirken ikisinin yeri değişebilir.

English: FIGURE 12.15 — Determining the order when not unique. Dependency arrows: `chicken` → `egg` and `penguin` → `egg`; there is no dependency between `chicken` and `penguin`. The valid top-to-bottom orders are `chicken, penguin, egg` and `penguin, chicken, egg`.

Türkçe: Şekil 12.15 — Tek bir sıra bulunmadığında sıralamanın belirlenmesi. Dependency okları `chicken` → `egg` ve `penguin` → `egg` şeklindedir; `chicken` ile `penguin` arasında dependency yoktur. Geçerli top-to-bottom sıralar `chicken, penguin, egg` ve `penguin, chicken, egg` biçimindedir.

Exploring a Bottom-Up Migration Strategy

English: The easiest approach to migration is a bottom-up migration. This approach works best when you have the power to convert any JAR files that aren't already modules. For a bottom-up migration, you follow these steps:

Türkçe: En kolay migration yaklaşımı bottom-up migration'dır. Bu yöntem, henüz module olmayan bütün JAR'ları dönüştürme yetkiniz olduğunda en iyi sonucu verir. Şu adımlar izlenir:

English: 1. Pick the lowest-level project that has not yet been migrated. (Remember the way we

Türkçe: 1. Henüz migrate edilmemiş en alt düzey projeyi seçin. (Projeleri önceki bölümde

English: ordered them by dependencies in the previous section?)

Türkçe: dependency'lerine göre nasıl sıraladığımızı hatırlayın.)

English: 2. Add a `module-info.java` file to that project. Be sure to add any exports to expose any package used by higher-level JAR files. Also, add a `requires` directive for any modules this module depends on.

Türkçe: 2. Projeye `module-info.java` ekleyin. Üst düzey JAR'ların kullandığı paketler için gerekli `exports` directive'lerini, bu modülün bağımlı olduğu modüller için de `requires` directive'lerini ekleyin.

English: 3. Move this newly migrated named module from the classpath to the module path. 4. Ensure that any projects that have not yet been migrated stay as unnamed modules on the classpath.

Türkçe: 3. Yeni dönüştürülen named module'ü classpath'ten module path'e taşıyın. 4. Henüz migrate edilmemiş projeleri classpath üzerinde unnamed module olarak bırakın.

English: 5. Repeat with the next-lowest-level project until you are done.

Türkçe: 5. Tamamlanana kadar bir sonraki en alt düzey projeye tekrarlayın.

English: You can see this procedure applied to migrate three projects in Figure 12.16. Notice that each project is converted to a module in turn.

Türkçe: Şekil 12.16 bu yöntemin üç projeye uygulanışını gösterir. Her proje sırayla module'e dönüştürülür.

English: With a bottom-up migration, you are getting the lower-level projects in good shape. This makes it easier to migrate the top-level projects at the end. It also encourages care in what is exposed.

Türkçe: Bottom-up migration önce alt düzey projeleri düzenler; böylece en sondaki üst düzey projelerin taşınması kolaylaşır ve hangi API'lerin dışa açıldığı dikkatle belirlenir.

English: During migration, you have a mix of named modules and unnamed modules. The named modules are the lower-level ones that have been migrated. They are on the module path and not allowed to access any unnamed modules.

Türkçe: Migration sırasında named ve unnamed module'ler birlikte bulunur. Migrate edilmiş alt düzey named module'ler module path üzerindedir ve unnamed module'lere erişemez.

English: FIGURE 12.16 — Bottom-up migration

1: `chicken`, `nest`, and `egg` start on the classpath.

2: `egg` becomes a named module on the module path; `chicken` and `nest` remain unnamed modules on the classpath.

3: `nest` and `egg` are named modules on the module path; `chicken` remains an unnamed module on the classpath.

4: `chicken`, `nest`, and `egg` are all named modules on the module path.

Türkçe: Şekil 12.16 — Bottom-up migration

1: `chicken`, `nest` ve `egg` classpath üzerinde başlar.

2: `egg`, module path üzerinde named module olur; `chicken` ile `nest` classpath üzerinde unnamed module olarak kalır.

3: `nest` ile `egg`, module path üzerinde named module olur; `chicken` classpath üzerinde unnamed module olarak kalır.

4: `chicken`, `nest` ve `egg` modüllerinin tamamı module path üzerinde named module'dür.

English: The unnamed modules are on the classpath. They can access JAR files on both the classpath and the module path.

Türkçe: Unnamed module'ler classpath üzerindedir. Hem classpath hem module path üzerindeki JAR dosyalarına erişebilirler.

Exploring a Top-Down Migration Strategy

English: A top-down migration strategy is most useful when you don't have control of every JAR file used by your application. For example, suppose another team owns one project. They are just too busy to migrate. You wouldn't want this situation to hold up your entire migration.

Türkçe: Top-down migration, uygulamanın kullandığı bütün JAR'lar üzerinde kontrolünüz olmadığında kullanışlıdır. Örneğin başka bir ekibin yönettiği ve henüz migrate edemediği bir proje, bütün migration sürecini durdurmak zorunda kalmaz.

English: For a top-down migration, you follow these steps:

Türkçe: Top-down migration için şu adımlar izlenir:

English: 1. Place all projects on the module path. 2. Pick the highest-level project that has not yet been migrated. 3. Add a module-info.java file to that project to convert the automatic module into a named module. Again, remember to add any exports or requires directives. You can use the automatic module name of other modules when writing the requires directive since most of the projects on the module path do not have names yet.

Türkçe: 1. Bütün projeleri module path'e yerleştirin. 2. Henüz migrate edilmemiş en üst düzey projeyi seçin. 3. Automatic module'ü named module'e dönüştürmek için projeye `module-info.java` ekleyin; gereken `exports` ve `requires` directive'lerini de yazın. Module path üzerindeki projelerin çoğu henüz named module olmadığından `requires` içinde onların automatic module adlarını kullanabilirsiniz.

English: 4. Repeat with the next-highest-level project until you are done.

Türkçe: 4. Tamamlanana kadar bir sonraki en üst düzey projeye tekrarlayın.

English: You can see this procedure applied in order to migrate three projects in Figure 12.17. Notice that each project is converted to a module in turn.

Türkçe: Şekil 12.17 bu yöntemin üç projeye uygulanışını gösterir. Her proje sırayla module'e dönüştürülür.

English: FIGURE 12.17 — Top-down migration

1: `chicken`, `nest`, and `egg` start on the classpath.

2: All three move to the module path: `chicken` becomes a named module, while `nest` and `egg` are automatic modules.

3: `chicken` and `nest` are named modules; `egg` remains an automatic module.

4: `chicken`, `nest`, and `egg` are all named modules on the module path.

Türkçe: Şekil 12.17 — Top-down migration

1: `chicken`, `nest` ve `egg` classpath üzerinde başlar.

2: Üçü de module path'e taşınır; `chicken` named module, `nest` ile `egg` automatic module olur.

3: `chicken` ile `nest` named module'dür; `egg` automatic module olarak kalır.

4: `chicken`, `nest` ve `egg` modüllerinin tamamı module path üzerinde named module'dür.

English: With a top-down migration, you are conceding that all of the lower-level dependencies are not ready but that you want to make the application itself a module.

Türkçe: Top-down migration'da alt düzey dependency'lerin henüz hazır olmadığını kabul eder, buna rağmen uygulamanın kendisini module'e dönüştürsünüz.

English: During migration, you have a mix of named modules and automatic modules. The named modules are the higher-level ones that have been migrated. They are on the module path and have access to the automatic modules. The automatic modules are also on the module path.

Türkçe: Migration sırasında named ve automatic module'ler birlikte bulunur. Migrate edilmiş üst düzey named module'ler module path üzerindedir ve yine module path'teki automatic module'lere erişebilir.

English: Table 12.18 reviews what you need to know about the two main migration strategies. Make sure you know it well.

Türkçe: Tablo 12.18 iki ana migration stratejisi için bilinmesi gerekenleri özetler.

English: TABLE 12.18 — Comparing migration strategies

Category — Bottom-up — Top-down

Project that depends on all others — Unnamed module on the classpath — Named module on the module path

Project that has no dependencies — Named module on the module path — Automatic module on the module path

Türkçe: TABLO 12.18 — Migration stratejilerinin karşılaştırılması

Kategori — Bottom-up — Top-down

Bütün diğer projelere dependency'si olan proje — Classpath üzerinde unnamed module — Module path üzerinde named module

Hiçbir dependency'si olmayan proje — Module path üzerinde named module — Module path üzerinde automatic module

Splitting a Big Project into Modules

English: For the exam, you need to understand the basic process of splitting a big project into modules.

You won't be given a big project, of course. After all, there is only so much space to ask a question.

Luckily, the process is the same for a small project.

Türkçe: Sınav için büyük bir projeyi modüllere ayırmanın temel sürecini anlamanız gerekir. Soruda büyük bir proje verilemez; ancak süreç küçük bir projede de aynıdır.

English: Suppose you start with an application that has a number of packages. The first step is to break them into logical groupings and draw the dependencies between them. Figure 12.18 shows an imaginary system's decomposition. Notice that there are seven packages on both the left and right sides. There are fewer modules because some packages share a module.

Türkçe: Çok sayıda package içeren bir uygulamayla başladığınızı varsayalım. İlk adım, package'ları mantıksal gruplara ayırıp aralarındaki dependency'leri çizmektir. Şekil 12.18 varsayımsal bir sistemin decomposition'ını gösterir. Her iki tarafta da yedi package vardır; bazı package'lar aynı module'ü paylaştığı için module sayısı daha azdır.

English: FIGURE 12.18 — First attempt at decomposition

Before: `zoo.tickets.cash`, `zoo.tickets.coupons`, `zoo.tickets.credit`, `zoo.tickets.etickets`, `zoo.tickets.promos`, `zoo.tickets.printer`, and `zoo.tickets.type` are packages in one project.

After: `zoo.tickets.delivery` contains `zoo.tickets.etickets` and `zoo.tickets.printer`; `zoo.tickets.model` contains `zoo.tickets.type`; `zoo.tickets.discount` contains `zoo.tickets.coupons` and `zoo.tickets.promos`; `zoo.tickets.payment` contains `zoo.tickets.cash` and `zoo.tickets.credit`. The dependency arrows are: `zoo.tickets.delivery` → `zoo.tickets.model`, `zoo.tickets.discount`; `zoo.tickets.discount` → `zoo.tickets.delivery`, `zoo.tickets.model`, `zoo.tickets.payment`; `zoo.tickets.payment` → `zoo.tickets.delivery`, `zoo.tickets.model`. Therefore, `zoo.tickets.delivery` and `zoo.tickets.discount` form a cycle.

Türkçe: Şekil 12.18 — Decomposition için ilk deneme

Önce: `zoo.tickets.cash`, `zoo.tickets.coupons`, `zoo.tickets.credit`, `zoo.tickets.etickets`, `zoo.tickets.promos`, `zoo.tickets.printer` ve `zoo.tickets.type` tek bir projedeki package'lardır.

Sonra: `zoo.tickets.delivery`, `zoo.tickets.etickets` ile `zoo.tickets.printer`; `zoo.tickets.model`, `zoo.tickets.type`; `zoo.tickets.discount`, `zoo.tickets.coupons` ile `zoo.tickets.promos`u; `zoo.tickets.payment` ise `zoo.tickets.cash` ile `zoo.tickets.credit`i içerir. Dependency okları: `zoo.tickets.delivery` → `zoo.tickets.model`, `zoo.tickets.discount`; `zoo.tickets.discount` → `zoo.tickets.delivery`, `zoo.tickets.model`, `zoo.tickets.payment`; `zoo.tickets.payment` → `zoo.tickets.delivery`, `zoo.tickets.model`. Bu nedenle `zoo.tickets.delivery` ile `zoo.tickets.discount` bir cycle oluşturur.

English: There's a problem with this decomposition. Do you see it? The Java Platform Module System does not allow for cyclic dependencies. A cyclic dependency, or circular dependency, is when two things directly or indirectly depend on each other. If the `zoo.tickets.delivery` module requires the `zoo.tickets.discount` module, `zoo.tickets.discount` is not allowed to require the `zoo.tickets.delivery` module.

Türkçe: Bu decomposition'da bir sorun vardır: Java Platform Module System cyclic dependency'ye izin vermez. Cyclic (circular) dependency, iki öğenin doğrudan veya dolaylı biçimde birbirine bağımlı olmasıdır. `zoo.tickets.delivery`, `zoo.tickets.discount` modülünü `requires` ediyorsa `zoo.tickets.discount`, `zoo.tickets.delivery` modülünü `requires` edemez.

English: Now that we know that the decomposition in Figure 12.18 won't work, what can we do about it? A common technique is to introduce another module. That module contains the code that the other two modules share. Figure 12.19 shows the new modules without any cyclic dependencies. Notice the new module `zoo.tickets.etc`. We created new packages to put in that module. This allows the developers to put the common code in there and break the dependency. No more cyclic dependencies!

Türkçe: Şekil 12.18'deki decomposition çalışmadığı için yaygın bir çözüm olarak üçüncü bir module eklenir. Bu module, diğer iki module'ün paylaştığı kodu içerir. Şekil 12.19'daki yeni `zoo.tickets.etc` module'üne ortak kod için yeni package'lar yerleştirilmiş ve cyclic dependency kırılmıştır.

Failing to Compile with a Cyclic Dependency

English: It is extremely important to understand that Java will not allow you to compile modules that have circular dependencies. In this section, we look at an example leading to that compiler error.

Türkçe: Java'nın circular dependency içeren modüllerin derlenmesine izin vermediğini bilmek çok önemlidir. Bu bölümde söz konusu compiler error'a yol açan bir örnek incelenir.

English: FIGURE 12.19 — Removing the cyclic dependencies

Before: `zoo.tickets.delivery` contains `zoo.tickets.etickets` and `zoo.tickets.printer`; `zoo.tickets.model` contains `zoo.tickets.type`; `zoo.tickets.discount` contains `zoo.tickets.coupons` and `zoo.tickets.promos`; `zoo.tickets.payment` contains `zoo.tickets.cash` and `zoo.tickets.credit`.

After: The new `zoo.tickets.etech` module contains `zoo.tickets.electronic` and `zoo.tickets.email`; the other four modules retain their packages. Dependency arrows: `zoo.tickets.delivery` → `zoo.tickets.etech`, `zoo.tickets.model`; `zoo.tickets.etech` → `zoo.tickets.model`; `zoo.tickets.discount` → `zoo.tickets.etech`, `zoo.tickets.model`, `zoo.tickets.payment`; `zoo.tickets.payment` → `zoo.tickets.delivery`, `zoo.tickets.model`. The revised arrows no longer form a cycle.

Türkçe: Şekil 12.19 — Cyclic dependency'lerin kaldırılması

Önce: `zoo.tickets.delivery`, `zoo.tickets.etickets` ile `zoo.tickets.printer`; `zoo.tickets.model`, `zoo.tickets.type`; `zoo.tickets.discount`, `zoo.tickets.coupons` ile `zoo.tickets.promos`u; `zoo.tickets.payment` ise `zoo.tickets.cash` ile `zoo.tickets.credit`i içerir.

Sonra: Yeni `zoo.tickets.etech` modülü `zoo.tickets.electronic` ile `zoo.tickets.email` package'larını içerir; diğer dört modül package'larını korur. Dependency okları: `zoo.tickets.delivery` → `zoo.tickets.etech`, `zoo.tickets.model`; `zoo.tickets.etech` → `zoo.tickets.model`; `zoo.tickets.discount` → `zoo.tickets.etech`, `zoo.tickets.model`, `zoo.tickets.payment`; `zoo.tickets.payment` → `zoo.tickets.delivery`, `zoo.tickets.model`. Yeniden düzenlenen oklar artık cycle oluşturmaz.

English: Consider the `zoo.butterfly` module described here:

Türkçe: Şimdi aşağıda tanımlanan `zoo.butterfly` modülünü ele alalım:

```
// Butterfly.java
package zoo.butterfly;
import zoo.caterpillar.Caterpillar;
public class Butterfly {
    private Caterpillar caterpillar;
}
// module-info.java
module zoo.butterfly {
    exports zoo.butterfly;
    requires zoo.caterpillar;
}
```

English: We can't compile this yet as we need to build `zoo.caterpillar` first. After all, our butterfly requires it. Now we look at `zoo.caterpillar`:

Türkçe: Önce `zoo.caterpillar`ı build etmemiz gerektiği için bunu henüz derleyemeyiz; `zoo.butterfly` onu `requires` ile ister. Şimdi `zoo.caterpillara` bakalım:

```
// Caterpillar.java
package zoo.caterpillar;
import zoo.butterfly.Butterfly;
public class Caterpillar {
    Butterfly emergeCocoon() {
        // logic omitted
    }
}
```

```
// module-info.java
module zoo.caterpillar {
    exports zoo.caterpillar;
    requires zoo.butterfly;
}
```

Java 17 editör notu: Kaynak snippet'lerde cross-package type import'ları gösterilmemiştir. Yukarıdaki `import` satırları, örneğin yalnız intended module-cycle derleme hatasını ölçmesi için tamamlanmıştır.

English: We can't compile this yet as we need to build `zoo.butterfly` first. Uh oh! Now we have a stalemate. Neither module can be compiled. This is our circular dependency problem at work.

Türkçe: Önce `zoo.butterfly`'yi build etmemiz gerektiği için bunu da henüz derleyemeyiz. Bir çıkmaza girdik: İki modülden hiçbiri derlenemiyor. Cyclic dependency sorunu tam olarak budur.

English: This is one of the advantages of the module system. It prevents you from writing code that has a cyclic dependency. Such code won't even compile!

Türkçe: Bu module system'in avantajlarından biridir: cyclic dependency içeren kod yazılmasını engeller. Böyle bir kod derlenmez.

English: You might be wondering what happens if three modules are involved. Suppose module `ballA` requires module `ballB` and `ballB` requires module `ballC`. Can module `ballC` require module `ballA`? No. This would create a cyclic dependency. Don't believe us? Try drawing it. You can follow your pencil around the circle from `ballA` to `ballB` to `ballC` to `ballA` to... well, you get the idea. There are just too many balls in the air!

Türkçe: Üç modül olduğunda da kural değişmez. `ballA`, `ballB`'yi; `ballB` de `ballC`'yi `requires` etsin. `ballC`, `ballA`'yı `requires` edebilir mi? Hayır; bu, `ballA` → `ballB` → `ballC` → `ballA` biçiminde bir cyclic dependency oluşturur.

English: Java will still allow you to have a cyclic dependency between packages within a module. It enforces that you do not have a cyclic dependency between modules.

Türkçe: Java, aynı modül içindeki paketler arasında cyclic dependency bulunmasına izin verir; ancak modüller arasında cyclic dependency bulunmasını engeller.

Summary

English: The Java Platform Module System organizes code at a higher level than packages. Each module contains one or more packages and a module-info.java file. The java.base module is most common and is automatically supplied to all modules as a dependency.

Türkçe: Java Platform Module System, kodu paketlerden daha üst bir düzeyde düzenler. Her modül bir veya daha fazla paket ile bir `module-info.java` dosyası içerir. En yaygın modül olan `java.base`, bütün modüllere otomatik olarak bağımlılık şeklinde eklenir.

English: The process of compiling and running modules uses the `--module-path`, also known as `-p`. Running a module uses the `--module` option, also known as `-m`. The class to run is specified in the format `moduleName/className`.

Türkçe: Modülleri derlerken ve çalıştırırken `--module-path` ya da kısa biçimi `-p` kullanılır. Bir modülü çalıştırmak için `--module` ya da kısa biçimi `-m` kullanılır. Çalıştırılacak sınıf `moduleName/className` biçiminde belirtilir.

English: The module declaration file supports a number of directives. The `exports` directive specifies that a package should be accessible outside the module. It can optionally restrict that export to a specific module or list of modules. The `requires` directive is used when a module depends on code in another module. Additionally, `requires transitive` can be used when all modules that require one module should always require another. The `provides` and `uses` directives are used when sharing and consuming a service. Finally, the `opens` directive is used to allow access via reflection.

Türkçe: Module declaration dosyası çeşitli directive'leri destekler. `exports`, bir paketi modül dışından erişilebilir kılar ve isteğe bağlı olarak erişimi belirli modüllerle sınırlandırabilir. `requires`, bir modül başka bir modüldeki koda bağımlı olduğunda kullanılır. Bir bağımlılığın kendisine bağımlı modüllere de aktarılması gerekiyorsa `requires transitive` kullanılır. `provides` ve `uses` bir service'i sunmak ve tüketmek için, `opens` ise reflection yoluyla erişime izin vermek için kullanılır.

English: Both the `java` and `jar` commands can be used to describe the contents of a module. The `java` command can additionally list available modules and show module resolution. The `jdeps` command prints information about packages used in addition to module-level information. The `jmod` command is used when dealing with files that don't meet the requirements for a JAR. The `jlink` command creates a smaller Java runtime image.

Türkçe: Bir modülün içeriğini açıklamak için hem `java` hem de `jar` kullanılabilir. `java` ayrıca kullanılabilir modülleri listeleyebilir ve module resolution'ı gösterebilir. `jdeps`, modül düzeyindeki bilgilere ek olarak kullanılan paketler hakkında bilgi yazdırır. `jmod`, JAR'ın karşılayamadığı dosya gereksinimleri için; `jlink` ise daha küçük bir Java runtime image oluşturmak için kullanılır.

English: There are three types of modules. Named modules contain a module-info.java file and are on the module path. They can read only from the module path. Automatic modules are also on the module path but have not yet been modularized. They might have an automatic module name set in the manifest. Unnamed modules are on the classpath.

Türkçe: Üç modül türü vardır. Named module'ler bir `module-info.java` dosyası içerir, module path üzerindedir ve yalnızca module path'ten okuyabilir. Automatic module'ler de module path üzerindedir ancak henüz modülerleştirilmemiştir; manifest içinde `Automatic-Module-Name` tanımlanmış olabilir. Unnamed module'ler classpath üzerindedir.

English: The two most common migration strategies are top-down and bottom-up migration. Top-down migration starts migrating the module with the most dependencies and places all other modules on the module path. Bottom-up migration starts migrating a module with no dependencies and moves one module to the module path at a time. Both of these strategies require ensuring that you do not have any cyclic dependencies since the Java Platform Module System will not allow cyclic dependencies to compile.

Türkçe: En yaygın iki strateji top-down ve bottom-up migration'dır. Top-down migration, en fazla bağımlılığı olan üst düzey modülden başlar ve diğer tüm modülleri module path'e yerleştirir. Bottom-up migration ise bağımlılığı olmayan bir modülden başlar ve modülleri module path'e birer birer taşır. Java Platform Module System cyclic dependency'lerin derlenmesine izin vermediğinden iki stratejide de bu bağımlılıkların giderilmesi gerekir.

Exam Essentials

English: Create module-info.java files. Place the module-info.java file in the root directory of the module. Know how to code exports, requires, provides, and uses directives. Additionally, be familiar with the opens directive.

Türkçe: `module-info.java` dosyasını modülün kök dizinine yerleştirmeyi; `exports`, `requires`, `provides`, `uses` ve `opens` directive'lerini doğru yazmayı bilin.

English: Use command-line operations with modules. The `java` command can describe a module, list available modules, or show the module resolution. The `jar` command can describe a module similar to how the `java` command does. The `jdeps` command prints details about a module and packages. The `jmod` command provides various modes for working with JMOD files rather than JAR files. The `jlink` command creates custom Java images.

Türkçe: Modüllerle ilgili command-line işlemlerini bilin. `java` bir modülü açıklayabilir, kullanılabilir modülleri listeleyebilir veya module resolution'ı gösterebilir. `jar` da benzer biçimde bir modülü açıklayabilir. `jdeps` modül ve paket ayrıntılarını yazdırır; `jmod` JMOD dosyalarıyla çalışmak için çeşitli modlar sağlar; `jlink` ise özel Java runtime image'ları oluşturur.

English: Identify the three types of modules. Named modules are JARs that have been modularized. Unnamed modules have not been modularized. Automatic modules are in between. They are on the module path but do not have a module-info.java file.

Türkçe: Üç modül türünü ayırt edin. Named module'ler modülerleştirilmiş JAR'lardır. Unnamed module'ler modülerleştirilmemiştir. İkisinin arasında yer alan automatic module'ler module path üzerindedir ancak `module-info.java` dosyaları yoktur.

English: List built-in JDK modules. The `java.base` module is available to all modules. There are about 20 other modules provided by the JDK that begin with `java.*` and about 30 that begin with `jdk.*`.

Türkçe: Yerleşik JDK modüllerini bilin. `java.base` bütün modüller tarafından kullanılabilir. JDK ayrıca adı `java.*` ile başlayan yaklaşık 20, `jdk.*` ile başlayan yaklaşık 30 modül sağlar.

English: Explain top-down and bottom-up migration. A top-down migration places all JARs on the module path, making them automatic modules while migrating from top to bottom. A bottom-up migration leaves all JARs on the classpath, making them unnamed modules while migrating from bottom to top.

Türkçe: Top-down ve bottom-up migration'ı açıklayabilin. Top-down migration bütün JAR'ları module path'e koyup automatic module haline getirerek yukarıdan aşağı ilerler. Bottom-up migration ise JAR'ları classpath üzerinde unnamed module olarak bırakıp aşağıdan yukarı ilerler.

English: Differentiate the four main parts of a service. A service provider interface declares the interface that a service must implement. The service locator looks up the service, and a consumer calls the service. Finally, a service provider implements the service.

Türkçe: Bir service'in dört ana bileşenini ayırt edin. Service provider interface, implementation'ın uyması gereken interface'i tanımlar. Service locator service'i bulur, consumer service'i çağırır, service provider ise service'i uygular.

Review Questions

English: The answers to the chapter review questions can be found in the Appendix.

Türkçe: Bölüm inceleme sorularının cevapları Ek'te bulunabilir.

English: 1. Which statement is true of the following module?

Türkçe: 1. Aşağıdaki modül için hangi ifade doğrudur?

```
|---zoo
|   |-- staff
|       |-- Vet.java
```

English: A. The directory structure shown is a valid module.

Türkçe: A. Gösterilen dizin yapısı geçerli bir modüldür.

English: B. The directory structure would be a valid module if module.java were added directly underneath zoo/staff.

Türkçe: B. `module.java` doğrudan `zoo/staff` altına eklenirse dizin yapısı geçerli bir modül olacaktır.

English: C. The directory structure would be a valid module if module.java were added directly underneath zoo.

Türkçe: C. `module.java` doğrudan `zoo` altına eklenirse dizin yapısı geçerli bir modül olacaktır.

English: D. The directory structure would be a valid module if module-info.java were added directly underneath zoo/staff.

Türkçe: D. `module-info.java` doğrudan `zoo/staff` altına eklenirse dizin yapısı geçerli bir modül olacaktır.

English: E. The directory structure would be a valid module if module-info.java were added directly underneath zoo.

Türkçe: E. `module-info.java` doğrudan `zoo` altına eklenirse dizin yapısı geçerli bir modül olacaktır.

English: F. None of these changes would make this directory structure a valid module.

Türkçe: F. Bu değişikliklerin hiçbiri dizin yapısını geçerli bir modül hâline getirmez.

English: 2. Suppose module puppy depends on module dog and module dog depends on module animal. Fill in the blank so that code in module dog can access the animal.behavior package in module animal.

Türkçe: 2. `puppy` modülünün `dog` modülüne, `dog` modülünün de `animal` modülüne bağımlı olduğunu varsayalım. `dog` içindeki kodun `animal` modülündeki `animal.behavior` paketine erişebilmesi için boşluğu doldurun.

```
module animal {  
    _____ animal.behavior;  
}
```

English: A. export

Türkçe: A. `export`

English: B. exports

Türkçe: B. `exports`

English: C. require

Türkçe: C. `require`

English: D. requires

Türkçe: D. `requires`

English: E. require transitive

Türkçe: E. `require transitive`

English: F. requires transitive

Türkçe: F. `requires transitive`

English: G. None of the above

Türkçe: G. Yukarıdakilerin hiçbiri

English: 3. Fill in the blanks so this command to run the program is correct:

Türkçe: 3. Programı çalıştıran komutun doğru olması için boşlukları doldurun:

```
java _____ zoo.animal.talks/zoo/animal/talks/Peacocks _____ modules
```

English: A. -d and -m

Türkçe: A. -d ve -m

English: B. -d and -p

Türkçe: B. -d ve -p

English: C. -m and -d

Türkçe: C. -m ve -d

English: D. -m and -p

Türkçe: D. -m ve -p

English: E. -p and -d

Türkçe: E. -p ve -d

English: F. -p and -m

Türkçe: F. -p ve -m

English: G. None of the above

Türkçe: G. Yukarıdakilerin hiçbiri

English: 4. Which of the following pairs make up a service?

Türkçe: 4. Aşağıdaki çiftlerden hangisi bir hizmet oluşturur?

English: A. Consumer and service locator

Türkçe: A. Consumer ve service locator

English: B. Consumer and service provider interface

Türkçe: B. Consumer ve service provider interface

English: C. Service locator and service provider

Türkçe: C. Service locator ve service provider

English: D. Service locator and service provider interface

Türkçe: D. Service locator ve service provider interface

English: E. Service provider and service provider interface

Türkçe: E. Service provider ve service provider interface

English: 5. A(n) _____ module is on the classpath while a(n) _____ module is on the module path. (Choose all that apply.)

Türkçe: 5. Bir _____ module classpath üzerindeyken bir _____ module, module path üzerindedir. (Tüm geçerli seçenekleri işaretleyin.)

English: A. automatic, named

Türkçe: A. `automatic`, `named`

English: B. automatic, unnamed

Türkçe: B. `automatic`, `unnamed`

English: C. named, automatic

Türkçe: C. `named`, `automatic`

English: D. named, unnamed

Türkçe: D. `named`, `unnamed`

English: E. unnamed, automatic

Türkçe: E. `unnamed`, `automatic`

English: F. unnamed, named

Türkçe: F. `unnamed`, `named`

English: G. None of the above

Türkçe: G. Yukarıdakilerin hiçbiri

English: 6. Which of the following statements are true in a `module-info.java` file? (Choose all that apply.)

Türkçe: 6. Aşağıdaki ifadelerden hangileri bir `module-info.java` dosyası için doğrudur? (Tüm geçerli seçenekleri işaretleyin.)

English: A. The `opens` directive allows the use of reflection.

Türkçe: A. `opens` directive'i reflection kullanımına izin verir.

English: B. The `opens` directive declares that an API is called.

Türkçe: B. `opens` directive bir API'nin çağrıldığını bildirir.

English: C. The `use` directive allows the use of reflection.

Türkçe: C. `use` directive reflection kullanımına izin verir.

English: D. The `use` directive declares that an API is called.

Türkçe: D. `use` directive bir API'nin çağrıldığını bildirir.

English: E. The uses directive allows the use of reflection.
Türkçe: E. `uses` directive reflection kullanımına izin verir.

English: F. The uses directive declares that an API is called.
Türkçe: F. `uses` directive bir API'nin çağrıldığını bildirir.

English: 7. An automatic module name is generated if one is not supplied. Which of the following JAR filenames and generated automatic module name pairs are correct? (Choose all that apply.)
Türkçe: 7. Açıkça belirtilmemişse automatic module adı otomatik üretilir. Aşağıdaki JAR dosya adı-automatic module adı çiftlerinden hangileri doğrudur? (Tüm geçerli seçenekleri işaretleyin.)

English: A. `emily-1.0.0.jar` and `emily`
Türkçe: A. `emily-1.0.0.jar` ve `emily`

English: B. `emily-1.0.0-SNAPSHOT.jar` and `emily`
Türkçe: B. `emily-1.0.0-SNAPSHOT.jar` ve `emily`

English: C. `emily_the_cat-1.0.0.jar` and `emily_the_cat`
Türkçe: C. `emily_the_cat-1.0.0.jar` ve `emily_the_cat`

English: D. `emily_the_cat-1.0.0.jar` and `emily-the-cat`
Türkçe: D. `emily_the_cat-1.0.0.jar` ve `emily-the-cat`

English: E. `emily$.jar` and `emily`
Türkçe: E. `emily$.jar` ve `emily`

English: F. `emily$.jar` and `emily.`
Türkçe: F. `emily$.jar` ve `emily.`

English: G. `emily$.jar` and `emily..`
Türkçe: G. `emily$.jar` ve `emily..`

English: 8. Which of the following statements are true? (Choose all that apply.)
Türkçe: 8. Aşağıdaki ifadelerden hangisi doğrudur? (Tüm geçerli olanları seçin.)

English: A. Modules with cyclic dependencies will not compile.
Türkçe: A. Cyclic dependency içeren module'ler derlenmez.

English: B. Packages with a cyclic dependency will not compile.
Türkçe: B. Cyclic dependency içeren package'lar derlenmez.

English: C. A cyclic dependency always involves exactly two modules.
Türkçe: C. Cyclic dependency her zaman tam olarak iki module içerir.

English: D. A cyclic dependency always involves at least two requires statements.
Türkçe: D. Cyclic dependency her zaman en az iki `requires` statement içerir.

English: E. An unnamed module can be involved in a cyclic dependency with an automatic module.
Türkçe: E. Unnamed module, automatic module ile cyclic dependency içinde olabilir.

English: 9. Suppose you are creating a service provider that contains the following class. Which line of code needs to be in your module-info.java?
Türkçe: 9. Aşağıdaki sınıfı içeren bir service provider oluşturduğunuzu varsayalım. `module-info.java` içinde hangi kod satırı bulunmalıdır?

```
package dragon;
import magic.*;
public class Dragon implements Magic {
    public String getPower() {
        return "breathe fire";
    }
}
```

English: A. provides dragon.Dragon by magic.Magic;
Türkçe: A. `provides dragon.Dragon by magic.Magic;`

English: B. provides dragon.Dragon using magic.Magic;
Türkçe: B. `provides dragon.Dragon using magic.Magic;`

English: C. provides dragon.Dragon with magic.Magic;
Türkçe: C. `provides dragon.Dragon with magic.Magic;`

English: D. provides magic.Magic by dragon.Dragon;
Türkçe: D. `provides magic.Magic by dragon.Dragon;`

English: E. provides magic.Magic using dragon.Dragon;
Türkçe: E. `provides magic.Magic using dragon.Dragon;`

English: F. provides magic.Magic with dragon.Dragon;
Türkçe: F. `provides magic.Magic with dragon.Dragon;`

English: 10. What is true of a module containing a file named module-info.java with the following contents? (Choose all that apply.)
Türkçe: 10. Aşağıdaki içeriğe sahip `module-info.java` dosyasını barındıran bir modül için hangileri doğrudur? (Tüm geçerli seçenekleri işaretleyin.)

```
module com.food.supplier {}
```

English: A. All packages inside the module are automatically exported.

Türkçe: A. Modül içindeki tüm paketler otomatik olarak dışa aktarılır.

English: B. No packages inside the module are automatically exported.

Türkçe: B. Module içindeki hiçbir package otomatik export edilmez.

English: C. A main method inside the module can be run.

Türkçe: C. Module içindeki bir `main()` method çalıştırılabilir.

English: D. A main method inside the module cannot be run since the class is not exposed.

Türkçe: D. Sınıf dışarı açılmadığı için modül içindeki bir `main()` metodu çalıştırılmaz.

English: E. The module-info.java file contains a compiler error.

Türkçe: E. `module-info.java` dosyası bir derleme hatası içerir.

English: F. The module-info.java filename is incorrect.

Türkçe: F. `module-info.java` filename'i yanlıştır.

English: 11. Suppose module puppy depends on module dog and module dog depends on module animal. Which lines allow module puppy to access the animal.behavior package in module animal? (Choose all that apply.)

Türkçe: 11. `puppy` modülünün `dog` modülüne, `dog` modülünün de `animal` modülüne bağımlı olduğunu varsayalım. Hangi satırlar `puppy` modülünün `animal` modülündeki `animal.behavior` paketine erişmesini sağlar? (Tüm geçerli seçenekleri işaretleyin.)

```
module animal {
  exports animal.behavior;
}
module dog {
  _____ animal; // line S
}
module puppy {
  _____ dog; // line T
}
```

English: A. require on line S

Türkçe: A. Line S üzerinde `require`

English: B. require on line T

Türkçe: B. Line T üzerinde `require`

English: C. requires on line S

Türkçe: C. Line S üzerinde `requires`

English: D. requires on line T

Türkçe: D. Line T üzerinde *requires*

English: E. require transitive on line S

Türkçe: E. Line S üzerinde *require transitive*

English: F. require transitive on line T

Türkçe: F. Line T üzerinde *require transitive*

English: G. requires transitive on line S

Türkçe: G. Line S üzerinde *requires transitive*

English: H. requires transitive on line T

Türkçe: H. Line T üzerinde *requires transitive*

English: 12. Which of the following modules are provided by the JDK? (Choose all that apply.)

Türkçe: 12. Aşağıdaki modüllerden hangisi JDK tarafından sağlanır? (Tüm geçerli olanları seçin.)

English: A. java.base

Türkçe: A. java.base

English: B. java.desktop

Türkçe: B. *java.desktop*

English: C. java.logging

Türkçe: C. *java.logging*

English: D. java.util

Türkçe: D. java.util

English: E. jdk.base

Türkçe: E. *jdk.base*

English: F. jdk.compiler

Türkçe: F. *jdk.compiler*

English: G. jdk.xerces

Türkçe: G. jdk.xerces

English: 13. Which of the following compiles and is equivalent to this loop?

Türkçe: 13. Aşağıdakilerden hangisi derlenir ve bu loop'a eşdeğerdir?

```
List<Unicorn> all = new ArrayList<>();
for (Unicorn current: ServiceLoader.load(Unicorn.class))
    all.add(current);
```

English: A.

Türkçe: A.

```
List<Unicorn> all = ServiceLoader.load(Unicorn.class)
    .getStream()
    .collect(Collectors.toList());
```

English: B.

Türkçe: B.

```
List<Unicorn> all = ServiceLoader.load(Unicorn.class)
    .stream()
    .collect(Collectors.toList());
```

English: C.

Türkçe: C.

```
List<Unicorn> all = ServiceLoader.load(Unicorn.class)
    .getStream()
    .map(Provider::get)
    .collect(Collectors.toList());
```

English: D.

Türkçe: D.

```
List<Unicorn> all = ServiceLoader.load(Unicorn.class)
    .stream()
    .map(Provider::get)
    .collect(Collectors.toList());
```

English: E. None of the above

Türkçe: E. Yukarıdakilerin hiçbirisi

English: 14. Which of the following are legal commands to run a modular program where n is the module name and c is the fully qualified class name? (Choose all that apply.)

Türkçe: 14. n modül adı, c fully qualified class name olmak üzere aşağıdakilerden hangileri modüler bir programı çalıştırmak için geçerli komutlardır? (Tüm geçerli seçenekleri işaretleyin.)

English: A. java --module-path x -m n.c

Türkçe: A. `java --module-path x -m n.c`

English: B. `java --module-path x -p n.c`

Türkçe: B. `java --module-path x -p n.c`

English: C. `java --module-path x-x -m n/c`

Türkçe: C. `java --module-path x-x -m n/c`

English: D. `java --module-path x -p n/c`

Türkçe: D. `java --module-path x -p n/c`

English: E. `java --module-path x-x -m n-c`

Türkçe: E. `java --module-path x-x -m n-c`

English: F. `java --module-path x -p n-c`

Türkçe: F. `java --module-path x -p n-c`

English: G. None of the above

Türkçe: G. Yukarıdakilerin hiçbiri

English: 15. For a top-down migration, all modules other than named modules are _____ modules and are on the _____.

Türkçe: 15. Top-down migration sırasında named module dışındaki bütün modüller _____ module'dür ve _____ üzerinde bulunur.

English: A. automatic, classpath

Türkçe: A. `automatic, classpath`

English: B. automatic, module path

Türkçe: B. `automatic, module path`

English: C. unnamed, classpath

Türkçe: C. `unnamed, classpath`

English: D. unnamed, module path

Türkçe: D. `unnamed, module path`

English: E. None of the above

Türkçe: E. Yukarıdakilerin hiçbiri

English: 16. Suppose you have separate modules for a service provider interface, service provider, service locator, and consumer. If you add a second service provider module, how many of these modules do you need to recompile?

Türkçe: 16. Service provider interface, service provider, service locator ve consumer için ayrı modüllerinin olduğunu varsayalım. İkinci bir service provider modülü eklerseniz, bu modüllerden kaç tanesinin yeniden derlenmesi gerekir?

English: A. Zero

Türkçe: A. Sıfır

English: B. One

Türkçe: B. Bir

English: C. Two

Türkçe: C. İki

English: D. Three

Türkçe: D. Üç.

English: E. Four

Türkçe: E. Dört

English: 17. Suppose we have a JAR file named `cat-1.2.3-RC1.jar`, and `Automatic-Module-Name` in the `MANIFEST.MF` is set to `dog`. What should an unnamed module referencing this automatic module include in `module-info.java`?

Türkçe: 17. `cat-1.2.3-RC1.jar` adlı JAR'ın `MANIFEST.MF` dosyasında `Automatic-Module-Name` değeri `dog` olarak ayarlanmıştır. Bu automatic module'e başvuran bir unnamed module, `module-info.java` içinde ne bulundurmalıdır?

English: A. `requires cat;`

Türkçe: A. `requires cat;`

English: B. `requires cat.RC;`

Türkçe: B. `requires cat.RC;`

English: C. `requires cat-RC;`

Türkçe: C. `requires cat-RC;`

English: D. `requires dog;`

Türkçe: D. `requires dog;`

English: E. None of the above

Türkçe: E. Yukarıdakilerin hiçbirisi

English: 18. Which commands are used to create a smaller Java image and work with native code, respectively?

Türkçe: 18. Sırasıyla daha küçük bir Java runtime image oluşturmak ve native code ile çalışmak için hangi komutlar kullanılır?

English: A. jimage and jlink

Türkçe: A. jimage ve jlink

English: B. jimage and jmod

Türkçe: B. jimage ve jmod

English: C. jlink and jimage

Türkçe: C. jlink ve jimage

English: D. jlink and jmod

Türkçe: D. jlink ve jmod

English: E. jmod and jimage

Türkçe: E. jmod ve jimage

English: F. jmod and jmod

Türkçe: F. jmod ve jmod

English: 19. Which are true statements about the following module? (Choose all that apply.)

Türkçe: 19. Aşağıdaki modülle ilgili doğru ifadeler nelerdir? (Tüm geçerli olanları seçin.)

```
class dragon {  
  exports com.dragon.fire;  
  exports com.dragon.scales to castle;  
}
```

English: A. All modules can reference the com.dragon.fire package.

Türkçe: A. Tüm modüller com.dragon.fire paketine referans verebilir.

English: B. All modules can reference the com.dragon.scales package.

Türkçe: B. Bütün module'ler com.dragon.scales package'ına reference verebilir.

English: C. Only the castle module can reference the com.dragon.fire package.

Türkçe: C. Yalnız castle module'ü com.dragon.fire package'ına reference verebilir.

English: D. Only the castle module can reference the com.dragon.scales package.

Türkçe: D. Yalnız castle module'ü com.dragon.scales package'ına reference verebilir.

English: E. None of the above
Türkçe: E. Yukarıdakilerin hiçbirisi

English: 20. Which would you expect to see when describing any module?
Türkçe: 20. Herhangi bir modülü açıklayan çıktıda hangisini görmeyi beklersiniz?

English: A. requires java.base mandated
Türkçe: A. `requires java.base mandated`

English: B. requires java.core mandated
Türkçe: B. `requires java.core mandated`

English: C. requires java.lang mandated
Türkçe: C. `requires java.lang mandated`

English: D. requires mandated java.base
Türkçe: D. `requires mandated java.base`

English: E. requires mandated java.core
Türkçe: E. `requires mandated java.core`

English: F. requires mandated java.lang
Türkçe: F. `requires mandated java.lang`

English: G. None of the above
Türkçe: G. Yukarıdakilerin hiçbirisi

English: 21. Suppose you have separate modules for a service provider interface, service provider, service locator, and consumer. Which module(s) need to specify a requires directive on the service provider?
Türkçe: 21. Service provider interface, service provider, service locator ve consumer için ayrı module'ler bulunduğunu varsayalım. Hangi module veya module'ler service provider implementation için `requires` directive belirtmelidir?

English: A. Service locator
Türkçe: A. Service locator

English: B. Service provider interface
Türkçe: B. Service provider interface

English: C. Consumer
Türkçe: C. Consumer

English: D. Consumer and service locator
Türkçe: D. Consumer ve service locator

English: E. Consumer and service provider
Türkçe: E. Consumer ve service provider

English: F. Service locator and service provider interface
Türkçe: F. Service locator ve service provider interface

English: G. Consumer, service locator, and service provider interface
Türkçe: G. Consumer, service locator ve service provider interface

English: H. None of the above
Türkçe: H. Yukarıdakilerin hiçbirisi

English: 22. Which are true statements? (Choose all that apply.)
Türkçe: 22. Hangileri doğru ifadelerdir? (Tüm geçerli olanları seçin.)

English: A. An automatic module exports all packages to named modules.
Türkçe: A. Automatic module bütün package'larını named module'lere export eder.

English: B. An automatic module exports only the specified packages to named modules.
Türkçe: B. Automatic module yalnız belirtilen package'ları named module'lere export eder.

English: C. An automatic module exports no packages to named modules.
Türkçe: C. Automatic module named module'lere hiçbir package export etmez.

English: D. An unnamed module exports only the named packages to named modules.
Türkçe: D. Unnamed module yalnız named package'ları named module'lere export eder.

English: E. An unnamed module exports all packages to named modules.
Türkçe: E. Unnamed module bütün package'larını named module'lere export eder.

English: F. An unnamed module exports no packages to named modules.
Türkçe: F. Unnamed module named module'lere hiçbir package export etmez.

English: 23. Which is the first line to contain a compiler error?
Türkçe: 23. Compile-time error içeren ilk satır hangisidir?

```
module snake {                                // line 1
  exports com.snake.tail;                     // line 2
  exports com.snake.fangs to bird;           // line 3
  requires skin;                             // line 4
  requires transitive skin;                  // line 5
}
```

English: A. Line 1

Türkçe: A. Line 1

English: B. Line 2

Türkçe: B. Line 2

English: C. Line 3

Türkçe: C. Line 3

English: D. Line 4

Türkçe: D. Line 4

English: E. Line 5

Türkçe: E. Line 5

English: F. The code does not contain any compiler errors.

Türkçe: F. Kod hiçbir derleme hatası içermez.

English: 24. Which are true statements about a package in a JAR on the classpath containing a module-info.java file? (Choose all that apply.)

Türkçe: 24. Classpath üzerinde bulunan ve `module-info.java` dosyası içeren bir JAR'daki paket hakkında hangileri doğrudur? (Tüm geçerli seçenekleri işaretleyin.)

English: A. It is possible to make the package available to all other modules on the classpath.

Türkçe: A. Paketi classpath üzerindeki diğer tüm modüllere kullanılabilir hale getirmek mümkündür.

English: B. It is possible to make the package available to all other modules on the module path.

Türkçe: B. Paketi module path üzerindeki diğer tüm modüllere açmak mümkündür.

English: C. It is possible to make the package available to exactly one other specific module on the classpath.

Türkçe: C. Paketi classpath üzerindeki yalnızca belirli bir modüle açmak mümkündür.

English: D. It is possible to make the package available to exactly one other specific module on the module path.

Türkçe: D. Paketi module path üzerindeki yalnızca belirli bir modüle açmak mümkündür.

English: E. It is possible to make sure the package is not available to any other modules on the classpath.

Türkçe: E. Paketin classpath üzerindeki hiçbir modüle açık olmamasını sağlamak mümkündür.

English: 25. Suppose you have separate modules for a service provider interface, service provider, service locator, and consumer. Which statements are true about the directives you need to specify?

Türkçe: 25. Service provider interface, service provider, service locator ve consumer için ayrı modülleriniz olduğunu varsayalım. Belirtmeniz gereken directive'lerle ilgili hangi ifadeler doğrudur?

English: (Choose all that apply.)

Türkçe: (Tüm geçerli olanları seçin.)

English: A. The consumer must use the requires directive.

Türkçe: A. Consumer, **requires** directive'ini kullanmalıdır.

English: B. The consumer must use the uses directive.

Türkçe: B. Consumer, **uses** directive'ini kullanmalıdır.

English: C. The service locator must use the requires directive.

Türkçe: C. Service locator, **requires** directive'ini kullanmalıdır.

English: D. The service locator must use the uses directive.

Türkçe: D. Service locator, **uses** directive'ini kullanmalıdır.

English: E. None of the above

Türkçe: E. Yukarıdakilerin hiçbiri

Appendix · Official Review Question Answers / Resmî Cevaplar

Aşağıdaki cevaplar kaynak Appendix bölümündeki sıra ve gerekçeleri korur. Türkçe bloklar doğal teknik çeviridir.

Official Answer 1

English: 1. E. Modules are required to have a module-info.java file at the root directory of the module. Option E matches this requirement.

Türkçe: 1. E. Her modülün kök dizininde bir **module-info.java** dosyası bulunmalıdır. E seçeneği bu gerekliliği karşılar.

Official Answer 2

English: 2. B. Options A, C, and E are incorrect because they refer to directives that don't exist. The `exports` directive is used when allowing a package to be called by code outside of the module, making option B the correct answer. Notice that options D and F are incorrect because of `requires`.

Türkçe: 2. B. A, C ve E seçenekleri, var olmayan directive'leri kullandıkları için yanlıştır. Bir paketin modül dışındaki kod tarafından kullanılmasına izin vermek için `exports` directive'i kullanılır; dolayısıyla doğru cevap B'dir. D ve F seçenekleri ise `requires` kullanımından dolayı yanlıştır.

Official Answer 3

English: 3. G. The `-m` or `--module` option is used to specify the module and class name. The `-p` or `--module-path` option is used to specify the location of the modules. Option D would be correct if the rest of the command were correct. However, running a program requires specifying the package name with periods (.) instead of slashes. Since the command is incorrect, option G is correct.

Türkçe: 3. G. Modül ve sınıf adını belirtmek için `-m` veya `--module`, modüllerin konumunu belirtmek için `-p` veya `--module-path` seçeneği kullanılır. Komutun geri kalanı doğru olsaydı D seçeneği doğru olurdu. Ancak bir program çalıştırılırken paket adı eğik çizgilerle (/) değil, noktalarla (.) yazılmalıdır. Komut hatalı olduğundan doğru cevap G'dir.

Java 17 editör notu: G sonucu doğrudur, fakat D yalnız slash'lar dot'a çevrilerek geçerli olmaz. `-p modules` launcher option'ı launch target'tan önce gelmelidir; doğru genel biçim `java -p modules -m zoo.animal.talks/zoo.animal.talks.Peacocks` olur.

Official Answer 4

English: 4. D. A service consists of the service provider interface and logic to look up implementations using a service locator. This makes option D correct. Make sure you know that the service provider itself is the implementation, which is not considered part of the service.

Türkçe: 4. D. Bir service; service provider interface ile implementation'ları bir service locator aracılığıyla bulma mantığından oluşur. Bu nedenle D seçeneği doğrudur. Service provider'ın implementation'ın kendisi olduğunu ve service'in bir parçası sayılmadığını unutmayın.

Official Answer 5

English: 5. E, F. Automatic modules are on the module path but do not have a `module-info.java` file. Named modules are on the module path and do have a `module-info`. Unnamed modules are on the classpath. Therefore, options E and F are correct.

Türkçe: 5. E, F. Automatic module'ler module path üzerinde bulunur ancak `module-info.java` dosyaları yoktur. Named module'ler module path üzerinde bulunur ve `module-info.java` dosyaları vardır. Unnamed module'ler ise classpath üzerindedir. Dolayısıyla E ve F seçenekleri doğrudur.

Official Answer 6

English: 6. A, F. Options C and D are incorrect because there is no use directive. Options A and F are correct because opens is for reflection and uses declares that an API consumes a service.

Türkçe: 6. A, F. `use` adlı bir directive bulunmadığından C ve D yanlıştır. `opens` reflection için kullanılır; `uses` ise bir API'nin bir service tükettiğini bildirir. Bu nedenle A ve F doğrudur.

Official Answer 7

English: 7. A, B, E. Any version information at the end of the JAR filename is removed, making options A and B correct. Underscores (_) are turned into dots (.), making options C and D incorrect.

Türkçe: 7. A, B, E. JAR dosya adının sonundaki sürüm bilgisi kaldırılır; bu nedenle A ve B doğrudur. Alt çizgiler (_) noktaya (.) dönüştürüldüğünden C ve D yanlıştır.

English: Other special characters like a dollar sign (\$) are also turned into dots. However, adjacent dots are merged, and leading/trailing dots are removed. Therefore, option E is correct.

Türkçe: Dolar işareti (\$) gibi diğer özel karakterler de noktaya dönüştürülür. Ardışık noktalar birleştirilir; baştaki ve sondaki noktalar kaldırılır. Bu nedenle E de doğrudur.

Official Answer 8

English: 8. A, D. A cyclic dependency is when a module graph forms a circle. Option A is correct because the Java Platform Module System does not allow cyclic dependencies between modules. No such restriction exists for packages, making option B incorrect. A cyclic dependency can involve two or more modules that require each other, making option D correct, while option C is incorrect. Finally, option E is incorrect because unnamed modules cannot be referenced from an automatic module.

Türkçe: 8. A, D. Cyclic dependency (döngüsel bağımlılık), module graph'ın bir çevrim oluşturmaktır. Java Platform Module System modüller arasında döngüsel bağımlılığa izin vermediğinden A doğrudur. Paketler için böyle bir kısıtlama bulunmadığından B yanlıştır. Bir döngüsel bağımlılık birbirini `requires` eden iki veya daha fazla modülü kapsayabilir; dolayısıyla D doğru, C yanlıştır. Ayrıca automatic module'ler unnamed module'lere başvuramaz; bu nedenle E de yanlıştır.

Java 17 editör notu: A ve D sonucu değişmez; E de sorudaki explicit `requires-cycle` anlamında yanlıştır. Ancak kaynağın “automatic module unnamed module'e başvuramaz” gerekçesi doğru değildir: Automatic module, runtime'da JVM'deki unnamed module'leri okur. Unnamed module'ün descriptor'ı ve `requires` directive'i olmadığı için resolution'daki named-module `requires cycle`'inin tarafı olamaz.

Official Answer 9

English: 9. F. The provides directive takes the interface name first and the implementing class name second and also uses with. Only option F meets these two criteria, making it the correct answer.

Türkçe: 9. F. `provides` directive'inde önce interface adı, sonra implementation sınıfının adı yazılır ve aralarında `with` kullanılır. Bu iki koşulu yalnızca F karşıladığı için doğru cevap F'dir.

Official Answer 10

English: 10. B, C. Packages inside a module are not exported by default, making option B correct and option A incorrect. Exporting is necessary for other code to use the packages; it is not necessary to call the `main()` method at the command line, making option C correct and option D incorrect. The `module-info.java` file has the correct name and compiles, making options E and F incorrect.

Türkçe: 10. B, C. Bir modülün paketleri varsayılan olarak export edilmez; bu nedenle B doğru, A yanlıştır. Başka kodların bu paketleri kullanabilmesi için export gerekir, ancak `main()` metodunu komut satırından çağırmak için gerekmez; dolayısıyla C doğru, D yanlıştır. `module-info.java` dosyasının adı doğrudur ve dosya derlenir; bu yüzden E ve F de yanlıştır.

Official Answer 11

English: 11. D, G, H. Options A, B, E, and F are incorrect because they refer to directives that don't exist.

Türkçe: 11. D, G, H. A, B, E ve F seçenekleri var olmayan directive'leri kullandıkları için yanlıştır.

English: The `requires transitive` directive is used when specifying a module to be used by the requesting module and any other modules that use the requesting module. Therefore, `dog` needs to specify the transitive relationship, and option G is correct. The module `puppy` just needs `requires dog`, and it gets the transitive dependencies, making option D correct.

Türkçe: `requires transitive`, bir modülün bağımlılığını hem kendisinin hem de kendisine bağımlı diğer modüllerin okuyabilmesini sağlar. Bu nedenle `dog` modülü transitive ilişkiyi belirtmelidir ve G doğrudur. `puppy` modülünün yalnızca `requires dog` yazması yeterlidir; `dog` üzerinden transitive bağımlılıkları da edinir. Dolayısıyla D de doğrudur.

English: However, `requires transitive` does everything `requires` does and more, which makes option H the final answer.

Türkçe: Ayrıca `requires transitive`, `requires`ın yaptığı her şeyi ve daha fazlasını yaptığı için H de doğrudur.

Official Answer 12

English: 12. A, B, C, F. Option D is incorrect because it is a package name rather than a module name.

Türkçe: 12. A, B, C, F. D seçeneği bir modül adı değil paket adı olduğundan yanlıştır.

English: Option E is incorrect because `java.base` is the module name, not `jdk.base`. Option G is wrong because we made it up. Options A, B, C, and F are correct.

Türkçe: E yanlıştır; doğru modül adı `jdk.base` değil `java.base`dir. G'deki ad ise uydurmadır. A, B, C ve F doğrudur.

Official Answer 13

English: 13. D. There is no `getStream()` method on a `ServiceLoader`, making options A and C incorrect. Option B does not compile because the `stream()` method returns a list of `Provider` interfaces and needs to be converted to the `Unicorn` interface we are interested in. Therefore, option D is correct.

Türkçe: 13. D. `ServiceLoader` sınıfında `getStream()` metodu bulunmadığından A ve C yanlıştır. B derlenmez; çünkü `stream()` bir `Stream<ServiceLoader.Provider<Unicorn>>` döndürür ve öğelerin ilgilendiğimiz `Unicorn` türüne, örneğin `Provider::get` ile, dönüştürülmesi gerekir. Dolayısıyla doğru cevap D'dir.

Official Answer 14

English: 14. C. The `-p` option is a shorter form of `--module-path`. Since the same option cannot be specified twice, options B, D, and F are incorrect. The `--module-path` option is an alternate form of `-p`. The module name and class name are separated with a slash, making option C the answer. Note that `x-x` is legal because the module path is a folder name, so dashes are allowed.

Türkçe: 14. C. `-p`, `--module-path` seçeneğinin kısa biçimidir. Aynı seçenek iki kez belirtilemeyeceğinden B, D ve F yanlıştır. `--module-path` de `-p`'nin uzun biçimidir. Modül adı ile sınıf adı eğik çizgiyle (/) ayrıldığı için doğru cevap C'dir. `x-x` bir dizin adı olduğundan içindeki tire yasalıdır.

Java 17 editör notu: C sonucu doğrudur; ancak Java launcher, `--module-path/-p` option'ının tekrarını genel olarak yasaklamaz. B, D ve F'nin asıl sorunu ikinci `-p` kullanımının da `module-path` option'ı sayılması ve komutta `-m module/class` launch target'ının hiç bulunmamasıdır.

Official Answer 15

English: 15. B. A top-down migration strategy first places all JARs on the module path. Then it migrates the top-level module to be a named module, leaving the other modules as automatic modules.

Türkçe: 15. B. Top-down migration stratejisinde önce tüm JAR'lar module path'e yerleştirilir. Ardından en üst düzey modül named module'e dönüştürülür; diğerleri automatic module olarak kalır.

English: Option B is correct as it matches both of those characteristics.

Türkçe: B seçeneği, bu özelliklerin her ikisine de uyduğu için doğrudur.

Official Answer 16

English: 16. A. Since this is a new module, you need to compile it. However, none of the existing modules needs to be recompiled, making option A correct. The service locator will see the new service provider simply by having that new service provider on the module path.

Türkçe: 16. A. Yeni modülün derlenmesi gerekir; ancak mevcut modüllerden hiçbirinin yeniden derlenmesine gerek yoktur. Bu nedenle A doğrudur. Yeni service provider module path'e eklendiğinde service locator onu doğrudan bulabilir.

Official Answer 17

English: 17. E. Trick question! An unnamed module doesn't use a module-info.java file. Therefore, option E is correct. An unnamed module can access an automatic module. The unnamed module would simply treat the automatic module as a regular JAR without involving the module.info file.

Türkçe: 17. E. Bu bir tuzak sorudur: unnamed module, `module-info.java` dosyası kullanmaz. Bu nedenle E doğrudur. Unnamed module bir automatic module'e erişebilir; automatic module'ü, modül tanımını dikkate almadan normal bir JAR gibi ele alır.

Official Answer 18

English: 18. D. The jlink command creates a directory with a smaller Java runtime containing just what is needed. The JMOD format is for native code. Therefore, option D is correct.

Türkçe: 18. D. `jlink`, yalnızca gereken bileşenleri içeren daha küçük bir Java runtime'ın bulunduğu dizini oluşturur. JMOD biçimi native code içindir. Bu nedenle doğru cevap D'dir.

Official Answer 19

English: 19. E. There is a trick here. A module definition uses the keyword module rather than class.

Türkçe: 19. E. Buradaki tuzak şudur: Bir modül tanımında `class` değil `module` anahtar kelimesi kullanılır.

English: Since the code does not compile, option E is correct. If the code did compile, options A and D would be correct.

Türkçe: Kod derlenmediğinden E doğrudur. Kod derlenebilseydi A ve D doğru olurdu.

Official Answer 20

English: 20. A. When running java with the -d option, all the required modules are listed. Additionally, the java.base module is listed since it is included automatically. The line ends with mandated, making option A correct. The java.lang is a trick since it is a package that is imported by default in a class rather than a module.

Türkçe: 20. A. `java` komutu `-d` seçeneğiyle çalıştırıldığında gereken tüm modüller listelenir. Otomatik olarak dahil edilen `java.base` da listede yer alır ve ilgili satır `mandated` sözcüğüyle biter; dolayısıyla A doğrudur. `java.lang` ise bir modül değil, sınıflara varsayılan olarak import edilen bir pakettir.

Java 17 editör notu: Kaynak sorudaki "any module" ifadesinin bir istisnası vardır: `java --describe-module java.base`, `java.base` modülünün kendi çıktısında `requires java.base` `mandated` göstermez. A seçeneği, `java.base` dışındaki ordinary named module'ler için beklenen satırdır.

Official Answer 21

English: 21. H. This question is tricky. The service locator must have a `uses` directive, but that is on the service provider interface. No modules need to specify `requires` on the service provider since that is the implementation. Since none are correct, option H is the answer.

Türkçe: 21. H. Bu bir tuzak sorudur. Service locator'da, service provider interface'i belirten bir `uses` directive'i bulunmalıdır. Service provider yalnızca implementation olduğundan hiçbir modülün onu `requires` ile belirtmesi gerekmez. Seçeneklerin hiçbirisi doğru olmadığı için cevap H'dir.

Official Answer 22

English: 22. A, F. An automatic module exports all packages, making option A correct. An unnamed module is not available to any modules on the module path. Therefore, it doesn't export any packages, and option F is correct.

Türkçe: 22. A, F. Automatic module bütün paketlerini export eder; bu nedenle A doğrudur. Unnamed module, module path üzerindeki hiçbir modül tarafından okunamaz. Bu bakımdan herhangi bir paketini bu modüllere export etmez; dolayısıyla F de doğrudur.

Java 17 teknik düzeltme: Yukarıdaki A, F kaynağın resmî anahtarıdır; formal Java 17 semantics altında doğru küme **A ve E** olarak değerlendirilmelidir. JLS 17 §7.7.5, unnamed module'ün ilişkili bütün package'ları her module'e export ve open ettiğini belirtir. Ordinary explicit named module'ün bunlara genellikle erişememesi export eksikliğinden değil, unnamed module'e yönelik readability edge'inin bulunmamasındandır. Kaynak anahtarı, kaynak sadakati için hemen üstte değiştirilmeden korunmuştur.

Official Answer 23

English: 23. E. The module name is valid, as are the exports statements. Lines 4 and 5 are tricky because each is valid independently. However, the same module name is not allowed to be used in two `requires` statements. The second one fails to compile on line 5, making option E the answer.

Türkçe: 23. E. Modül adı ve `exports` ifadeleri geçerlidir. 4. ve 5. satırların her biri tek başına geçerli görünse de aynı modül adı iki ayrı `requires` ifadesinde kullanılamaz. İkinci kullanım olan 5. satır derlenmez; doğru cevap E'dir.

Official Answer 24

English: 24. A. Since the JAR is on the classpath, it is treated as a regular unnamed module even though it has a `module-info.java` file inside. Remember from learning about top-down migration that modules on the module path are not allowed to refer to the classpath, making options B and D incorrect. The classpath does not have a facility to restrict packages, making option A correct and options C and E incorrect.

Türkçe: 24. A. JAR classpath üzerinde bulunduğu için, içinde `module-info.java` olsa bile normal bir unnamed module olarak değerlendirilir. Top-down migration konusunda görüldüğü üzere module path üzerindeki modüller classpath'e başvuramaz; bu nedenle B ve D yanlıştır. Classpath'in paket erişimini kısıtlayan bir mekanizması yoktur; dolayısıyla A doğru, C ve E yanlıştır.

Official Answer 25

English: 25. A, C, D. Options A and C are correct because both the consumer and the service locator depend on the service provider interface. Additionally, option D is correct because the service locator must specify that it uses the service provider interface to look it up.

Türkçe: 25. A, C, D. Hem consumer hem de service locator, service provider interface'e bağımlı olduğundan A ve C doğrudur. Ayrıca service locator, arama yapacağı service provider interface'i **uses** ile belirtmelidir; bu nedenle D de doğrudur.

Coverage ledger

- Chapter body marker'ları: 661–720
- Appendix answer marker'ları: 949–951
- Resmî cevap hedefi: 1–25
- Kod blokları özgün dilinde tutulmuştur.
- Çeviri ayrıntıları ünite vocabulary ve grammar kaynaklarıyla desteklenir.